

The Australian National University
2600 ACT | Canberra | Australia



Australian
National
University

School of Computing

College of Engineering, Computing
and Cybernetics (CECC)

Beyond Stochastic Selection: Guiding Musical Predictions with Monte Carlo Tree Search

— Honours project (S2/S1 2024–2025)

A thesis submitted for the degree

Bachelor of Advanced Computing (Research & Development) (Honours)

By:

Scott Howsam

Supervisor:

Charles Martin

May 2025

Declaration:

I declare that this work:

- upholds the principles of academic integrity, as defined in the [University Academic Misconduct Rules](#);
- is original, except where collaboration (for example group work) has been authorised in writing by the course convener in the class summary and/or Wattle site;
- is produced for the purposes of this assessment task and has not been submitted for assessment in any other context, except where authorised in writing by the course convener;
- gives appropriate acknowledgement of the ideas, scholarship and intellectual property of others insofar as these have been used;
- in no part involves copying, cheating, collusion, fabrication, plagiarism or recycling.

May, Scott Howsam

Abstract

Interactive musical prediction systems are a significant subfield of New Interfaces for Musical Expression (NIME). These systems can perform collaboratively in a call-and-response manner with a user, with a neural network taking over from a human performer in real-time and predicting musical continuations. For musical predictions, there is never one correct answer for what the next note should be, and so neural networks usually rely on random sampling from a distribution of note options to generate a prediction. However, this random sampling is entirely arbitrary, and does not take into account how well each potential note fits into the context of the musical piece being played. This can lead to a lack of musical coherence in generated predictions. In contrast, older generative music techniques often used rule-based approaches, which were excellent at producing coherent music which adheres to musical rules, but lacked the expressiveness and creativity of modern machine learning based systems. This thesis introduces a novel musical prediction architecture which combines the strengths of both approaches, by using repeated neural network predictions to build a Monte Carlo search tree of viable musical continuations, and guiding the search with heuristics inspired by rules-based techniques. The evaluation measures improvements in prediction accuracy of 67.5% and 145.2% for two datasets, and qualitative analysis of improvisations performed with the system showed that this increase did not come at the cost of creativity, with the expressiveness and playability of the system being significantly improved by using this search based prediction architecture. This thesis represents a significant advancement for musical prediction systems and human-computer interaction more broadly, acting as a proof of concept for the prediction architecture developed in this thesis, and leading to future work possibilities for many domains where search based and machine learning based methods could be combined.

Table of Contents

1	Introduction	1
1.1	Methodology	3
1.2	Research Questions	5
1.3	Thesis Structure	5
2	Background	7
2.1	New Interfaces for Musical Expression (NIMEs)	8
2.1.1	Controller Mappings	8
2.1.2	Collaborative Networked Instruments	10
2.1.3	Interactive Musical Prediction Systems	10
2.2	Machine Learning in Music	11
2.2.1	Markov Models	11
2.2.2	Feedforward Neural Networks	14
2.2.3	Recurrent Neural Networks	15
2.2.4	Long Short Term Memory Networks	16
2.2.5	Mixture Density Recurrent Neural Networks	18
2.3	Heuristic-Based AI in Music	19
2.3.1	Rule Based Systems	20
2.3.2	Search Based Systems	21
2.4	Tree Search Algorithms	22
2.4.1	Beam Search	22
2.4.2	Monte Carlo Tree Search	23
2.4.3	Continuous Monte Carlo Tree Search	25
2.5	Applications of Combined Machine Learning and Search	26
2.5.1	Game Playing AI	26
2.5.2	Natural Language Processing	26
2.5.3	Robot Motion Planning	27
2.6	IMPSY	28
3	System Design	31
3.1	Problem Formulation	31
3.1.1	Research Scope	32

Table of Contents

3.2	The Prediction Algorithm	32
3.2.1	Monte-Carlo Tree Search	33
3.3	Heuristic Components	38
3.3.1	Key and Modal Conformity	40
3.3.2	Tempo and Swing Conformity	44
3.3.3	Pitch Interval Markov Model	45
3.3.4	Time Multiple Markov Model	47
3.3.5	Repetition Markov Model	48
3.3.6	Failed Parameters	50
3.3.7	Potential Heuristics for Other Control Dimensions	50
3.4	Overview of Meta-Parameters	51
4	Evaluation	55
4.1	Prediction Accuracy	56
4.1.1	Datasets	56
4.1.2	Pitch Prediction Accuracy	58
4.1.3	Duration Prediction Accuracy	60
4.1.4	Total Prediction Accuracy	61
4.1.5	Heuristic Component Contribution Assessment	63
4.1.6	Performance Scaling of the Combined Heuristic	65
4.2	Qualitative Performance Experience	67
4.2.1	Structured Improvisations	69
4.2.2	Freeform Improvisations	71
4.2.3	General Observations	71
5	Discussion	73
5.1	Impact	73
5.2	Limitations	76
5.3	Future Work	78
5.3.1	Improving the MCTS Search	78
5.3.2	Improving and Expanding upon Heuristics	79
5.3.3	Improving User Interactivity and Control	80
6	Conclusion	83
6.1	Summary of Key Findings	83
6.2	Answers to the Research Questions	84
	Bibliography	87

Introduction



Figure 1.1: A collaborative improvisation session with MCTS-IMPSY.

New Interfaces for Musical Expression (NIMEs) are an important field of research for making musical interfaces more accessible, expressive, creative, and fun. In real-time musical performance, NIMEs provide new ways for musicians to interact with digital systems, expanding the possibilities for improvisation, collaboration, and musical exploration. Music is a fundamental aspect of human culture and self-expression, and improving the ways people can interface with it can increase inclusion and participation in this important aspect of human culture. Additionally, as AI-driven creative tools become more sophisticated, they have the potential to redefine artistic collaboration be-

1 Introduction

tween humans and machines, opening new pathways for artistic expression. Improving the capabilities of NIMEs not only enhances artistic possibilities, but also contributes to a deeper, more accessible, and more human-centered approach to digital music interaction. By augmenting or transforming traditional instruments, or even introducing entirely new modes of interaction, NIMEs allow performers to push the boundaries of human-computer interaction in music. Real-time interactive NIMEs are a subgroup which can interact with human performers in expressive ways, making them particularly valuable in live performance contexts. However, designing a system that can both understand and contribute meaningfully to a live musical performance remains a challenging task, requiring a balance between structured musical coherence and the flexibility of creative improvisation. This is a problem when developing interactive musical prediction systems which can improvise with a human in a call-and-response fashion, and is the focus of this thesis.

Recent developments in NIMEs which can be applied in real-time musical performance have broadly fallen into two categories. Heuristic based approaches break down musical harmony and structure symbolically into discrete components which can be logically reasoned with. By converting the expression of a human performer into this symbolic language, a NIME can then dynamically interact with the performer by performing calculations in this logical space (Fu et al., 2023). This includes rule-based methods, where hardcoded musical constraints direct composition, and search based techniques, which explore the space of future musical directions, while being guided by heuristics. These methods are excellent for generating structured, coherent, and theoretically sensible musical outputs. However, they suffer from restrictions on their breadth of musical expression, which often leads to robotic and less creative sounding results (Anders and Miranda, 2011).

In contrast, other methods have used recurrent neural networks (RNNs) trained on human performance to repeatedly predict the most likely next step in a performance. When used for NIMEs, these techniques solve many of the issues of heuristic-based approaches, as they are formulated to continuously predict the most human sounding continuation of a musical performance. However, these approaches aim for only the most likely next step, often leading to human-like, but very predictable and repetitive musical outputs. This is usually mitigated by adding randomness to the prediction process, by way of increased temperature, softmax sampling, or random sampling using Mixture Density Networks (MDNs). While adding randomness to the prediction process can make the performance more creative, it is a very arbitrary way to add this creativity and diversity, and has the tendency to very quickly lead to incoherence if the randomness is too high (Martin and Tørresen, 2018, 2019).

This motivates the question of whether a more guided approach than the addition of pure randomness to the prediction process could be used, and such a guided approach is what is presented in this thesis. This guided approach is in the form of a hybrid system, shown in Figure 1.2, which uses both heuristic and mixture density recurrent neural network (MDRNN) based approaches, with a search algorithm being used to guide the choice of

outputs from the gaussian mixture model (GMM) of output likelihoods provided by an MDRNN prediction. From making repeated MDRNN predictions, and treating these predictions as the action space for each node (musical note) within Monte Carlo Tree Search (MCTS), a tree of possible simulated musical continuations is generated. Then, from the measurement of structural parameters of the music; such as key signature, tempo, and interval probabilities; a heuristic is created which measures the level of conformity of each simulated musical continuation to established musical parameters in the piece. Finally, these heuristic measurements are backpropogated through the search tree and used to guide the next musical output toward a choice which is consistent with the previous state of the performance. This allows for the system to maintain the benefit of MDRNNs, which is the ability to generate flexible outputs which allow for more creativity, without sacrificing musical coherence.

This new hybrid system is then applied as a new prediction algorithm for IMPSY ([Martin and Tørresen, 2019](#)), an interactive musical prediction system which uses an MDRNN to continue human improvisation in a call-and-response manner, generating collaborative improvisations with a human performer. For clarity, IMPSY with this MCTS-based prediction architecture is referred to as MCTS-IMPSY, and with the original single MDRNN prediction architecture is referred to as MDRNN-IMPSY. This new prediction system for IMPSY is the first application of a search-based system with a neural network generated action space in an interactive musical system. IMPSY is notably unique in that it predicts control gestures for an instrument, instead of the musical output itself. IMPSY is also designed for users to be able to train their own small prediction models on their own performances, so that the system can imitate their musical style. Currently, IMPSY randomly samples from a GMM output at each step to mitigate the predictability standard RNNs. This thesis aims to guide this sampling instead through the described method. A diagram of how the call-and-response mode of MCTS-IMPSY operates can be seen in Figure 1.2.

1.1 Methodology

The methodology guiding the choice of approach to this research is based on two notable findings from the literature. The first of these is the split between heuristic and machine learning based approaches to generative music, and particularly the notable difference in outcomes when each of these approaches is used in isolation. Heuristic and rule-based approaches are excellent at generating coherent music which adheres to established musical structure, but lack the expressiveness and creativity of modern machine learning systems. In contrast, machine learning based approaches are capable of generating more human-like and creative outputs, but often at the cost of coherence and structure. However, research into approaches combining both of these methods for music generation tasks are extremely limited, with no research being found combining both approaches for any interactive musical system. The second notable finding is that in other fields, such as robot motion planning, the combination of heuristic and machine learning based

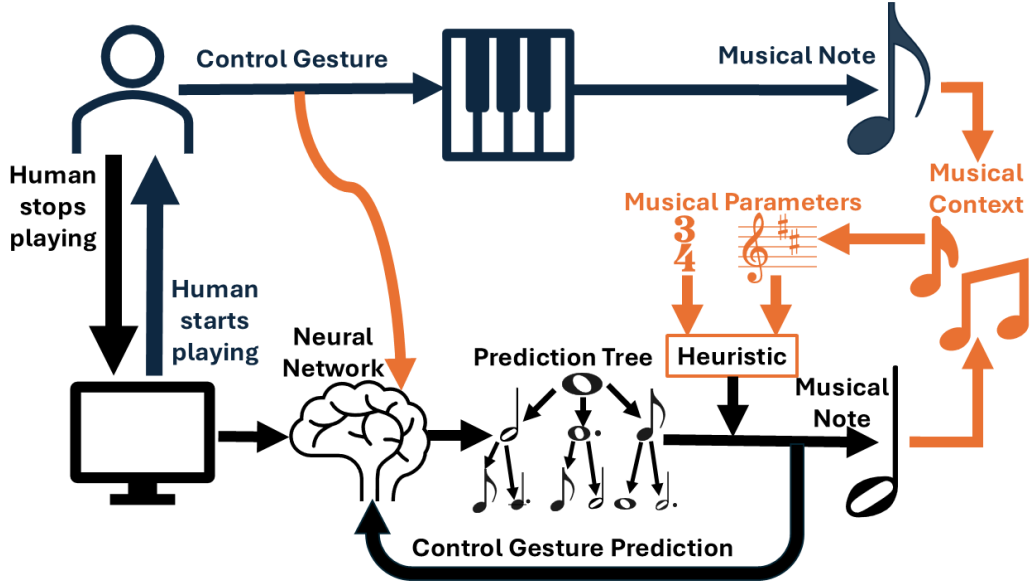


Figure 1.2: An overview diagram of the MCTS-IMPSY call-and-response system, with human actions in blue, system prediction actions in black, and interconnecting processes in orange.

approaches has been shown to be extremely effective for improving the accuracy of a system by guiding the probabilistic outputs of neural networks with search. In this field, very similar combinations of MCTS and neural networks to this system design have advanced the field of self-driving cars (Lei et al., 2021), air combat simulations (Zhang et al., 2022), and more. This suggests that a similar methodology could be effective for interactive musical prediction systems. These two findings led to the development of a novel system design where rule-based approaches can be added as component heuristics to a larger combined heuristic which guides the search process of MCTS, itself defined on the action space generated by the predictions of an MDRNN.

The methodology for evaluating the system is based on the aim of combining the strengths of both heuristic and machine learning based approaches, while limiting the weaknesses of each. Since the baseline is MDRNN-IMPSY, which is a machine learning based approach, the goal of MCTS-IMPSY is to improve musical coherence, structure, and predictive accuracy, while maintaining the expressiveness and creativity of the original system. Hence, the evaluation will be split into two parts. The first part will be a quantitative evaluation of the system, where the predictive accuracy of MCTS-IMPSY will be compared to MDRNN-IMPSY. The second part will be a qualitative evaluation of the system based on recorded improvisations with the system, where the playability, expressiveness, and creativity of MCTS-IMPSY will be compared to that of MDRNN-IMPSY. This evaluation will be conducted using both a hand crafted dataset of personal improvisations, to test the system for the practical use case IMPSY was originally de-

signed for, and a larger dataset of over 1000 folk music melodies, to test the system for a more general and structured use case.

1.2 Research Questions

This thesis aims to address the following questions:

1. What are the strengths and weaknesses of heuristic and machine learning based approaches in NIMEs, and how can they be effectively combined?

In Chapter 2, the literature on NIMEs will be reviewed, including both heuristic and machine learning based approaches. The strengths and weaknesses of each approach will be discussed, as well as how they can be combined to create a more effective system. This section will also look into other fields where combinations of heuristic and machine learning based approaches have been successfully applied. This includes discussion of some specific papers with very similar approaches to ours: using MDRNNs to generate GMM outputs, and then using those GMM outputs as continuous action domains for MCTS. Chapter 3 then describes the implementation of the combined MDRNN and MCTS algorithm, and its integration into IMPSY. The details of the algorithm, including each design choice, and an explanation of how an effective combination of approaches is developed, are in Section 3.

2. How can an effective heuristic be designed for MCTS-IMPSY to intelligently guide the search process?

Chapter 3 will also describe the heuristic developed for measuring the coherence of a musical performance. Each musical parameter which influences a component of the heuristic is described in detail, including musical and/or psychological research on its relevance to musical prediction and understanding in humans, and how it is calculated.

3. Can MCTS-IMPSY improve on the prediction accuracy and playability of MDRNN-IMPSY, while maintaining creativity?

In Chapter 4, the system is evaluated using both quantitative and qualitative measures. It will be compared to MDRNN-IMPSY for measures of both musical quality and improvisational prediction ability. Chapter 5 and 6 then also discuss the implications of this thesis for the future of NIMEs and human-computer interaction more broadly.

1.3 Thesis Structure

This thesis consists of 6 chapters. After this introduction in Chapter 1, Chapter 2 covers the necessary background information which is important for understanding this thesis. This includes a brief overview of NIMEs more generally, as well as the development of both machine learning and heuristic based approaches to NIMEs. It also covers combinations of these methods which are similar to MCTS-IMPSY, but applied in different fields. Finally, it discusses IMPSY, the system this thesis expands upon, and

1 Introduction

its limitations which MCTS-IMPSY addresses. Chapter 3 details the system design and implementation process, including the novel algorithm for sequential musical prediction, and the details of each component in the heuristic used in this algorithm. Chapter 5 then details both quantitative and qualitative measures of evaluation used to assess the performance of the system, and the results of this evaluation. Chapter 6 discusses the implications of these results, noting their broader impact and avenues of future work created, but also the limitations of this thesis. Finally, Chapter 7 concludes the thesis by summarising the findings of this thesis and directly answering the research questions posed in the previous section.

Background

This thesis relates to work in multiple different fields, including musical controllers, generative music, and robot motion planning, and this chapter provides an overview of each of these fields, connecting back to their relevance to this thesis. It will cover the advantages and disadvantages of various AI methods and their applications to computer music, and discuss gaps in the literature which will be covered within this thesis. This chapter also covers the machine learning and search algorithms used in the system design, their previous applications, and their potential to be applied within this thesis. This chapter begins with an overview of New Interfaces for Musical Expression (NIMEs), including a brief history of developments in computer music which led to the creation of the field, important NIME papers, and the major subclasses of NIMEs. Then the role of machine learning in computer music will be discussed, and different types of neural networks will be explored, building toward the Mixture Density Recurrent Neural Networks (MDRRNs) used within IMPSY. This is followed by a look into heuristic based AI, a different approach to machine learning based techniques, and its advantages and disadvantages in music generation. Next, the background for search methods not commonly used in previous computer music literature, but which are used in MCTS-IMPSY, or in other fields where similar combinations of machine learning and search are utilised, is covered. Then existing research combining machine learning and search which have already achieved impressive results in other fields, and inform the system design of MCTS-IMPSY, are reviewed. Finally, the IMPSY system is covered in more detail, including its impact on the field and advantages, as well as what its weaknesses and gaps in function are, and how applying combined machine learning and search techniques found in other fields could potentially address some of these weaknesses.

2.1 New Interfaces for Musical Expression (NIMEs)

The Cambridge Companion to Electronic Music by Collins and D’Escriván (2012) describes three eras of music programming. The first era began with MUSIC-1, the first digital computer music synthesis program, which was followed by a series of versions collectively referred to as MUSIC-N. MUSIC-N introduced the concept of algorithmic composition, and is the precursor to modern computer music. The second era was defined by the introduction of real-time music programming languages, such as Max (Puckette, 2002) and Pure Data (Puckette, 1997). Pure data is a graphical drag and drop modular programming language which allows for algorithmic compositions to be edited during performance, by modifying the code of the system while a performance is in progress. While live performance with pure data is rudimentary compared to modern alternatives, it was a large step forward in the computer music space. The third and current era of music programming refers to the surge in more real-time and interactive musical interfaces, and a shift from programming being a tool to create musical algorithms, instruments, and performances, to programs becoming the instruments themselves. NIMEs are at the core of what this current era represents. The term NIME originally comes from a conference series of the same name, which began in 2001 and now annually showcases the latest NIME related research.

The key category of NIMEs relevant for this thesis are real-time interactive NIMEs. These are NIMEs where the user and system interact in real-time during musical performance, and can be approximately split into three main subcategories. The first is NIMEs focused on the mapping from a controller to a musical output, such as gesture based controllers, or augmented instruments. The second involves simultaneous collaborative performances involving independently acting users and systems, such as human-robot collaborative ensembles. Finally, the third subcategory is interactive musical prediction systems, where the system learns from the user and contributes accompaniment, including in the form of call and response. The call and response mode of interactive musical prediction systems is the main focus area of this thesis.

2.1.1 Controller Mappings

A type of controller which drove innovation in mapping controller inputs to musical outputs is gesture-based controllers. Early gestural musical controllers such as the Radio Baton and Conductor Program helped to establish early frameworks for three-dimensional gestural control (Lawson and Mathews, 1977; Mathews, 1991). These advances were a step toward the use of this technology in commercial devices such as the Wii Remote and Microsoft Kinect, devices which have influenced gestural controllers in many fields including music, and made gestural technology accessible and recognisable to the broader public (Kiefer et al., 2008; Zhang, 2012; Takase and Shiramatsu, 2020).

Other work focused on designing mapping strategies which were generalisable to many types of input and output. Hunt et al. (2000) introduced a generalised mapping strategy which uses a two-stage rules-based concept which could allow for any input controller to

2.1 New Interfaces for Musical Expression (NIMES)

control the same musical output function. This was done by the initial stage, which is controller specific, converting controller inputs into values of broad musical concepts, and a second stage converting those values into musical outputs. The Wekinator, created by Fiebrink et al. (2009) was even more generalisable, allowing performers to train custom machine learning models to map controller inputs to musical outputs through user provided training examples. Figure 2.1 shows this interactive learning process, which is similar to the process used in IMPSY (Martin and Tørresen, 2019). More recent applications involve mapping strategies where the mapping itself can perform the vast majority of the music generation, while still providing a level of control to the user, such as text to music generation models (Wu et al., 2024).

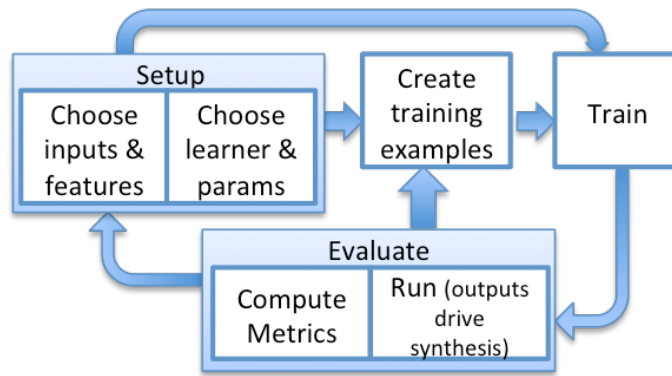


Figure 2.1: Reproduction of Figure 1 from *A Meta-Instrument for Interactive, On-the-Fly Machine Learning* (Fiebrink et al., 2009). A flowchart of the interactive mapping generation process in Wekinator.

Augmented traditional instruments represent another significant approach, and can add enhanced control capabilities to instruments, adapt their outputs, or both, without sacrificing their familiarity. Machover and Chung (1989) created the hyperinstruments project, which was an early work focusing on control capabilities by enhancing traditional instruments with sensors and digital processing capabilities. A more recent example is the Magnetic Resonator Piano by McPherson (2010), which utilises electromagnetic actuators to directly manipulate acoustic piano strings, creating new sound possibilities while maintaining the acoustic piano as a base.

MCTS-IMPSY aims to be agnostic to the type of controller, mapping strategy, and output used in a similar way to the Wekinator (Fiebrink et al., 2009), as any instrument which can map its controls to a set number of dimensions can be used to train a model for the system. This is particularly important for this thesis, as the system is designed to adapt to the unique control strategies of each performer, and it would be significantly limited to restrict this adaptation to only a single instrument.

2 Background

2.1.2 Collaborative Networked Instruments

Collaborative networked instruments have extended the concept of NIMEs beyond individual performance, allowing multiple performers to interact simultaneously across physical and virtual spaces, and for AI and human performers to collaborate in simultaneous performance. While more primitive collaborative networked performances had been conducted earlier, the spreading of high speed internet in the early 2000s is what led to it becoming a more commonly researched topic (Renaud et al., 2007). Early research often focused on producing some kind of engaging performance which utilised a distributed computer architecture, with the core difficulties being synchronization strategies and communication protocols. For example the Stanford Laptop Orchestra (Wang et al., 2009) conducted many such performances. Other work has focused more on systems where perception of what the other performers are doing is the main difficulty, such as a system designed by Weinberg et al. (2009) where AI systems and human performers must interact and improvise in real-time to produce coherent musical output. While this thesis does not work with simultaneous collaborative systems, it does still require call-and-response collaboration between one human and one AI performer, and so this work on perceiving human playing style in order to respond coherently is a core part of the system design. By tracking musical parameters important to the human perceptual framework, such as rhythmic stability (Desain and Honing, 2002), system predictions can match these parameters when responding to the human performer and hence better follow on from human improvisation. While these methods are very common in collaborative networked systems, they are not as common in the machine learning based interactive musical prediction systems that MCTS-IMPSY aims to improve upon, and so integrating these methods into MCTS-IMPSY could fill this gap in research.

2.1.3 Interactive Musical Prediction Systems

Interactive musical prediction systems anticipate and respond to performer actions based on learned patterns and musical knowledge. These systems include rule-based systems based on knowledge of musical theory, either learned or hardcoded; Markov-based approaches, probability trees, and other types of probabilistic predictions of a next musical action; and various neural network architectures capable of learning to make musical predictions from scratch (Collins, 2007). An example of a Markov-based system is the Continuator by Pachet (2002), which builds a database of recognised musical patterns from a training set of human improvisation, and from this database generates a Markov model to predictively continue on from human performance in call-and-response fashion. These earlier types of call-and-response systems form the background for the development of IMPSY (Martin and Tørresen, 2019). IMPSY uses a mixture density recurrent neural network (MDRNN) to allow for collaborative call and response performances between human performers and an AI system by predicting the most likely next instrument control values. IMPSY will be explored in more depth later in the chapter.

2.2 Machine Learning in Music

Machine learning techniques expand the possibilities of NIMEs significantly by allowing for sophisticated and human-like generative music. Unlike rule-based approaches which rely on the hardcoding of musical knowledge, machine learning models can discover patterns and relationships directly from musical data, allowing for more adaptive musical interactions. A well-trained musical prediction model can anticipate likely continuations of musical phrases, suggest complementary harmonizations, or generate real-time accompaniments to human performance on a variety of instruments. NIMEs can also be generalisable to multiple models trainable by users, allowing for easy expansion to different instruments and performance styles. The creation of effective machine learning based NIMEs requires an ability to capture both local patterns and global structures inherent in music. They must also balance the advantages of purely exploitative strategies, which aim to consistently generate the most likely next note, with explorative ones, which aim for more unique and human-like results. Many different machine learning architectures have been explored in attempt to balance these objectives, and each offers their own advantages and disadvantages. This section examines five key approaches that have proven particularly valuable for musical prediction tasks, and which build upon each other toward the MDRNNs utilised in IMPSY.

2.2.1 Markov Models

Markov models have been widely used in algorithmic composition due to their ability to capture local musical patterns and transitions, and are used to create multiple component heuristics in the system design of the thesis. These probabilistic models operate on the principle that future states depend only on the current state, as well as sometimes a finite history of states, making them particularly suitable for modelling sequential musical events where local context strongly influences prediction. They are incredibly efficient computationally, requiring only a transition matrix to represent the probabilities of moving between states. However, this simplicity comes at the cost of limited expressive power, as Markov models struggle to capture long-term dependencies and hierarchical structures present in music.

The most basic form of Markov model is a first-order Markov chain. These chains model transitions where the next state depends solely on the current state, represented by a simple transition probability matrix. While computationally efficient, they often produce musically simplistic results that cannot replicate long-term structure, as shown by Figure 2.2. In contrast, higher-order Markov chains incorporate longer histories by conditioning predictions on sequences of previous states, enabling more coherent musical phrases and motifs. Figure 2.3 shows a basic first-order Markov model of note transitions, and Figure 2.3 is a valid potential second-order version of this first order model. Note how the second order model is larger in size and more complex, but can also capture more detail, such as that the $D \rightarrow A$ transition should only ever occur when the previous note was $F^\sharp$, and the note sequence $A \rightarrow D \rightarrow A$ cannot occur.

2 Background

(a) Input source material



(b) First-order transition table output



Starting over

(c) Primarily second-order transition table output



(d) All second-order output



Figure 2.2: Reproduction of Figure 13 from *M and Jam Factory* (Zicarelli, 1987). Outputs from an entirely first order, mixed, and entirely second order Markov model given (a) as source material

Higher order Markov chains, while more powerful, suffer from exponential growth as their order increases. This problem led to Ron et al. (1996) developing the Variable-order Markov model (VOMM), which adaptively determines the appropriate context length for prediction based on available training data. This adaptability allows VOMMs to capture both short patterns when data is sparse and longer sequences when following significantly well travelling paths in the training data, making them particularly valuable for interactive systems where response time and predictive accuracy must be balanced.

Hidden Markov Models (HMMs) are another type of Markov Model where the states are only partially observable, and the goal of the system is to predict the most likely series of states which generate a visible set of transitions. For example, Allan and Williams

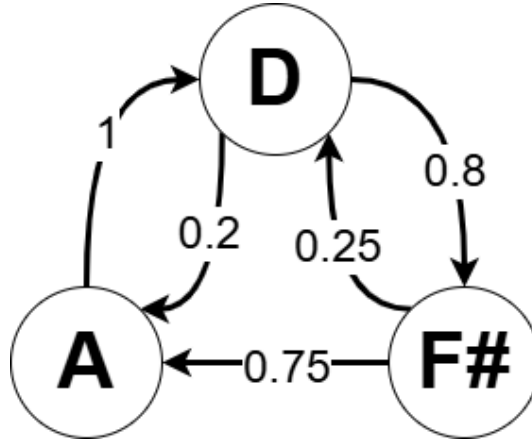


Figure 2.3: A first-order Markov model for basic note transitions.

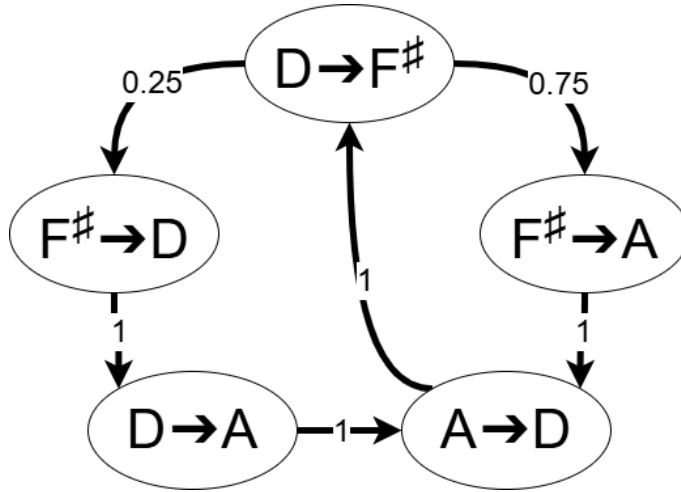


Figure 2.4: One valid second-order version of the first order Markov model in Figure 2.3. Note how additional complexity can be encoded.

(2004) show how having chords as hidden states, but melody as a visible transition, can create a model which predicts viable chord accompaniments to melodies. Simon et al. (2008) develops this concept further, allowing users with no musical experience to sing and have accompaniment automatically generated for them by a HMM. HMMs are particularly useful in these kinds of cases where there is a clear distinction between a known element, and an unknown element which must be generated based on the known element.

Markov models effectively generate simple accompaniments, recognise common patterns in musical styles, create reasonable results with limited training data, and can easily be made computationally efficient enough for real-time applications. However, they

2 Background

are limited by their simplicity, as they can only capture local patterns and transitions, and struggle to model complex musical structures. This limitation has motivated the development of more sophisticated neural network architectures that can capture more complex and less well-defined aspects of music.

2.2.2 Feedforward Neural Networks

Feedforward neural networks (FNNs) are a step in-between Markov models and recurrent neural networks. Unlike Markov models, which function based on explicit transition probabilities, FNNs can discover non-linear relationships within musical data, which can capture more complex musical structures. However, FNNs face limitations in handling sequential data, as they require fixed-size inputs and lack mechanisms for capturing long-term dependencies. The name ‘feedforward’ refers to the one directional flow of information through the network, where each layer’s output serves as the input to the next layer, without any loops or backward flowing feedback connections. Figure 2.5 shows how a feedforward network could be designed for an augmented instrument mapping strategy, where the first layer is the input layer, which could encode the controls of the instrument, there are then many hidden layers which transform these controls in complex ways, and then finally an output layer which produces a sound. In this figure this example is a remapping of a keyboard to allow it to produce vibrato.

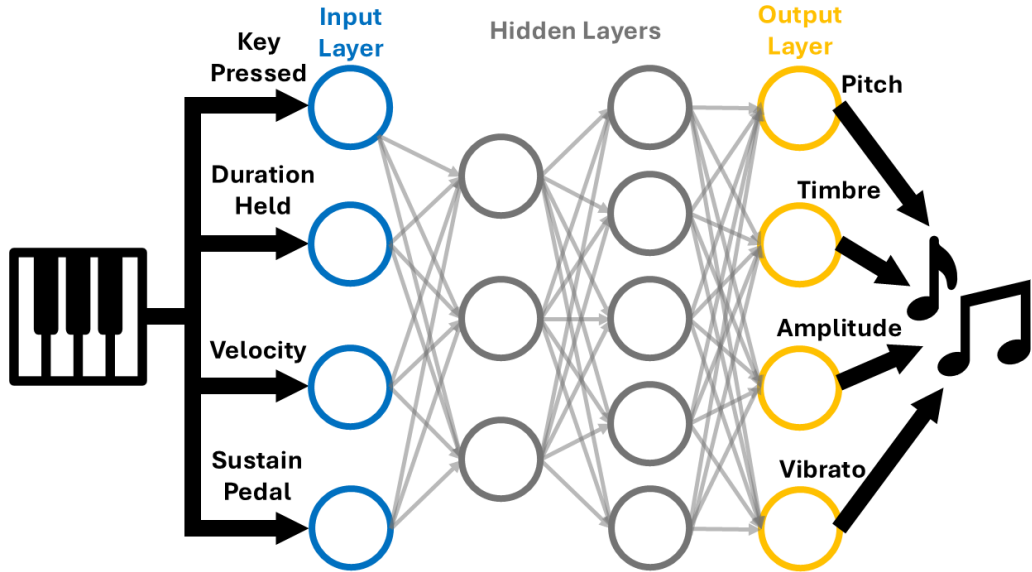


Figure 2.5: A hypothetical feedforward network mapping keyboard inputs to new set of outputs, including vibrato.

While FNNs could theoretically be used for simple sequential musical functions, they are much better suited to musical classification tasks, such as evaluating rhythms (Dolson, 1989), chord classification (Laden and Keefe, 1989), and musical style classifica-

tion (Dannenberg et al., 1997). This is because their limitations made them unsuitable for sequential tasks such as musical prediction or generation. Their requirement of a fixed size input size means that musical sequences must be encoded into fixed length vectors, often forcing dimensionality reduction which leads to the loss of important information. Additionally, their lack of feedback connections means that they cannot remember previous inputs, making them unsuitable for temporal data like music. However, FNNs established important foundations for the later development and use of recurrent neural networks.

2.2.3 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) extend beyond the limitations of both Markov models and feedforward neural networks by maintaining internal states that can capture more complex temporal dependencies in musical sequences. Unlike FNNs, which process inputs independently with no inbuilt mechanism for processing sequences, RNNs incorporate feedback connections which allow information to persist across time steps (LeCun et al., 2015). This is in the form of parameter sharing, which allows for propagation of learning weights within the network over multiple time steps, preventing the need for the network to repeatedly relearn already determined musical rules which are likely to persist over time. However, RNNs can suffer from the vanishing and exploding gradient problems, where gradient information either diminishes or grows exponentially as it is propagated through time (Bengio et al., 1994). In practice, this means that basic RNNs often fail to capture important long-range dependencies that create coherent musical phrases and sections.

RNN architecture consists of a hidden state that acts as a form of memory, allowing information to persist across time steps. This hidden state is updated at each time step based on both new input and its previous value, making RNNs particularly suited for sequence modelling tasks. Weights are kept when the hidden state is updated, allowing the network to base its prediction of the entire previous prediction history. Figure 2.6 shows a simple RNN architecture, where the input is fed into the network at each time step, and the hidden state is updated based on both the input and the previous hidden state. The output of the network is then generated from the hidden state, which can be used to predict the next note in a sequence. When successive input layers repeatedly increase or decrease network weights, their transfer to future time steps can lead to exploding/vanishing gradients in the hidden layers.

Todd (1989) was an early example of using RNNs for composition, and split music into a series of discrete 16th note blocks, which were then used to train a simple RNN to predict the next note in the sequence. The recurrence in the network allowed it to learn melodies up to 20 notes in length, and this helped it to generate melodies with longer dependencies than previous Markov model approaches. Another example is the CONCERT system (Mozer, 1994), which includes pitch, duration, and chord as outputs. This removes the need for discrete 16th note blocks, as duration is now a continuous variable. It can also learn at the phrase level to generate higher level musical structures,

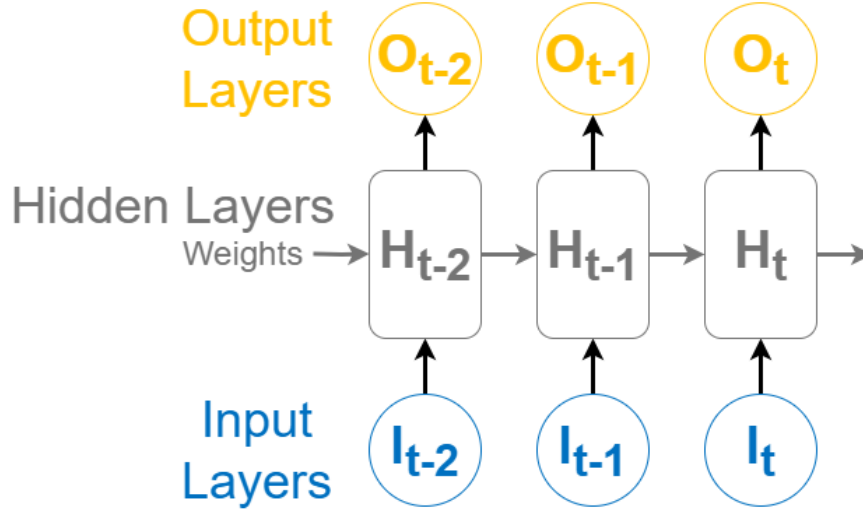


Figure 2.6: A simple RNN architecture, where the input, hidden, and output layers are compressed representations of the layers shown in Figure 2.5.

and was shown to be both more concise and more general as a model than third order Markov models.

RNNs established the foundation for neural sequence modelling in music generation systems and demonstrated the potential for learning-based approaches to capture musical relationships across multiple time scales. However, their architectural limitations, particularly in regards to gradient issues preventing long-term memory, ultimately motivated the development of more sophisticated recurrent architectures specifically designed to address the challenges of long-term dependency modelling.

2.2.4 Long Short Term Memory Networks

Long Short-Term Memory networks (LSTMs), created by [Hochreiter and Schmidhuber \(1997\)](#), are a type of RNN which addresses the vanishing and exploding gradient problems of standard RNNs, enabling more effective learning of long-term dependencies crucial for musical coherence. Their gated architecture provides selective memory capabilities that better align with musical structures spanning multiple time scales. LSTMs can remember critical information over extended periods, but also forget irrelevant information, keeping them computationally efficient and effective for music generation tasks. Most of the time that RNNs are mentioned in other sections of this thesis, they refer to LSTM RNNs, as these are the most commonly used RNNs in music generation tasks today ([Ji et al., 2024](#)).

The LSTM architecture introduces three specialized gates; the input, forget, and output gates; which regulate information flow through the network. These gates determine which information should be added to the cell state, which should be removed, and which

should be output, allowing the network to selectively preserve important features while discarding irrelevant ones. This selective memory mechanism is particularly valuable for music, where relationships between notes can span varying temporal distances depending on the musical context. Figure 2.7 shows a diagram of the technical workings of the LSTM architecture, although a detailed understanding of how each of these gates work on a technical level is not necessary for following this thesis.

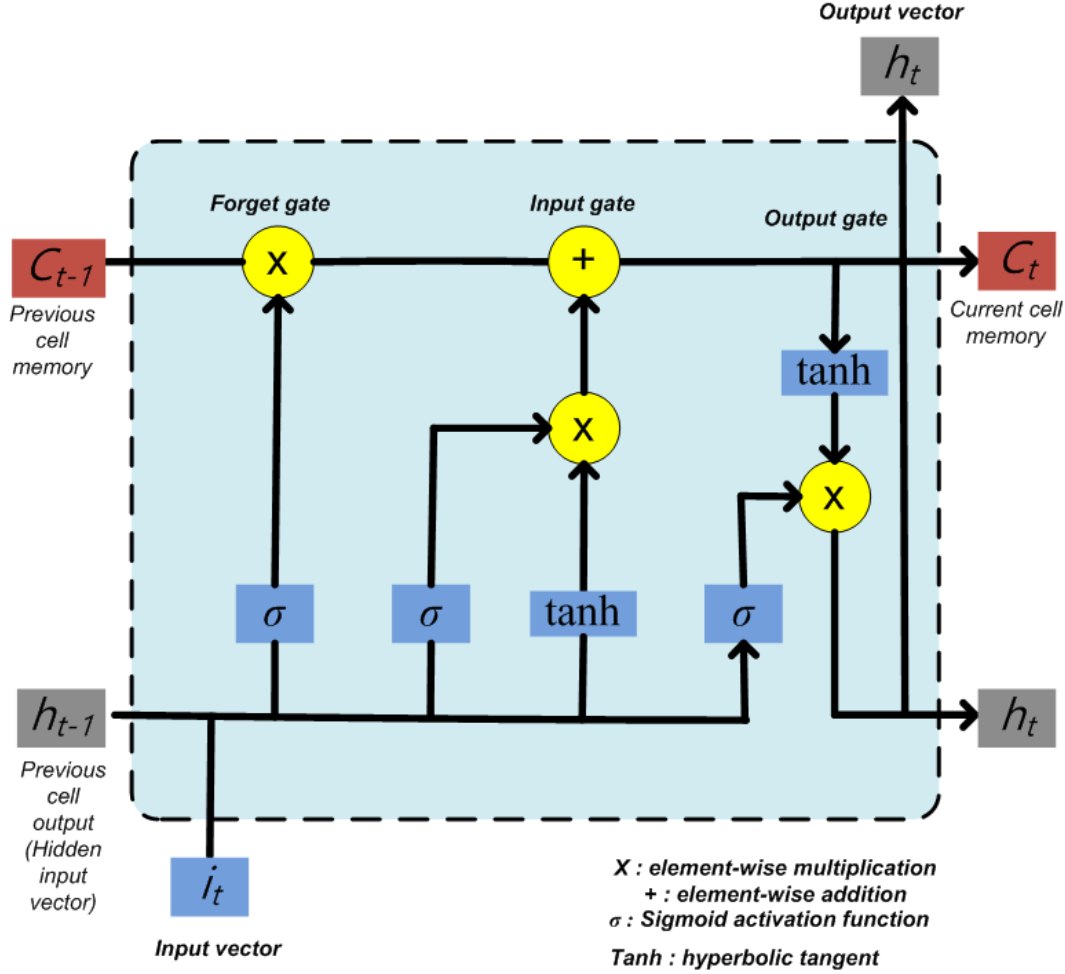


Figure 2.7: Reproduction of Figure 4 from *Transfer Learning-Based Intrusion Detection System for a Controller Area Network* (Khatari et al., 2023). The Long Short-Term Memory (LSTM) cell architecture.

In music generation, LSTMs have demonstrated remarkable capabilities. Unlike standard RNNs, they can maintain coherent harmonic progressions across extended musical phrases while simultaneously managing multiple independent voices. The first notable use of LSTMs for music generation was by Eck and Schmidhuber (2002), which trained an LSTM to generate blues improvisations. It had improved structural coherence over

2 Background

standard RNNs, although still not close to the structural coherence of human performance. A more modern LSTM prediction system is DeepBach (Hadjeres et al., 2017), which attempted to generate convincing chorale harmonizations in the style of Bach. The results showed that on average, around 50% of listeners thought a given DeepBach harmonization was composed by Bach, compared to scores of around 80% for Bach’s actual compositions. This shows that LSTMs can produce results which are very close to human performance, and even fool expert listeners in some cases. However, this system still had issues with understanding some musical concepts such as transposition, as it was not translation invariant, and was still only operating with relatively short pieces in a very limited and rules-based style of music. Johnson (2017) provided a solution to the former problem by developing a method to make translation invariant LSTMs, however, this method was only effective when trained on a single musical style, similarly to how DeepBach was, and training on multiple styles would lead to confusing stylistic shifts within a piece. The DeepJ model (Mao et al., 2018) later solved this issue by adding specific style conditions at each layer of the network.

Despite their advantages over standard RNNs and multiple decades of improvements, LSTMs still face multiple limitations (Ji et al., 2024). They still struggle with very long-term dependencies, such as musical motifs. They also struggle with more variable musical structures, such as those found in jazz or free improvisation as opposed to classical music, where the relationships between notes are less predictable. This is because partially due to LSTMs producing only a single output instead of a probability distribution over possible continuations. A solution to this latter limitation is to use a softmax layer at the output of the LSTM, which allows for a probability distribution over the possible next notes. However, this approach is limited to discrete possibilities, and so cannot capture the full range of possibilities, especially for many dimensional continuous musical parameters for instrument output.

2.2.5 Mixture Density Recurrent Neural Networks

Mixture Density Networks (MDNs) are a type of neural network which output a Gaussian Mixture Model (GMM), which is a weighted sum of multiple Gaussian distributions across any number of dimensions. This GMM is used as a probability distribution for the output note, as opposed to traditional networks, which output only a single output option (Bishop, 1994). Mixture Density Recurrent Neural Networks (MDRNNs) are an RNN with a GMM output, and can solve the issue of RNN methods being seen as producing results which are too robotic or predictable over the long term to be truly human-like, by allowing for the benefits of softmax outputs to be extended to continuous musical parameters. It also extends recurrent architectures in a way which allows them to better handle multiple continuous dimensions simultaneously, such as pitch, time, timbre, and many more, enabling more holistic prediction models.

MDRNNs incorporate a GMM output by outputting the necessary parameters vectors π , μ , and σ ; each of which have the same dimension as the required output. The π parameter is the weight of each Gaussian in the mixture, the μ parameter is the mean

of each Gaussian, and the σ parameter is the standard deviation of each Gaussian. The output of the MDRNN is then a sample from the GMM, which can be used as the next note in the sequence. Figure 2.8 shows a simple MDRNN architecture for sequential note prediction. The output of the MDN are the inputs to a GMM, which is then used to sample the next note in the sequence.

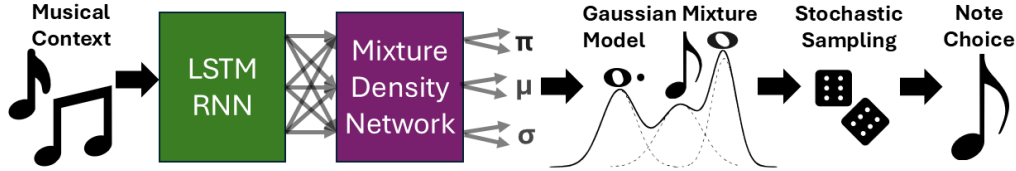


Figure 2.8: How MDRNNs are used for sequential note prediction.

MDRNNs have been used in multiple human prediction domains, including generating handwriting (Graves, 2013), sketching (Ha and Eck, 2017), and touchscreen performances (Martin and Tørresen, 2018). In all cases GMMs were used to generate more flexible outputs in continuous domains where the goal was to seem human-like, and performed better than softmax outputs. Hence, MDRNNs are a natural fit for music prediction systems, where this is also the goal, and have succeeded in this domain previously too (Martin and Tørresen, 2019). This is because MDRNNs can probabilistically express future likelihoods for continuous parameters such as timing, dynamics, timbre, and articulation; creating natural variations in expression that more closely resemble human musical composition.

However, the benefits of MDRNNs to musical prediction tasks come with a caveat. While the addition of probabilistic outputs introduces useful variability, this approach is essentially adding arbitrary noise to otherwise deterministic predictions. This is in contrast to human improvisation, which introduces variations with intentional structural and semantic meaning rather than random deviations. In this sense, MDRNNs take the disadvantages of standard RNN outputs being too structured, and move too far in the opposite direction, producing outputs with arbitrary randomness instead. Ideally, a system would be able to balance these two extremes, producing outputs which are both structured and meaningful, but also varied and creative. This is exactly what this thesis aims to achieve, by replacing the final step of note generation in a standard MDRNN, random selection from the generated GMM, with a search algorithm which meaningfully guides the selection process.

2.3 Heuristic-Based AI in Music

Heuristic methods serve as an alternative to machine learning based approaches for AI music generation and prediction. While neural networks excel at capturing the statistical patterns in music, they often lack the structured knowledge that can be encoded through rules and search algorithms. Heuristic approaches provide a framework that can enforce

2 Background

musical constraints, guide exploration of the solution space, and incorporate domain knowledge that may be difficult to learn from data alone. This section examines two approaches to heuristic-based AI in music: rule-based systems, and search algorithms. The search algorithms subsection focuses on hybrid approaches that combine heuristics with machine learning models.

2.3.1 Rule Based Systems

Rule-based approaches have been used in algorithmic composition since the 1950s ([Hiller and Isaacson, 1959](#)), providing well-defined frameworks for generating music according to hardcoded constraints. These systems typically encode music theory knowledge, stylistic conventions, and compositional techniques into these constraints, which then guide note selection and musical structure. This allows for the generation of music that is very structured and coherent, at the cost of creativity, flexibility, and adaptability to novel musical contexts.

The earliest known example of a computer generated composition was the ILLIAC Suite ([Hiller and Isaacson, 1959](#)), which, while technically a Markov model based system, used the Markov model only for adding randomness to an otherwise completely rule based system. Another technique for rule-based composition is the use of grammars, which define a set of rules for recursively generating musical structures. [Steedman \(1984\)](#) used such a grammar to generate jazz chord progressions, demonstrating how formal grammars can capture the underlying structure of musical styles. Rules based systems like these almost always focused on a single style, and were the most prevalent method of computer music generation in its early years ([Moorer, 1972](#)).

Sometimes rule-based systems utilise constraint satisfaction programming (CSP), a hybrid between rule-based and search-based systems for very complex sets of musical rules, where the only objective of search is to find a solution which adheres to rules, which are encoded as constraints in a CSP algorithm. This is a declarative method, where only the rules are defined and then the system itself decides how to apply them. This allows CSPs to find solutions where multiple competing constraints, such as voice leading and harmonisation, can co-exist. An example of this approach is the CHORAL system, intended to harmonise four-part chorales in the style of Bach by combining a set of over 270 musical rules ([Ebcioğlu, 1988](#)). [Anders and Miranda \(2011\)](#) survey research into both the creation of useful constraints for music, and their use in general systems, most of which employ CSPs.

In the context of NIMEs, rule-based systems offer several advantages: they provide predictable, theory-compliant outputs; they can incorporate domain-specific knowledge about particular instruments and styles; and they can generate results in real-time with computation time dependant only on the complexity of the ruleset. However, pure rule-based systems face significant limitations. They struggle with the flexibility and nuance of human performance, often producing robotic results that lack the subtle variations of true expressive composition. Additionally, the manual encoding of musical knowledge

becomes infeasible when attempting to capture the totality of musical composition across musical styles, as extensive hand-crafting of separate rule sets is usually required just for a single genre and instrument. This leads to a lack of adaptability to novel musical contexts not specifically encoded within the rules of the system.

2.3.2 Search Based Systems

Search algorithms combined with musical heuristics enable more sophisticated composition by exploring possible musical continuations and selecting optimal musical paths according to defined criteria. These approaches view music generation as a search problem, where the varying array of possible musical continuations is an important component of composition, and pleasing musical composition comes from exploring the space of possible continuations and selecting the most promising ones through the use of effective heuristics.

The use of tree search algorithms as the sole process of music generation is rare, as typically either constraint based search methods such as CSPs will be used, or search algorithms will be used in combination with machine learning models. Hence, the discussed papers all combine tree search with some form of machine learning architecture. Such hybrid approaches represent a promising direction for NIMEs, leveraging the pattern-recognition capabilities of neural networks while enforcing structural coherence through rules and search strategies. These systems typically employ neural networks for generating action spaces, but constrain or guide these generations using rule-based systems. This thesis is a hybrid approach of this type, and so research into these systems is particularly relevant. However, it is important to note that all papers in this category are purely generative systems and are not interactive. They also all lack generalisability and customisability, and are not designed to be used in real-time.

[Fu et al. \(2023\)](#) developed one of the few examples of a Monte Carlo Tree Search (MCTS) based musical prediction system, using a transformer model as a successor function to explore the space of possible continuations for a monophonic instrument, and selecting the most promising ones using heuristics founded on rule-based methods. This approach allows for the generation of music that is both structured and creative, as it can explore a wide range of possibilities while still adhering to musical constraints. The results showed high coherence, and better accuracy at making musical predictions than either purely rule-based or purely machine learning based methods. This is promising for the potential of this thesis, which also utilises MCTS to increase the coherence and predictive accuracy of a machine learning based system. While this paper is a promising example of the use of MCTS for guiding music generation, it doesn't include another key aspect of MCTS-IMPSY, which is the detection and continuation of key musical parameters in a piece through MCTS guidance. However, this use case of MCTS in music generation has been explored by [Ferreira et al. \(2022\)](#), who used MCTS to steer a neural language model toward a particular desired emotion, and showed very promising results compared to other methods in human listening trials. Despite its specificity to a single musical parameter, this work demonstrates the potential of MCTS for guiding the generation of

2 Background

music toward specific goals, and in combination with the research of [Fu et al. \(2023\)](#), provides promising background for the effectiveness of techniques used in this thesis.

The effectiveness of hybrid approaches provides a key insight for NIME development: rather than viewing heuristic-based and learning-based methods as competing approaches, they can be integrated to create systems that leverage the strengths of both. However, there is a gap in the application of hybrid approaches, as research into the guiding interactive MDRNN based search using MCTS with heuristics based on established musical parameters does not exist, despite all three components having been tested with positive results in the literature ([Martin and Tørresen, 2019](#); [Fu et al., 2023](#); [Ferreira et al., 2022](#)). This is the gap that MCTS-IMPSY aims to fill, as MDRNNs in interactive contexts could benefit from the structured guidance of heuristic search algorithms similarly to how generative approaches have in this related research.

2.4 Tree Search Algorithms

This thesis focuses on the ability of tree search algorithms, in particular Continuous Monte Carlo Tree Search (C-MCTS), to guide the selection process from a GMM output by using it to define a continuous action space from which a search tree can be generated. This requires an understanding of C-MCTS, and the algorithms it is built upon, Beam Search and Monte Carlo Tree Search (MCTS). This section will provide a theoretical overview of these algorithms, and some of their early applications in fields outside of music, given that their use within music is limited, to provide the background necessary for understanding the technical aspects of this thesis. It will also provide a brief overview of why they are applicable to the research goals of this thesis.

2.4.1 Beam Search

Beam Search offers a computationally efficient alternative to more exhaustive search methods by maintaining only the most promising partial solutions at each step of sequence generation. It can be thought of as a compromise between greedy search, which selects only the current most optimal candidate at each step, and exhaustive search methods such as Breadth First Search (BFS), which explore all possible branches to a given depth. This is because beam search retains the top- k candidate sequences based on cumulative heuristic scores at each step, extending greedy search while maintaining computational efficiency ([Bisiani, 1992](#)).

The algorithm operates by expanding all possible next steps from the current set of partial sequences, scoring each expanded sequence using heuristic evaluations, then pruning all but the k -highest scoring sequences. This process continues iteratively until complete sequences are generated or a termination condition is met.

An early use of beam search was the Harpy system created by [Lowerre \(1976\)](#) for speech recognition, which maintained a set of hypotheses for possible phonetic sequences in a tree structure, and kept to a limited number of hypotheses by using beam search.

Beam search continued to be used in speech recognition systems for many years, proving its effectiveness in this domain [Steinbiss et al. \(1994\)](#). A similar method of multiple hypothesis maintenance was used effectively by [Sutskever et al. \(2014\)](#) for sequence-to-sequence tasks, such as language translation, showing how a combination of beam search and neural networks can be effective.

The primary advantages of Beam Search include its relative computational efficiency and straightforward implementation. However, its limitations become apparent in generation tasks requiring exploration of diverse possibilities or long-term planning. Because Beam Search maintains only a limited set of candidates based on immediate scores, it can suffer from horizon effects where initially promising paths lead to suboptimal long-term outcomes, similarly to greedy search. In short, it mixes the advantages and disadvantages of greedy and exhaustive search methods.

2.4.2 Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) addresses the limitations of Beam Search by balancing exploration and exploitation through a combination of tree search and random sampling. MCTS constructs a search tree incrementally through repeated simulation, building confidence in promising paths over time through selection, expansion, simulation, and backpropagation. This process is explained below, and is also shown in [Figure 2.9](#).

1. **Selection:** Starting from the root node, the algorithm traverses the existing tree by selecting child nodes according to a tree policy, typically using the Upper Confidence Bound for Trees (UCT) formula ([Kocsis and Szepesvári, 2006](#)):

$$UCT = \frac{w_i}{n_i} + c\sqrt{\frac{\ln N}{n_i}} \quad (2.1)$$

Where w_i is the total reward of the child node, n_i is the number of visits to the child node, N is the total number of visits to the parent node, and c is a constant controlling the exploration-exploitation trade-off. The UCT formula balances exploitation (favouring nodes with high rewards) and exploration (favouring nodes with low visit counts), and this balance can be tweaked by adjusting the exploration constant.

2. **Expansion:** Upon reaching a leaf node, the tree is expanded by adding a child node from the action space.
3. **Simulation:** From the newly added node, a simulation is conducted to estimate the value of the node, typically by sampling actions according to a simple default policy involving some degree of randomness until reaching a terminal state. In simulations involving very large depth, the simulation can be cut off at a certain depth and the value of the node can be estimated by a heuristic.
4. **Backpropagation:** The simulation result is propagated back up the tree, updating visit counts and heuristic values for all nodes along the path.

2 Background

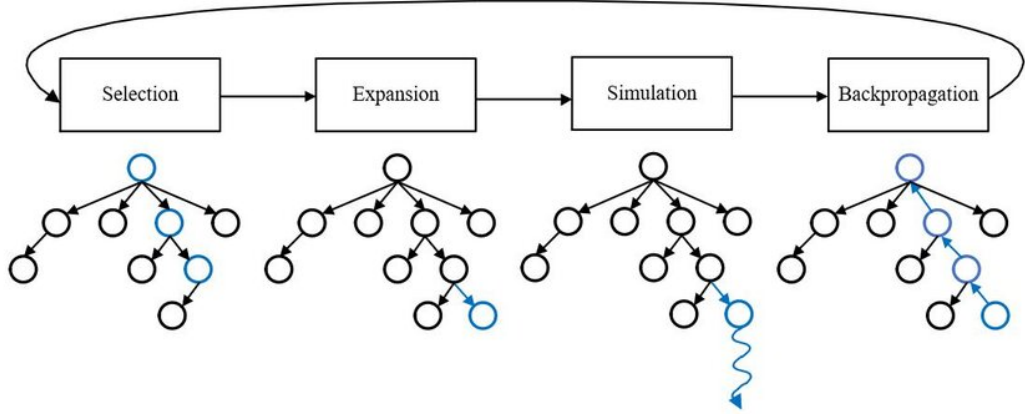


Figure 2.9: Reproduction of Figure 1 from *Multi-robot Source Navigation Method Based on Coordination Graph Monte Carlo Tree Search* (Xu, 2024). A diagram showing the individual steps of Monte Carlo Tree Search.

While monte carlo methods, where random sampling is used to simulate outcomes to deterministic problems, were first developed by Metropolis and Ulam (1949) in the 1940s and have been used in search algorithms for many years (Brügmann, 1993), the first use of MCTS in its modern form was by Coulom (2006), who used it to develop Crazy Stone, a Go playing AI. The UCT selection formula was also only developed in 2006 (Kocsis and Szepesvári, 2006), and so it is only from this point that the effectiveness of MCTS has become widely recognised. Coulom (2006) demonstrated that MCTS could significantly outperform MiniMax search in games with large branching factors, such as Go, by effectively balancing exploration and exploitation and reaching high depths through simulation. The algorithm’s ability to learn from random simulations allowed it to adaptively focus on promising regions of the search space, leading to improved performance in complex decision-making tasks. MCTS was very quickly adopted into a variety of domains, including game playing, sample-based planning, and optimization problems, where these vast search spaces are common (Browne et al., 2012). More recently, MCTS has also seen a lot of success when combined with deep learning, and such hybrid cases will be discussed in the next section.

A significant advantage of MCTS over Beam Search is its ability to balance exploration of the search space with exploitation of promising paths in a more natural way. Instead of maintaining a fixed set of candidates, MCTS adapts its search strategy to focus more on promising regions of the search space as it accumulates more information. Through repeated simulation, MCTS gradually builds confidence about the value of different choices, enabling more sophisticated long-term planning. However, standard MCTS implementations face challenges in continuous domains, where the state and action spaces cannot be easily discretised.

2.4.3 Continuous Monte Carlo Tree Search

Continuous Monte Carlo Tree Search (C-MCTS) extends traditional MCTS to handle continuous state and action spaces, making it particularly suitable for exploring continuous domains, such as the GMM outputs in this thesis. Several approaches have been developed to handle continuous spaces in MCTS (Lei et al., 2021), but the most relevant to this thesis will be focused on.

Progressive Widening addresses the challenge of infinite branching factors in continuous spaces by gradually increasing the number of child nodes as the parent node receives more visits. Formally, if a node has been visited n times, the number of allowed children is limited to kn^α , where k and α are tuning parameters. When new children are not permitted, standard UCT selection is used to select a child node. This allows for promising nodes to be explored sufficiently before adding new children. When a new child node can be generated, it can either be randomly sampled from the probability distribution of the action space, for example the GMM output of an MDRNN in the case of MCTS-IMPSY, or can use a modified probability distribution influenced by priorities such as uniqueness, exploration, and exploitation.

Kernel-based methods use kernel functions to generalize rewards across similar states and actions in continuous spaces. Rather than treating each continuous value as entirely distinct, kernel methods establish similarity metrics that allow information sharing between nearby points in the parameter space. This provides an advantage over standard progressive widening as it doesn't simply reduce continuous domains back down into discrete search, but instead maintains an understanding of nearby points in a continuous space being related. However, this comes at the cost of increased computational complexity. Kernel Regression Upper Confidence Trees (KR-UCT) and their use in MCTS selection are discussed by Yee et al. (2016), and an advanced kernel-based selection strategy which modifies the favoured balance of exploitation, exploration, and uniqueness over time called KR-AUCB (Lei et al., 2021) has more recently been developed for autonomous driving.

Bayesian optimisation methods take the techniques of kernel-based methods a step further. Instead of only using a kernel function to smooth nearby values in parameter space, Bayesian optimisation can provide estimated heuristic values for the entire continuous parameter space of a GMM based on sampled values. This allows for a more accurate estimate of the value of a given action, but with even higher computational cost than kernel-based methods. It has been primarily implemented for solving partially observable Markov decision processes (POMDPs) (Mern et al., 2021). But has also been used as an expansion strategy for C-MCTS search (Lei et al., 2021).

Since the implementation of C-MCTS in this thesis uses has one continuous dimension, the computational costs of both kernel regression and Bayesian optimisation are too high, and so only standard progressive widening is used. However, for instruments with more continuous control dimensions these methods could be helpful.

2.5 Applications of Combined Machine Learning and Search

The integration of machine learning prediction models with search algorithms has emerged as a powerful AI method across multiple domains. By combining the pattern recognition capabilities of neural networks with the structured exploration of search algorithms, these hybrid approaches achieve results that surpass what either technique could accomplish independently. These examples demonstrate how the combination of prediction and search has been successful in other domains, and provide valuable insights into how these successes could be applied to interactive musical prediction systems.

2.5.1 Game Playing AI

Game playing systems have demonstrated perhaps the most obvious successes of combining neural evaluation with tree search methods, achieving superhuman performance in games once thought to be beyond computational reach. AlphaGo, developed by [Silver et al. \(2016\)](#), combined deep convolutional neural networks for position evaluation with Monte Carlo Tree Search for action selection to defeat Lee Sedol, considered one of the greatest ever Go players, in a five game match. This hybrid approach was further refined in AlphaGo Zero ([Silver et al., 2017](#)), which learned solely through self-play, eliminated human knowledge entirely and demonstrating even stronger performance, and then AlphaZero ([Silver et al., 2018](#)), which additionally displayed strong performance in both Chess and Shogi. Monte-carlo methods were later also used to develop Pluribus, the first superhuman level six-player texas-holdem poker AI ([Brown and Sandholm, 2019](#)). In these systems, MCTS was used primarily as a policy improvement operator, boosting the performance of an already strong neural network by looking deeply into the consequences of its predictions, or to learn from self-play. While MCTS-IMPSY doesn't use MCTS for policy improvement in the same way, the goal of this thesis is still to use MCTS to improve the performance of a neural network, and so the success of these systems provides strong evidence for the potential effectiveness of this approach.

2.5.2 Natural Language Processing

Natural language processing systems have increasingly adopted hybrid approaches that combine the human imitation of neural language models with the structured guarantees provided by search and grammar constraints. Neural language models excel at capturing patterns in language, but may generate outputs that violate syntactic or semantic constraints. Search algorithms guided by these neural models can explore the space of possible outputs while ensuring adherence to formal rules. Beam search has become the main decoding strategy for neural machine translation and text generation systems ([Sutskever et al., 2014](#)). In Google's Neural Machine Translation system, sophisticated beam search with length normalization and coverage penalties was a significant factor in the quality of their results ([Wu et al., 2016](#)). [Hokamp and Liu \(2017\)](#) extended this approach with grid beam search, which enforces lexical constraints while generating text. The success of hybrid NLP systems suggests that similar techniques could effectively balance learned

patterns with formal structure in the musical domain.

2.5.3 Robot Motion Planning

Robot motion planning and autonomous driving systems face particularly stringent real-time constraints while requiring both the adaptability of learned models and the guarantees of structured search algorithms. For motion planning problems, regular search techniques such as A* cannot generate effective paths, as during the execution of the path the environment around the agent will be dynamically changing in an unpredictable way, which can affect the validity and optimality of generated paths. Hence, because the states after executing an action are non-deterministic, RNN are used to generate predicted nodes for each point in the search tree. These RNNs are good at accounting for the complex dynamic environment, and can provide reasonable predictions for what some realistic future scenarios could be after executing a set of actions. The agent can then search through these RNN created environmental prediction trees, often using MCTS, to find paths which more effectively account for the unpredictable environment. This is a similar process to the algorithm in this thesis.

One hybrid method used in this space is to use an RNN to convert a non-deterministic partially-observable problem into a more simple deterministic problem which can be solved by a standard planner. [Karkus et al. \(2017\)](#) developed QMDP-net, which embeds planning within a neural architecture, integrating perception mechanisms with planning to allow for planning algorithms to better match the true environment. An inverse methodology is to reduce the action and state space to discrete actions for performing a search or planning algorithm, using a neural network at the end of simulation nodes to evaluate states. This approach is similar to AlphaGo ([Silver et al., 2016](#)), and is used effectively for motion planning by [Weingertner et al. \(2020\)](#).

The most relevant approach to MCTS-IMPSY is where a neural network is used to generate action probability distributions; either of the agent itself, of other agents, or of the environment; and then MCTS is used to search through this generated action space. One example of this is the deep-MCTS algorithm by [Chen et al. \(2020\)](#), which generated control commands for autonomous vehicles by using a combination of two deep neural networks and MCTS. The first neural network was the vehicle state prediction network, which predicted the future state of the vehicle given a current state and performed action. The second neural network was the action value prediction network, which for all possible actions provides action probabilities and expected rewards. This latter network guides the selection and expansion of nodes in the MCTS tree, which then searches predicted positive actions over multiple time steps to determine which action to take. While this approach is already very similar to that of MCTS-IMPSY, the KB-Tree algorithm by [Lei et al. \(2021\)](#) is even more similar, as it uses an MDRNN to generate a GMM as an action probability distribution, and then uses C-MCTS with progressive widening to search through the generated action space. KB-Tree outperformed the state-of-the-art in both reinforcement learning and MCTS-based techniques in simulated parking tests.

2 Background

The similarity of these mentioned approaches to this thesis is notable, as while the domains are entirely different, the techniques used are very similar and the results are very promising. Given that these approaches are applicable to interactive musical prediction systems in a similar way, as this domain faces similar challenges to motion planning, it seems that the lack of research into applying these methods to musical prediction is a significant gap in the literature.

2.6 IMPSY

While this thesis pulls from a diverse range of background knowledge, as has been explored in previous subsections, it directly builds upon IMPSY, developed by [Martin and Tørresen \(2019\)](#). Hence, an understanding of IMPSY; including its design purpose, strengths, and weaknesses; is critical to understanding the motivation behind MCTS-IMPSY.

IMPSY is a real-time interactive NIME that uses an MDRNN based approach to generate musical predictions. It is designed to create collaborative improvisations with humans, and individual MDRNN models for various instruments and performer styles can be trained quickly by feeding logs from human performer improvisations on the instrument of choice into the training algorithm. Specifically, the model is trained to predict control inputs to the instrument, not its outputs, meaning that it predicts the values of instrument controls rather than directly predicting their output. This provides the benefit of the model being agnostic to the mapping strategy from instrument controls to musical output. IMPSY also has multiple different collaborative modes, such as a battle mode where a human user and the MDRNN play simultaneously, but the interaction mode focused on within this thesis is the call and response mode. In this mode, the MDRNN is conditioned while the user plays, and begins to generate notes itself only when the user stops, letting the user take over again whenever they begin to use the instrument. The use of an MDRNN within IMPSY solves the problem traditional RNNs have of only being able to output discrete predictions. By outputting a probability distribution of potential control values for the next note, randomness is added into predictions, reducing the overly rigid and robotic playing style of many traditional RNNs. However, this randomness isn't a perfect solution, as it is arbitrary. Humans don't just add pure gaussian randomness when improvising, such as the type that MDRNNs generate. Instead they add controlled, guided, and meaningful randomness, which balances exploration of new ideas to create a unique performance, with exploitation of known musical concepts to ensure the performance is coherent and structured.

This thesis is an attempt to more closely replicate this human improvisation process. While MDRNNs have achieved great progress in the field by combining the exploitative advantages of RNNs with the exploratory advantages of MDNs, these MDNs are unguided in their exploration. Research into other fields, particularly motion planning, has shown that guiding neural network predictions with search algorithms can lead to stronger outputs. Hence, this thesis aims to combine the randomness of MDNs with

the structure of heuristic-based search algorithms to create a system that can generate more expressive and coherent musical predictions. Figure 2.10 shows how the MDRNN-IMPSY prediction process compares to the MCTS-IMPSY prediction process. Note that the replacement of stochastic selection with a guided approach is closer to the human improvisational process, because humans don't select randomly from note options they consider viable. Notes are selected with reason, whether that reason is explicit or intuitive.

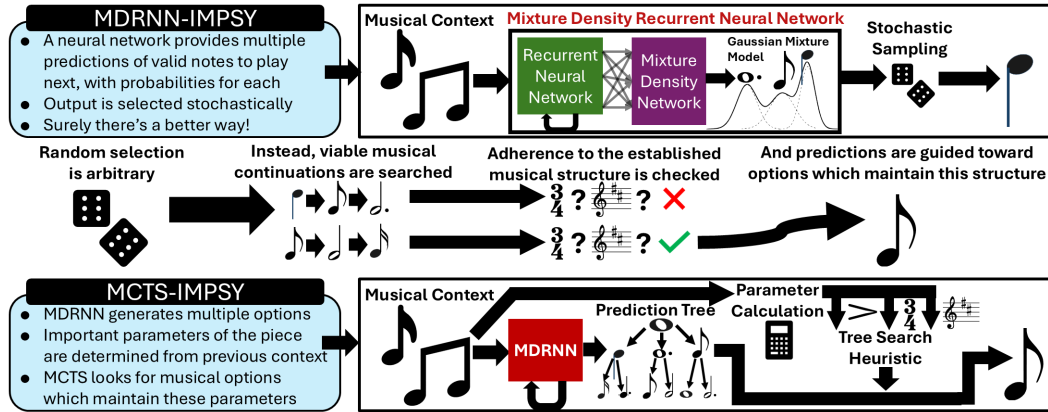


Figure 2.10: Comparison of the MDRNN-IMPSY prediction process to the methodology and outline of the MCTS-IMPSY prediction process.

System Design

This chapter describes the main content of this thesis. It begins by providing an overview of the problem MCTS-IMPSY is trying to solve, defining components necessary for understanding this problem, and explaining the scope of this problem tackled in this thesis. An overview of the new prediction algorithm created to solve these problems is then provided, before going into more depth on each the components of this algorithm, including details on their implementation. This includes the Monte Carlo Tree Search (MCTS) algorithm and how it has been integrated into IMPSY, the musical parameter used to create components of the heuristic, and the meta-parameters used to tune the system. The code for MCTS-IMPSY can be found on Github¹.

3.1 Problem Formulation

It is important to clearly define the problem this system is trying to solve in a more formal manner than previously, before explaining the system itself. MCTS-IMPSY is built as a modification of the musical prediction system IMPSY (Martin and Tørresen, 2019), which is a system that uses an MDRNN to produce a GMM of possible next notes, with this GMM being randomly sampled from to produce a note. The call and response mode of IMPSY is mode explored in this thesis, where the user performs for a period of time, and then when the user stops, the system takes over and aims to predictively continue on from where the user left off. When the system takes over, the MDRNN prediction algorithm is triggered. The input for this MDRNN algorithm is the previous control input played, which is an n dimensional vector representing the state of control parameters for an instrument, necessarily including a time dimension representing the length of time between this control input and when the next control input should happen. Formally, this is referred to as the time interval between control

¹<https://github.com/squatter1/impsy>

3 System Design

inputs, but since in practical terms for this system it is equivalent to note duration, this term will also be used to describe the time interval dimension. The input state also implicitly includes memory of previous control inputs through the LSTM RNN architecture. The output of the MDRNN is a random sample from an n dimensional GMM, with any sample from the GMM representing a control input for the instrument. The problem formulation for MCTS-IMPSY is the same as for MDRNN-IMPSY, with the main goal being to improve the prediction process, not to fundamentally change the functionality of IMPSY. Figure 3.1 shows the original MDRNN-IMPSY prediction algorithm, and Figure 3.2 highlights how the MCTS-IMPSY prediction system design incorporates and expands upon this prediction process.

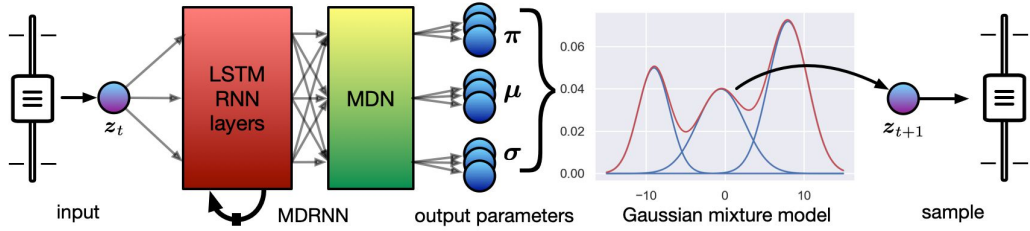


Figure 3.1: Reproduction of Figure 2 from *An Interactive Musical Prediction System with Mixture Density Recurrent Neural Networks* (Martin and Tørresen, 2019) showing the MDRNN-IMPSY prediction algorithm.

3.1.1 Research Scope

This thesis aims to serve as a proof of concept for the use of combined MDRNN and MCTS musical prediction systems, and so the scope of this thesis has been limited to a single instrument, a monophonic 2D keyboard. While this system design is generalisable to other instruments, testing on a simple instrument increases focus on the core components of the system without adding unnecessary complexity. The first dimension is the required time interval dimension between control inputs, and the second dimension is pitch, which is rounded to the nearest semitone for output. The scope is also limited to a single user, myself, a trained musician who has experience with improvisation. Ideally, a larger user study would be conducted, but this determined to not be a practical option for this thesis, and is left to future work. For the evaluation of the system, two separate datasets are used to train separate models on the keyboard, and so breadth of style within the limits of a 2D monophonic keyboard is within the scope of this thesis.

3.2 The Prediction Algorithm

The developed prediction algorithm is aimed at guiding the choice of the next control input, instead of randomly sampling it from the GMM. To do this, a tree of possible future control inputs over multiple predictive time steps is created, and the most promising branch of this tree to take based on a heuristic is chosen. This heuristic aims to

imitate a human performers checks and balances when improvising, ensuring that intuitive musical options, which are represented by the GMM, are also musically coherent. Hence, the heuristic is based mostly on musical parameters that psychological research, as well as musical studies, have shown to be relevant the perceptions and predictions of music by humans. The algorithm is designed to be flexible, with the exact heuristics used being customisable based on instrument, specific goals of performance, or musical styles. All but one of the parameters used in the heuristic are generalisable to any instrument with a continuous pitch and time dimension, with the exception, the key and modal parameter, having the only additional limitation of a discrete tuning system. This is a benefit of choosing a lower dimensional instrument, and allows the heuristic configuration to act as a baseline for more specific ones to build upon. Figure 3.2 shows the overall structure of the MCTS-IMPSY prediction algorithm, and each referenced component will be explained in more detail in the following sections.

3.2.1 Monte-Carlo Tree Search

To generate and search through a tree of predictions, the Monte Carlo Tree Search (MCTS) algorithm is used. MCTS builds a search tree incrementally and asymmetrically, focusing on the most promising parts of the search space. This starts with the root node of the search, which is z_t , the previously output control state. From this the goal is to output a z_{t+1} , which will be one of the children of z_t expanded in the MCTS prediction tree. Each z_{t+1} is assessed by the heuristic values of its explored potential future expansions. The MCTS algorithm consists of looping a sequence of four main steps: selection, expansion, simulation, and backpropagation; until a set time or iteration limit for search is reached. By iterating through these steps repeatedly, MCTS gradually builds a search tree and improves its predictions. The algorithm’s effectiveness comes from its ability to focus computational resources on the most promising parts of the search space, and at the end of search, the most visited child node of the root node is selected as the best action to take and output as z_{t+1} . Each of these steps will now be explained in more detail. A pseudocode version of the algorithm is shown in Algorithm 1.

Selection: Starting from the root node, the algorithm selects successive child nodes until a leaf node is reached. The MCTS-IMPSY selection process uses a modified version of the Upper Confidence Bound for Trees (UCT) algorithm (Kocsis and Szepesvári, 2006), which is a popular method for balancing exploration and exploitation in MCTS. The original UCT algorithm selects child nodes based on the formula:

$$UCT = \frac{w_i}{n_i} + c\sqrt{\frac{\ln N}{n_i}} \quad (3.1)$$

where w_i is the total reward of the child node, n_i is the number of visits to the child node, N is the total number of visits to the parent node, and c is a constant controlling the exploration-exploitation trade-off. The algorithm selects the child node with the highest UCT value. This means that it is choosing the child node with the highest average reward after simulating a potential future path. UCT has been modified for MCTS-IMPSY in two ways.

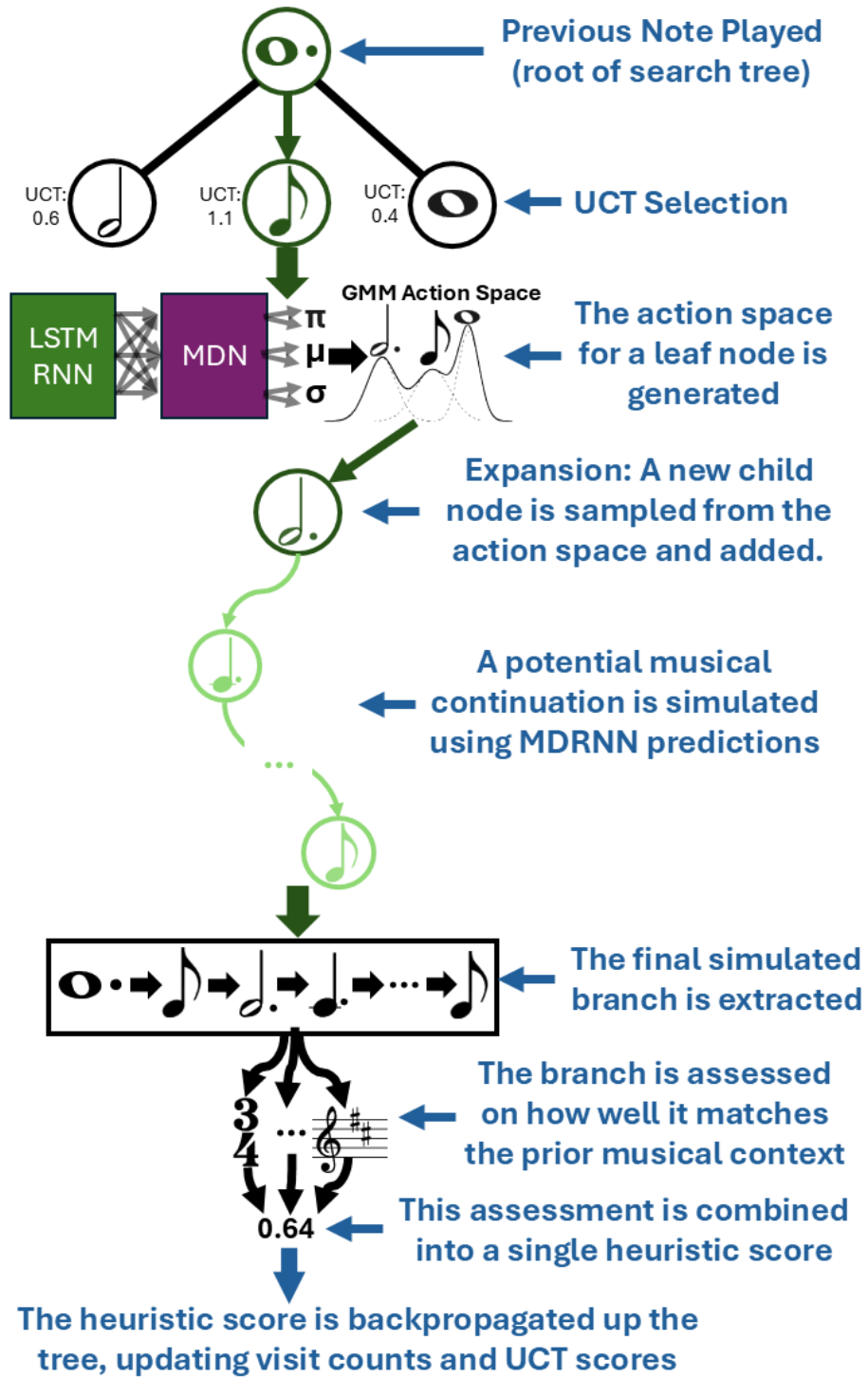


Figure 3.2: The MCTS IMPSY prediction algorithm.

3.2 The Prediction Algorithm

Firstly, since the action space for the children of each node is a GMM and hence continuous, the action space for selection needs to be made discrete. This is done by using progressive widening. Instead of expanding all children at once as is usually the case when there is a finite number of children, progressive widening only expands one randomly sampled action from the action space when first expanding a node. Additional nodes are added during selection if a node reaches a threshold of visits. The number of children a node z is permitted, N_z , is given by the formula:

$$N_z = kn^\alpha \quad (3.2)$$

where n is the number of visits to the node, and k and α are constants that control the growth of the action space. Whenever the calculated N_{z_t} is greater than the actual number of children of z_t , the expansion process is triggered and a new child node is added if possible.

It is important to note the aspects of progressive widening often used in other C-MCTS implementations that are not used in MCTS-IMPSY. The most common of these are kernel regression based selection strategies such as KR-AUCB (Lei et al., 2021). These selection strategies use modified w_i and n_i values to account for the fact that in a continuous action space where similar actions have similar rewards, you can have some knowledge of the reward of an action based on the rewards of its peers. w_i and n_i hence partially take on the values of neighbouring actions. Given that during the expansion process uniqueness for duration is heavily favoured, and pitch is necessarily substantially unique, the benefit provided by kernel regression based selection strategies is minimal and outweighed by the computational cost. It is likely that a model trained on an instrument with more continuous dimensions could reach a point where kernel regression based selection strategies would be beneficial, and so it is worth mentioning here.

The second modification to UCT is based on the fact that UCT optimises entirely for the average reward of a child node. This is ideal for adversarial games, where another agent is fighting against you and so the average is the best you can hope for. It is also beneficial where randomness plays a large role, as an average scenario is a reasonable assumption where significant randomness is at play. However, the search tree is not completely controlled by random processes, as while MDRNN sampling is random, the prediction function is not. This means that when selecting the next node at the end of search, the part of the tree under that chosen node is retained, meaning that the best found path can realistically be followed. This allows us to instead aim for paths with the maximum reward. Switching entirely to a greedy strategy would not be ideal, as it would lead to local optima and a lack of exploration. Instead, a modified version of the UCT algorithm is used which combines a weighted average of the best child node and the average child node. This is done by modifying the UCT formula to:

$$UCT = \beta w_{best} + (1 - \beta) \frac{w_i}{n_i} + c \sqrt{\frac{\ln N}{n_i}} \quad (3.3)$$

where w_{best} is the best heuristic evaluation found by the child node, and β is a constant that controls the weighting between the best child node and the average child node,

3 System Design

which balances greediness in the exploitation term. This allows the algorithm to focus on the most promising future directions, while still exploring other child nodes to avoid local optima.

Expansion: After reaching a leaf node, the MDRNN prediction algorithm is run to generate a GMM of possible next control inputs. This GMM is saved for the node and sampled once to generate a new child node. This child node is then added to the tree and is the starting point for the simulation process. The generated GMM will be reused for further expansion whenever progressive widening is triggered during the selection process.

During progressive widening, c candidate actions are randomly sampled from the GMM. Each of these candidates are checked to ensure a distance of at least ϵ away from all existing child actions. As each dimension in the action space represents a different control parameter, different ϵ values are applied to each dimension, and a candidate action must be unique in at least one of them. ϵ also doesn't have to function as a linear distance metric, multiplicative distances and other equations can also be used. If at least one such unique action is found, then the most unique is added, where uniqueness is the sum across all dimensions of the number of ϵ multiples away from the closest existing node, with a scaling factor applied to each dimension to account for different scales of uniqueness. If no candidate action is found to be unique, progressive widening fails and no new child node is added. This is done to ensure that the search tree does not grow without reasonable exploratory potential. If progressive widening is attempted and failed 5 times in a row, it is presumed that all potential actions with any reasonable likelihood of occurring have already been added to the search tree, and so progressive widening will be disabled for this node for the remainder of the search. Figure 3.3 visually explains the progressive widening process.

Another component of progressive widening used for expansion in the literature is Bayesian optimisation (Mern et al., 2021). Bayesian optimisation aims to account for three key factors when expanding the action space. Firstly, which part of the action space is the most likely to be optimal based on previous searches around the area, secondly which part of the action space is the most uncertain, and thirdly how unique the candidate action is from pre-existing ones. MCTS-IMPSY accounts for uniqueness, but does not account for uncertainty or optimality. This is because nodes which were regularly visited would soon begin to fail progressive widening, meaning that no more unique actions could be added for which the MDRNN assigns a reasonable likelihood. This means that the entire action space is effectively already explored except for actions which are extremely unlikely or not unique enough, and hence all optimal and uncertain areas are explored without the need for a computationally expensive Bayesian optimisation process. As with kernel regression based selection strategies, it is likely that a model trained on an instrument with more continuous dimensions or a higher number of gaussians in the MDRNN model could reach a point where Bayesian optimisation would be beneficial, but this is not the case for the model used in this thesis.

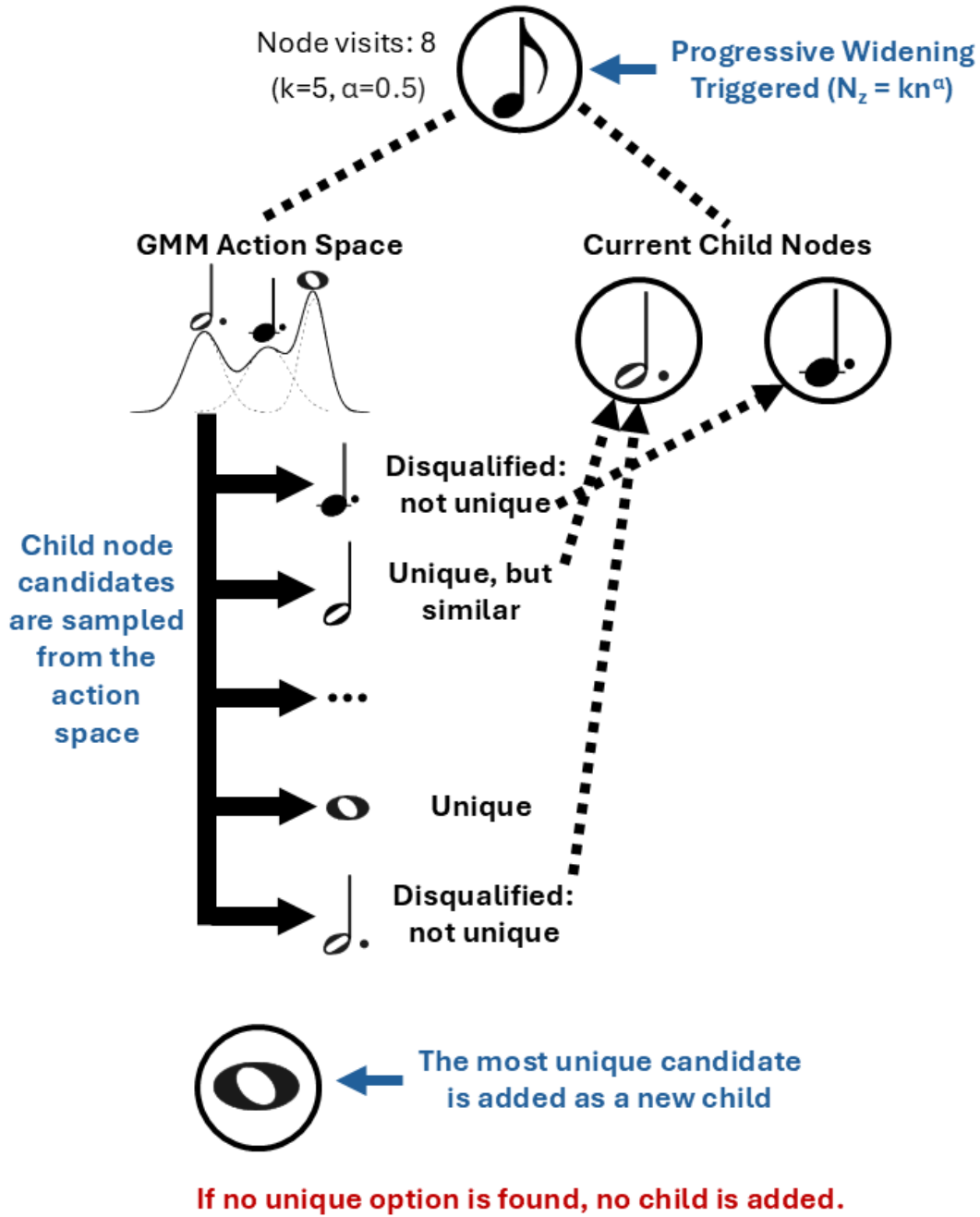


Figure 3.3: The progressive widening process in MCTS-IMPSY.

Simulation: Typically in MCTS a simulation takes place from an expanded node all the way to a terminal state using a simplified or random expansion process, and the result of that terminal state is then backpropogated. However, musical prediction and impro-

3 System Design

visation does not have a terminal state, and IMPSY will continue predicting indefinitely until the user manually exits the program. Hence, a set simulation depth constant S_d is used to ensure the simulation ends, and a heuristic is then run on the final state of the simulation to be used as a value for backpropagation. The simulation expansion strategy is to use default IMPSY predictions, with repeated MDRNN predictions with a random GMM sample until the simulation depth is reached. This is a computationally expensive simulation process, and so the optimal additional simulation depth after expansion was found after testing to be only 2 for search times of 100ms. This is unsurprising given the low number of iterations that can be performed within this time and the relatively expensive cost of additional simulation depth.

Backpropagation: After the heuristic is run, the result is backpropogated through the tree beginning from the expanded node. The backpropagation process updates the visit count and value of each node in the path from the expanded node to the root node, as well as updating the new best heuristic value for the node if the new heuristic value is better than the previous best. These updated values are then used in the selection process, including to determine whether progressive widening should be triggered.

3.3 Heuristic Components

For the C-MCTS algorithm to be useful, it must be guided by heuristics which detect beneficial musical branches. The heuristic in this system is based on a set of structural parameters which are measured as the user plays, and that the heuristic then attempts to emulate. To do this, the heuristic takes in the parameter values at the end of human play, and also measures the parameter values for the simulated branch of the search tree. The negative of the sum of the differences between these is returned, as the system maximises heuristic values and high levels of difference in parameters is unideal. Each of these parameters is based on psychological and musical research which has found these elements of composition to be parameters by which humans either intuitively or through musical learning understand musical structure and coherence and predict the future direction of music. They provide a representation for the current state of the piece a user is performing, providing a goal to which the system can aim to replicate. By guiding the musical prediction process toward outputs that humans have been shown to predict as more likely, one of the hypotheses of this thesis is that the predictive accuracy of the system can be improved. Given that the human ability to predict music is one of the core aspects of its enjoyability (Huron, 2008), another hypothesis is that this will maintain musically pleasant results which do not limit creativity and variation.

By virtue of the choice of test instrument, all but one of the parameters described here are generalisable to any instrument with a continuous pitch and time dimension. The exception is the key and modal conformity parameter, which is specific to 12TET instruments. This is added as an example of how a parameter can be added into the system to fine tune it to a specific instrument.

Algorithm 1: C-MCTS Prediction Algorithm

```

1 Function MCTSSearch(memory, heuristicFunction, timeLimit):
2   endTime  $\leftarrow$  currentTime + timeLimit
3   while currentTime < endTime do
4     selectedNode  $\leftarrow$  SelectAndExpand(root)
5     simulationResult  $\leftarrow$  Simulate(selectedNode, memory,
6       heuristicFunction)
7     Backpropagate(selectedNode, simulationResult)
8   end
9   return mostVisitedChild(root)
10 Function SelectAndExpand(node):
11   /* Select the best child node using UCT, widening the action
12     space where necessary */
13   current  $\leftarrow$  node
14   while current.children is not empty do
15     ApplyProgressiveWidening(current)
16     current  $\leftarrow$  UCTSelectBestChild(current)
17   end
18   /* Expand final selected node */
19   current  $\leftarrow$  ApplyProgressiveWidening(current)
20   return current
21 Function Simulate(node, memory, heuristicFunction):
22   current  $\leftarrow$  node
23   for i  $\leftarrow$  1 to simulationDepth do
24     current  $\leftarrow$  SampleFromGMM(current.gmm)
25     current.gmm  $\leftarrow$  PredictWithMDRNN(current)
26   end
27   return heuristicFunction(Concatenate(memory, ExtractBranch(current)))
28 Function Backpropagate(node, result):
29   current  $\leftarrow$  node
30   while current is not null do
31     current.visits  $\leftarrow$  current.visits + 1
32     current.value  $\leftarrow$  current.value + result
33     current  $\leftarrow$  current.parent
34   end

```

3 System Design

To visually explain the relevance of this heuristic guidance, examples have been generated which display how the C-MCTS algorithm can be guided. Firstly, Figure 3.4 displays a tree searched with a null heuristic. This showcases potential musical directions that IMPSY would explore without a modified prediction system. Over multiple notes this becomes a wide breadth of paths due to the randomness inherent within IMPSY’s predictions. As this example shows, at the particular place in the piece where the prediction algorithm has been run, a long note has been played, and the system seems to consider two reasonable future directions, continuing briefly to play longer notes before moving to shorter notes, and returning immediately to shorter notes. Hence, despite the multitude of potential paths explored, all of them lead to shorter notes within six note predictions. In Figure 3.5, the same C-MCTS prediction is shown, but now using a very basic heuristic to guide the search, which favours the lowest standard deviation of time interval from the root node. This heuristic is not particularly useful in a practical sense, but provides a stark visual difference within the search tree, which is helpful for visualising the power of directed search. When guided toward paths which maintain a similar long note duration to the root note, the entire search tree shifts towards options which continue along this path. In Figure 3.6, the most visited path of the search tree from Figure 3.5 is shown. This is the path that would have been taken given the heuristic guidance, and, while remaining in the realms of potential musical directions in early depths, it eventually significantly diverges in musical path from the unguided search tree. These redirections of musical path which remain subtle and entirely plausible note by note, but over a longer time period adhere to heuristic guidance, is at the core of what MCTS-IMPSY achieves. By guiding the search tree toward plausible musical paths, the system is able to maintain a coherent musical direction while still allowing for the creativity and variation afforded by MDRNN predictions.

Each of the following subsections details one of the musical parameters used to create a component of the final combined heuristic, along with a brief justification for its importance and a description of its implementation. All of the output values of these parameters are normalised to typically remain in a range of 0 to 1, and are later given multipliers based on how well they perform for a particular model, the assignments of which will be discussed in the evaluation chapter. The n last human notes played, referred to as the output memory, is kept and used to determine current parameter values to aim toward.

3.3.1 Key and Modal Conformity

Key and modal conformity measures how well musical branches adhere to the established musical scale and mode in the output memory, if such a scale and mode clearly exists. Temperley (2008) explores this through the concept of a key profile, noting that keys and modes are important factors in human predictions of musical continuations, particularly in Western music. The key profiles model is based on two observations about melodies which conform to a particular key. Firstly that notes within that key are significantly more likely to be used than those outside of it, and secondly that notes of

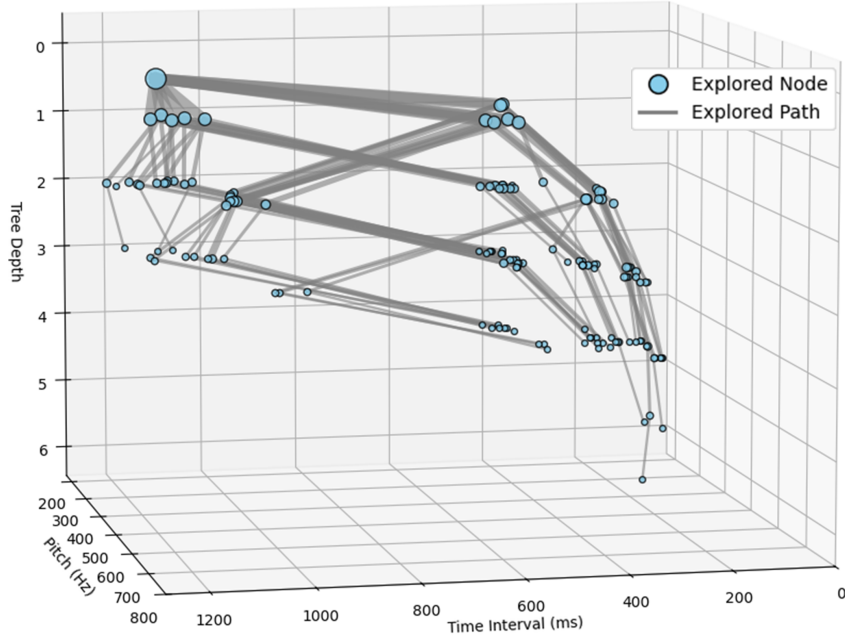


Figure 3.4: C-MCTS algorithm with a null heuristic. Nodes with ≥ 3 visits are shown, with logarithmic size based on visit count.

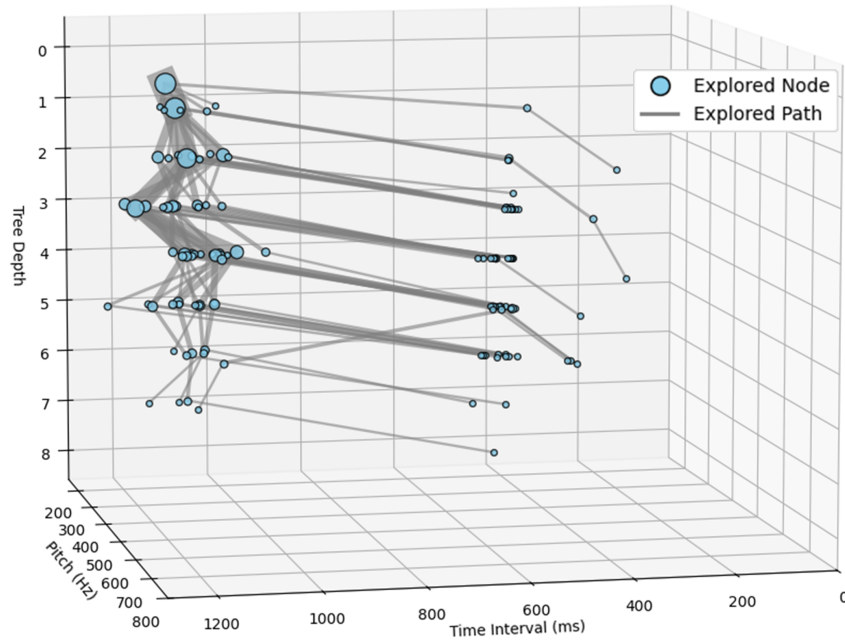


Figure 3.5: The same C-MCTS prediction as Figure 3.4, but with a heuristic favouring the lowest standard deviation of time interval from the root node.

3 System Design

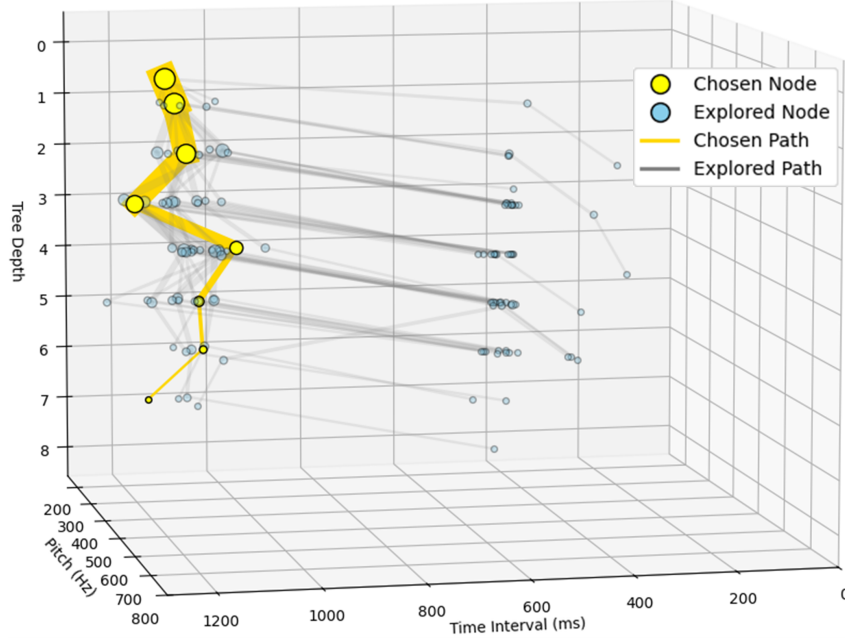


Figure 3.6: The guided C-MCTS search from Figure 3.5 with the most visited path highlighted.

the tonic triad, the root, third, and fifth of the scale, are slightly more likely to be used than other notes within the key. However, the likelihoods of each note are combined into a single key profile in this paper, which only accounts for natural major and minor scales, showing the increased use of the sharp sixth and seventh degrees in minor scales as small increases in usage within a natural minor key profile, instead of recognising the distinction of melodic and harmonic minor scales, and completely ignoring the use of other modes and key types.

To address these limitations the key profiles methodology from the above paper is converted into two parameters, a key conformity and modal conformity parameter, which combine into one of the five heuristic components used in search. Key conformity doesn't account for a root note, and instead only looks at the relative use of notes within a given scale. Because of this it is a lot easier to include more varieties of scale, and this parameter tests against four common scale types: major, pentatonic, blues, and harmonic minor; although it is trivial to add more if relevant to a particular style. For each scale type, the algorithm considers all possible transpositions (root notes) and calculates a conformity value for the output memory using the formula:

$$\text{key conformity} = \frac{12k - n}{12 - n} \quad (3.4)$$

Where k is the proportion of notes in the output memory which belong to the scale and n is the number of notes in the scale being tested. This formula normalizes the confor-

3.3 Heuristic Components

mity measurement across scales with different numbers of notes, yielding values between -1 (complete non-conformity) and 1 (complete conformity), with a value of 0 being expected for a random sequence of notes. The system selects the scale type and root note combination that produces the highest conformity value, provided it exceeds a minimum threshold constant which through testing has been set at 0.7. If no scale exceeds this threshold, the system does not use key or modal conformity for that prediction tree.

The modal conformity parameter extends the key conformity parameter when a valid key match is found by determining which mode within the identified scale is being emphasized. This allows every potential mode of any scale to be checked, as opposed to only major or minor. For each potential mode the system establishes which notes are within the root triad and then calculates a mode conformity value for the output memory using the formula:

$$\text{mode conformity} = \frac{nk_t - 3}{n - 3} \quad (3.5)$$

Where k_t is the proportion of scale-conforming notes that also belong to the root triad and n is the number of notes in the scale. This once again normalizes the conformity measurement across scales with different numbers of notes and yields values between -1 and 1. The system selects the mode with the highest conformity value, provided it exceeds a minimum threshold constant which through testing has been set at 0.25. If no mode exceeds this threshold, the system does not use modal conformity for that prediction tree, only key conformity.

These initial calculations are performed for the output memory when the system first takes over performance and a new MCTS tree instance is created. Hence, when evaluating potential branches in the search tree, key and mode conformity values only need to be calculated against the established key and mode of the output memory, if such a key and mode was determined to exist. The returned heuristic value is calculated as:

$$\text{heuristic} = \left(|b_k - m_k| + \min \left(\frac{|b_m - m_m|}{d_m}, l_m \right) \right) \cdot \text{multiplier} \quad (3.6)$$

Where b_k and m_k are the branch and memory key conformity values, b_m and m_m are the branch and memory mode conformity values, d_m is a constant representing the weight of the node conformity compared to key conformity which through testing is set to 3.0, l_m is a constant limiting the effect of mode difference when a mode is completely different which was set to 0.15, and a multiplier scales the value of this heuristic component compared to others included in the final heuristic calculation.

This dual approach to key profiles allows the system to maintain consistency with the performer's key and mode if they are adhering to one, while not affecting results if it is not clear that they are. The algorithm also adapts to the performer's level of key adherence, preferencing occasional out of key notes if they are present in the output memory. The separation into two parameters also allows for more versatility, incorporating any desired scale in any mode. Unlike the other heuristics in the system, key and modal conformity is

3 System Design

specific to instruments using twelve-tone equal temperament (12TET) or another tuning system with discrete notes if adapted, as the existence of keys and modes relies on this. Hence, in a loose sense this is an example of how more instrument and style specific parameters can be integrated into the system to enhance its performance for particular contexts.

3.3.2 Tempo and Swing Conformity

Tempo and swing conformity measures how well musical branches adhere to the established tempo and swing in the output memory, if a constant tempo and swing clearly exists. Tempo and swing are fundamental rhythmic parameters in music that play a large part in rhythmic expectation and prediction. [Laroche \(2001\)](#) discusses this importance, and establishes a method for estimating the tempo (speed), swing (rhythmic inequality of eighth notes), and downbeat (the first and usually strongest beat of a measure) of a piece of music. This method takes an audio track as input, first establishing the location of notes from the audio, before performing analysis on these note locations to establish rhythmic parameters. Since MCTS-IMPSY receives note locations directly as control inputs, the first step of this process is skipped. Establishing downbeats also relies on the volume of notes, which is not available in the 2D keyboard implementation, and so only tempo and swing are included in this heuristic component, with a modified implementation.

The tempo and swing component heuristic operates by examining the durations between note onsets to determine the most likely tempo in beats per minute (BPM) and the type of swing being employed. The system evaluates these patterns by comparing actual note durations against expected durations that would occur in various tempo and swing combinations. For any given tempo, an algorithm calculates expected note durations for common rhythmic values, from sixteenth notes up to whole notes, as well as the dotted (1.5x length) variants of these notes. For swing, the standard eighth note is replaced with two swung eighth notes, a long and short note, with a ratio determined by their level of swing. For this model only triplet swing (2:1 ratio) is checked, as this is the most common type of swing in Western music, however, it is trivial to add more swing types if desired.

To determine the rhythm of the output memory, the algorithm scans across a range of tempos (default 70-138 BPM in 2 BPM increments) for both no swing and triplet swing, and calculates the average deviation score of the output memory. The deviation score for each note is calculated as:

$$\text{deviation} = \text{penalty} + \frac{|\text{actual} - \text{expected}|}{\text{expected}} \quad (3.7)$$

Where the penalty is a small additional cost applied to less likely but still possible note durations in a given tempo, such as dotted notes. The system also applies a penalty when consecutive long swing notes are detected, as this pattern typically indicates a

misclassification of the rhythm rather than actual swing. A tempo conformity score is then calculated for each tempo and swing option:

$$\text{tempo conformity} = 1 - \frac{\text{average deviation score}}{\text{maximum acceptable deviation}} \quad (3.8)$$

Where the maximum acceptable deviation (default 0.08) represents the threshold beyond which a performance would be considered rhythmically inconsistent. The tempo and swing value with the maximum tempo conformity score are selected as the best fit for the output memory, provided there is at least one positive score. If no positive score is found, there is no clear static tempo, and the system does not apply this component heuristic.

When evaluating potential branches in the search tree, the average deviation of the branch for the established tempo and swing of the output memory is calculated. For branches with a lower average deviation than the output memory, the heuristic returns zero, allowing other heuristics to guide the search. For branches that receive a higher score, the heuristic returns the following value:

$$\text{heuristic} = m_c \cdot (b_d - m_d) \cdot \text{multiplier} \quad (3.9)$$

Where m_c is the memory tempo conformity, b_d is the branch average deviation, m_d is the memory average deviation, and multiplier is the heuristic component scaling factor.

This approach scales the relative effect of this heuristic based on the level of tempo conformity in the output memory, as well as with an additional overall scaling multiplier. It rewards branches that best maintain the established tempo and swing of the output memory where this tempo and swing is clear, and does not have an effect where it is not, hence not negatively affecting performances where the performer is using variable tempo or swing.

3.3.3 Pitch Interval Markov Model

The Pitch Interval Markov Model component heuristic maintains melodic patterns by analyzing the intervals between consecutive notes. Early experiments into melody distortion by [White \(1960\)](#) tested which changes to melody most affected recognition, and established the importance of melodic contour and interval size. [Dowling and Fujitani \(1971\)](#) conducted more rigorous experiments on these factors in particular, and showed that there are three levels to melody recognition. The most fundamental of these is the maintenance of contour, or the direction of pitch movement from each note to the next. One step above this is the maintenance of intervals, or the distance between each note. The final level is the maintenance of absolute pitch, or in essence repetition of the same melody. Moving up each of these levels significantly improves melody recognition. These factors have later been used to establish their importance in melody prediction ([Temperley, 2008](#)), with it seeming that memory of the direction of pitch and intervals between notes in a melody significantly influence perceived probability of melodic continuations.

3 System Design

Hence, this component heuristic aims to recreate this prediction process as a measurable parameter.

Initial testing showed that melodic contour based heuristics provided minimal effectiveness, likely due to them being relatively simple and hence likely mostly encompassed already by the neural network model. Hence, the second level of melody recognition is the focus of this parameter, the maintenance of intervals. To replicate as best as possible the findings of [Temperley \(2008\)](#), a Markov model is used to predict the next interval based on intervallic movement patterns within the output memory. This component heuristic is set up to allow for any n-order Markov model, but a first-order model was found to be the most effective during testing and is the version which will be described here.

The model is constructed as follows:

- Extract the sequence of pitch values from the output memory and convert them to MIDI note numbers (integers from 0-127)
- Calculate the intervals between consecutive notes (the difference between adjacent pitch values)
- For each interval, record the frequency with which each possible interval follows that interval in the output memory
- Represent the model as a dictionary where the keys are the intervals and the values are dictionaries mapping each possible next interval to its frequency

For example, a key of 2 with a value of $\{1 : 5, -4 : 2\}$ indicates that after the melody moved up by 2 semitones in the output memory, there were five instances where it then moved up by 1 semitone and two instances where it moved down by 4 semitones.

When evaluating branches in the search tree, the branches are converted to interval sequences in the same way as the output memory. The heuristic then calculates the probability of each interval transition in the branch according to the generated Markov model. This is based on the frequencies in the Markov model, with Laplace smoothing applied to account for the fact that intervals not observed in the output memory should still have some probability of occurring. The formula for calculating the transition probability with smoothing is:

$$P(I_{t+1} | I_t) = \frac{M[I_t][I_{t+1}] + \alpha}{\sum_I (M[I_t][I]) + \alpha |M|} \quad (3.10)$$

Where I_t is the current interval, I_{t+1} is the next interval, and M is the Markov model dictionary, meaning that $M[I_t][I_{t+1}]$ is the count of how many times the next interval follows the current interval, $\sum_I (M[I_t][I])$ is the total number of transitions from the current interval in the model, and $|M|$ is the number of unique transitions observed from the current interval. α is the Laplace smoothing parameter, which through testing was set to 0.1.

The system calculates the average probability across all transitions in the branch and returns the inverse value (multiplied by a scaling factor) as the heuristic score:

$$\text{heuristic} = (1 - \text{average probability}) \cdot \text{multiplier} \quad (3.11)$$

This formulation ensures that branches with interval patterns similar to those in the performer’s playing receive lower heuristic scores (indicating better conformity), while branches containing unusual or unseen interval sequences receive higher scores. The interval Markov model is particularly effective at capturing the characteristic ‘vocabulary’ of melodic patterns a performer uses while allowing for creative variations within that style. By focusing on intervals rather than absolute pitches, the model generalizes across different keys very well, making it applicable to a wide range of musical contexts and styles, from simple diatonic melodies to complex chromatic passages. It is particularly useful during sections where repeated intervallic patterns are present, such as an arpeggio.

3.3.4 Time Multiple Markov Model

Rhythmic contour has been found to be at least as important as pitch contour in melody perception (Dowling et al., 1999). Similar experiments to those performed by Dowling and Fujitani (1971) for pitch contour have also been performed for rhythmic contour, and have shown similar results (Schmuckler and Moranis, 2023). Given the similarities between these two factors, for this heuristic a similar approach to the pitch interval Markov model heuristic is used. However, there are two important differences in model construction. Firstly, the time multiple Markov model uses the ratios between consecutive note durations rather than the difference between them. This is because it is relative proportions which determine rhythmic patterns and equate to the role of intervals in pitch. Technically, for pitch intervals ratios were also used, however since frequency values are converted to MIDI notes numbers, multiplicative distances are converted to additive distances. The second difference is that time operates on a continuous scale in the model, whereas pitch operates on a discrete scale. Hence for a Markov model to work, multiples between pitches must first be quantized to a discrete scale. These time multiples are quantized to the multiplicatively closest value in the set Valid multiples = {0.125, 0.25, 0.33, 0.5, 0.67, 1.0, 1.5, 2.0, 3.0, 4.0, 8.0}, with a small distance penalty for multiples outside of the set Common multiples = {0.25, 0.5, 1.0, 2.0, 4.0}.

The model is constructed as follows:

- Extract the sequence of time intervals from the output memory
- Calculate the multiple between consecutive time intervals (eg 1.2 seconds to 0.3 seconds would be 0.25)
- Quantize the multiples to the nearest value in the set of valid multiples, with a small distance penalty for less common multiples

3 System Design

- For each quantized multiple, record the frequency with which each other quantized multiple follows that multiple in the output memory
- Represent the model as a dictionary where the keys are the quantized multiples and the values are dictionaries mapping each possible next quantized multiple to its frequency

For example, a key of 0.25 with a value of $\{4 : 3, 3 : 1\}$ indicates that after a given note had a quarter of the duration of its predecessor, there were three instances where the next note had four times the duration of that given note, and one instance where the next note had three times the duration.

Once this Markov model is constructed, the process to evaluate branches is the same as for the pitch interval Markov model. The transition probabilities are calculated using the same Laplace smoothing constant of $\alpha = 0.1$, using the formula:

$$P(T_{t+1} | T_t) = \frac{M[T_t][T_{t+1}] + \alpha}{\sum_T (M[T_t][T] + \alpha |M|)} \quad (3.12)$$

Where T_t is the time multiple at time t .

The system then calculates the average probability across all transitions in the branch and returns a scaled inverse as the heuristic score:

$$\text{heuristic} = (1 - \text{average probability}) \cdot \text{multiplier} \quad (3.13)$$

The Time Multiple Markov Model is particularly effective at capturing rhythmic patterns, and can do so even when there is no strict tempo due to the note by note way that quantization is performed.

3.3.5 Repetition Markov Model

The Repetition Markov Model heuristic identifies repetition by combining pitch intervals and time multiples from the previous two heuristics into a single second-order Markov model. [Duane \(2016\)](#) shows that identifying repetition plays a key role in musical prediction probabilities, but also that to maintain originality and prominent sections within a piece, this repetition must be balanced with variation. To create this balance, strict rules are placed on when the repetition model comes into effect. No Laplace smoothing is applied, and the second-order model ensures that this heuristic will only influence the search tree when a clear motif is established through multiple note transitions in a row which are repeated. The maintainance of relative instead of absolute measures of pitch and time also allows for transposed motifs to be recognised, which is important for the system to be able to change keys and perform well in different octaves.

The model is constructed as follows:

- Extract the sequence of pitch values and time intervals from the output memory, converting pitches to MIDI note numbers

3.3 Heuristic Components

- Calculate the pitch intervals and quantized time multiples between consecutive notes
- For each tuple of (pitch interval, time multiple), record the frequency with which each possible next tuple follows that tuple in the output memory
- Represent the model as a nested dictionary where:
 - The keys of the outer dictionary represent a state, which is a tuple of two consecutive paired pitch and time values
 - The keys of the inner dictionaries are each individual paired pitch and time values which follow from that state in the output memory
 - The values of the inner dictionaries are frequency counts for these continuations

For example, a key of $((2, 1.0), (-3, 0.5))$ with a value of $\{(1, 2.0) : 3\}$ indicates that after an upward step of 2 semitones with equal duration followed by a downward movement of 3 semitones at half the previous duration, the next event was always an upward step of 1 semitone with twice the previous duration, and this specific pattern occurred three times in the output memory.

Unlike the other Markov models, the Repetition Markov Model does not apply Laplace smoothing to unseen transitions. Instead, it assigns a probability of zero to any state-transition pair that was not observed in the performer's playing. This design choice deliberately emphasizes exact repetition of musical motifs, and leads to the following transition probability calculation:

$$P(N_{t+1} \mid s) = \begin{cases} \frac{M[s][N_{t+1}]}{\sum_N (M[s][N])}, & \text{if the transition was observed} \\ 0, & \text{otherwise} \end{cases} \quad (3.14)$$

Where s is the current state, representing the current and previous note, and N_{t+1} is the next note.

The system calculates the average probability across all transitions in the branch and returns a scaled inverse as the heuristic score:

$$\text{heuristic} = (1 - \text{average probability}) \cdot \text{multiplier} \quad (3.15)$$

This formulation ensures that branches containing exact repetitions of motifs from the performer's playing receive lower heuristic scores (indicating better conformity), while branches with novel combinations receive higher scores. However, in instances where the performer has not established a clear motif, this component heuristic will not influence the search tree, allowing the rest of the heuristic to guide the search. This encourages the continuation of established motifs and thematic material to reinforce the structural coherence of the performance, but limits its potential negative impact on the originality of the generated music.

3.3.6 Failed Parameters

Some parameters were designed and tested but ultimately not included in the final system due to their ineffectiveness. However, they are worth mentioning given that being able to adapt to various different heuristics tuned to different control dimensions and styles are an important part of the system design, and so more general commentary on how to avoid ineffective heuristics is appropriate.

There were three parameters that were created and found to be ineffective. These were the pitch range, pitch height, and pitch proximity heuristics. These were all based on the work of [Temperley \(2008\)](#), who also informed the design of the successful key and modal heuristic and pitch interval Markov model heuristic, and of [Eitan and Granot \(2006\)](#) who conducted highly regarded work on musical parameters which discusses the importance of pitch height. Pitch range measured the overall range from the lowest to highest note in the output memory, pitch height measured the average pitch of the output memory, and pitch proximity measured the average distance between consecutive notes in the output memory. These were found by [Temperley \(2008\)](#) to be important metrics for melody perception, but they reduced the performance of MCTS-IMPSY when integrated into the heuristic. This is likely due to the fact that these parameters are relatively simple and hence likely already encompassed by the neural network model. Thus, they only serve to limit the MDRNNs predictions instead of constructively guiding them. This is a key lesson for future work: that overly simplistic parameters can be detrimental to the performance of the system.

3.3.7 Potential Heuristics for Other Control Dimensions

There are many potentially effective heuristics which were not included in MCTS-IMPSY due to them not being relevant to the 2D monophonic keyboard model. However, many other instruments will have varying control dimensions, each of which could include additional heuristics to encompass these dimensions. For example, loudness controls could lead to algorithms which perceive strong and weak beats, allowing for heuristics relating to time signatures and downbeat placement ([Laroche, 2001](#)). Timbre related controls could lead to mood related parameters being more accurate and hence relevant ([Hailstone et al., 2009](#)). And the addition of polyphony could lead to voice leading ([Wall et al., 2020](#)) and other harmony related parameters being more relevant too. One potential heuristic which was considered but not included was consonance, as although it has been found to be relevant for melody cognition ([Ahlbäck, 2004](#)), it is made redundant by the pitch interval Markov model heuristic. However, in a polyphonic setting it would no longer be redundant, and so could be included. Stylistic heuristics could also be added to the system if relevant, for example if the system was being used to perform in baroque counterpoint, a style where very specific rules must be adhered to ([Gauldin, 2013](#)), then heuristics could be added to enforce these rules. This is a core aim of MCTS-IMPSY: to be extensible and adaptable to different styles and instruments, and so heuristics can be easily added, removed, or modified as needed.

3.4 Overview of Meta-Parameters

Due to the amount of meta-parameters controlling the system, it was not possible to perform rigorous testing on the best possible combination of them. Instead, a series of smaller tests were performed on each parameter individually, some brief evaluations of multiple possible values were conducted, and some meta-parameters were adjusted based on the feel of the system when performing with it. Each of these meta-parameters will be covered here in turn, excluding those relevant only to specific component heuristics and hence mentioned in those sections. This serves not only to state the values used in the final system, but also to provide a summary of the relevance each of these parameters has to the system.

Firstly, there are the π and σ temperature meta-parameters used to tune the MDRNN. π temperature affects the weights of each gaussian in the gaussian mixture model, with lower temperatures significantly increasing the likelihood of the highest weight gaussian being selected, and higher temperatures flattening the distribution of weights. σ temperature affects the standard deviations of gaussians in a gaussian mixture model, with low values favouring the exact mean of each gaussian, and higher values providing more leeway from the mean of each gaussian. Short informal experiments were conducted to test prediction accuracy with different values of these parameters. The optimal values for both remained relatively unchanged from MDRNN-IMPSY to MCTS-IMPSY, with π temperature set to 1.5 and σ temperature set to 0.01. Given that the effect of higher σ temperature would create the opportunity for weaker predictions to be selected, and slightly better but too close predictions to be ignored during progressive widening, it is unsurprising that a low value remained ideal, if kernel regression was implemented in selection the optimal value could increase. For π temperature, due to the use of progressive widening, predictions from all gaussians were usually added as child nodes for well explored parent nodes, even with reasonably high differences in probabilities between them, and so lower π temperature mostly effects the order. Because there is some benefit to expanding more likely child nodes first in MCTS (Silver et al., 2016), but also a benefit to having high enough probability of less likely gaussians that they are eventually still explored, it is also not unusual that a moderate value of 1.5 remains optimal. The third relevant meta-parameter for MDRNN predictions is the number of gaussians the model generates for each GMM. This was set to 5, as this was the number used in the original MDRNN-IMPSY model, but other values were not rigorously tested. It could be that higher numbers of mixtures are useful for generating more diverse predictions in the search tree, and so this parameter is worth exploring in more depth in future work.

The next set of parameters are used for progressive widening. This includes α and k , which together form the check for whether a node z has enough visits to be expanded, $N_z = kn^\alpha$. The value of k is set to 2.5, and the value of α is set to 0.25. These were set mostly from qualitative testing of the system, and observations of the search tree, noting whether enough nodes were being expanded. These values provide a high initial expansion rate with a rapid decrease as the number of visits increases. This high initial

3 System Design

expansion rate was useful given the low number of MCTS iterations that are possible in each search, as adding strong child options early was essential for lowering bias. The low α providing a steep dropoff was due to the fact that only five mixtures were used in the GMM, and so numbers of children after this would often be sampled from the same GMM peaks, which begins to provide a lower amount of benefit. Other progressive widening meta-parameters are the minimum distance requirements from sibling nodes for a candidate child to be considered unique, the number of candidate actions generated for each progressive widening attempt, and the maximum number of progressive widening attempts before giving up. The minimum distance requirement was not relevant for pitch as it is not continuous for a keyboard, and was set to a variation of 8% for duration as qualitative testing showed that this was approximately the boundary at which note duration would be different enough to clearly be considered a different note. The number of candidate actions was set to 10, with a maximum of 5 attempts. This was set to a relatively low number as the system is designed to be able to perform in real time, and so the number of attempts must be limited. 10 candidate actions also provided a high chance at exploring multiple gaussians with the GMM action space. After 5 attempts at progressive widening, meaning at least 50 non-unique GMM samples in a row, any future added children would likely be from very low probability sections of the GMM which are unlikely to be useful, and all areas of the GMM with a reasonably high probability have very likely already been explored, and so this value seemed to be a good balance.

Another two meta-parameters are used for the UCT selection process, the greedy weight (β) and the exploration weight (c). Prediction accuracy for multiple β values were tested, and seemed to show that a value around 0.4 was optimal. This balances the use of the best found heuristic result almost evenly with the average heuristic result, which seems reasonable for avoiding local maxima while still exploiting known strong paths. The exploration weight was set to 0.1, which was enough to ensure that MCTS continues to balance exploration and exploitation, but still remains quite low so that higher search depths can be reached, which is important for getting accurate results from the branch based heuristic.

The final MCTS meta-parameter is the simulation depth S_d , the amount of additional nodes to simulate after expansion before using the heuristic. Values of 1, 2, 3, and 4 were tested, and a value of $S_d = 2$ was found to be optimal. Given the high computational search cost of simulation in MCTS-IMPSY, this low value is unsurprising.

The length of the output memory, the memory array used to store the last n notes played, is set to 45. Different values provided similar accuracy results in testing, but varied among each heuristic component, and favoured different styles of performance. Relatively low memory lengths (around 20) were found to be better for more structured styles, and favoured the key and modal and the tempo and swing heuristic components. Higher memory lengths (around 50) were found to be better for more freeform styles, and favoured the Markov model based heuristic components. This is likely due to more data leading to more complete and useful Markov models. Given that the system struggled more with freeform styles already, a higher memory length was selected to help to

3.4 Overview of Meta-Parameters

balance this. It could be useful to explore this further in future work, as using different memory lengths for each heuristic component, and adapting length based on other piece information, could potentially lead to better results.

The final meta-parameters are the weightings of each heuristic component in the final combined heuristic. However, since these were highly dataset dependent, and reliant on the results of the full evaluation to establish good values for, they will be discussed in the evaluation section.

Evaluation

Within NIME research, what constitutes and evaluation is much more broad than in many other fields. [Barbosa et al. \(2015\)](#) discusses this breadth, noting that there are not only many aspects within prediction systems that can be evaluated, but many perspectives from which to evaluate them from, many goals the evaluation could target, many potential criteria the NIME could aim to meet, and many approaches to employ. Their paper covers these aspects, and notes that all of these aspects should be both considered and explicitly addressed during an evaluation in order to ensure that it is both comprehensive and clear. Hence, these aspects and how they informed the design of the evaluation will be covered first, and then the results themselves will be presented.

- **Target aspect:** The main aspect of the system being evaluated is the output. The input, mapping, training, and methods of user interaction within the system have remained relatively unchanged from the prior MDRNN-IMPSY system ([Martin and Tørresen, 2019](#)), and so they will not be evaluated. Given that the change in prediction algorithm creates a relevant distinction from MDRNN-IMPSY only when it changes the output of the system, it makes sense for this to be the focus of the evaluation.
- **Perspective:** The evaluation takes place from the perspective of the designer and the performer, as a user of MCTS-IMPSY will be both a performer and a designer, given the level of customisation available in the system. From a designers perspective, the key technical metric MCTS-IMPSY aims to improve, the prediction capability, is evaluated. From the performers perspective, the system is evaluated in a brief qualitative trial, where improvisations are performed with the system and then analysed.
- **Goal:** The main goal is to evaluate the quality of the MCTS-MDRNN system output compared to MDRNN-IMPSY.

- **Criteria:** The criteria used varies by section. First the accuracy and speed of predictions are evaluated, and then the playability and expressiveness of the system are checked.
- **Approach:** Prediction accuracy and speed are evaluated quantitatively, whereas the playability and expressiveness of the system are evaluated qualitatively.

4.1 Prediction Accuracy

Prediction accuracy is the only relevant objective quantitative metric for evaluating the system. Given that the system is an interactive musical prediction system, its prediction accuracy is essential to its success. If the system is not able to predict the next note in a piece of music with any accuracy, then it will not be able to create a coherent improvisation. This section explores prediction accuracy against multiple datasets, each dimension of output, and each component heuristic. The detailed exploration means that this section provides the best overview of how each component within the system design contributes to the performance of the system as a whole. After the datasets and how they are evaluated is explained, prediction accuracy is evaluated for each output dimension, pitch and duration, and then for the output as a whole. This is followed by an analysis of each component heuristics contribution to the overall performance of the system, by ensuring that performance drops when they are excluded. Finally, prediction speed is evaluated, with the number of MCTS iteration compared against prediction accuracy, and being linked to search time.

All evaluations were conducted on a Dell G15 laptop with a 12th generation Intel i7-12700H CPU and 16GB of RAM. Since only CPU not GPU was used, and RAM was never fully utilised by the system, the processor is the main important element. The system was implemented in Python 3.11 using Tensorflow 2.15 for the neural network, with tflite models used for increased prediction speed.

4.1.1 Datasets

Two datasets were used to evaluate the system, the first is a dataset of 15 improvisations performed by myself. IMPSY is intended to function based on small models trained on personalised datasets of improvisations tuned to a specific performers style, and so this dataset is important for evaluating the system on one of its core intended use cases. The 15 improvisations contain a total of 11142 notes, and were performed as monophonic keyboard improvisations with the details of the model in mind. Cross-validation was used for all evaluations of this dataset, creating 15 models, each trained on 14 improvisations and evaluated on the remaining improvisation. This ensures that the model does not do well on evaluation metrics by simply overfitting to training data, and each improvisation had slightly different styles, tempos, keys and improvisational goals in order to further limit the possibility of overfitting.

The weighting of each heuristic component for this dataset was based on prediction

accuracy for individual heuristics and informal qualitative testing, and is as follows:

- Key and modal heuristic: 10%
- Tempo and swing heuristic: 3.3%
- Pitch interval Markov model heuristic: 66.7%
- Time multiple heuristic: 6.7%
- Repetition heuristic: 13.3%

The Nottingham Folk Music Database ([Foxley](#)) which is no longer available from its original source, was used as the second dataset. It was obtained from a GitHub repository containing a cleaned MIDI version of the dataset with separated melodies and chords ([jukedeck, 2017](#)). Because this dataset splits each piece into clearly defined chords and a monophonic melody, it is a good dataset to evaluate the system on. Only the melody part of each piece is used, for a total of 1034 pieces and 196961 notes, and convert them into log files containing only the two dimensions of note duration and pitch for training. This dataset is also useful as it is all within a similar style, and so is good for stylistic evaluation. From these, 28 random melodies of at least 200 notes were selected as a test set, for a total of 11237 notes. The rest of the dataset was used to train a single model. A larger model size was used for this dataset, which is another benefit of using this second dataset, as it allows testing of whether the system improves performance even for models trained on larger datasets. A notable limitation of this dataset is that it does not contain tempo information, and so random tempos are assigned to each piece.

The weighting of each heuristic component for this dataset was determined the same way as for the improvisation dataset, and is as follows:

- Key and modal heuristic: 12.2%
- Tempo and swing heuristic: 14.6%
- Pitch interval Markov model heuristic: 12.2%
- Time multiple heuristic: 12.2%
- Repetition heuristic: 48.8%

When evaluating a particular test piece from one of these datasets, the model takes the first 45 notes as its initial output memory, and then predicts the 46th note. Then the first note is dropped from memory, the actual 46th note is added, and the model predicts the 47th note. This process continues until the end of the test set, at which point it moves on to the next until all test sets have been evaluated. A prediction is considered correct if the exact correct pitch is predicted and the note duration is within an 8% margin of error from the exact correct value. This 8% figure was established through experiments with the tempo and swing heuristic, as from testing an average deviation of 8% seemed to be the limit for where a tempo was clearly perceptible. Hence, it made sense to also use this as the cutoff figure for duration deviation in the evaluation. This may provide

4 Evaluation

a slight bias in favour of the tempo and swing heuristics performance, as the evaluation metric matches how the heuristic was setup, but given that it was established through repeatable and logical testing and not a purposeful intention toward bias, it is not a major concern, and would have a very minimal affect on results anyway. The number of correctly predicted notes is tracked and divided by the total number of predicted notes to give a prediction accuracy percentage.

In the following subsections all predictions are conducted with a 100ms search time limit unless otherwise specified, to imitate a typical amount of search time which may be available in a real-time performance setting. All component heuristic evaluations are compared to two baselines, the predictive accuracy of the unguided MDRNN, as is used in MDRNN-IMPSY, and the combined heuristic, representing the performance of MCTS-IMPSY. It is important to note that the two datasets provide quite different scales of accuracy, with accuracy being much higher for the Nottingham dataset. This is likely due to the fact that the Nottingham dataset is a larger and more structured dataset, while the improvisation dataset is smaller and more freeform. The Nottingham dataset also contains significantly more repetition than the improvisation dataset. These differences make the variation in performance for the improvisation dataset much more difficult to see on a combined graph, and the graph also becomes more cluttered. Hence, separate figures are provided for each dataset for all evaluations, and for clarity at a glance, improvisation dataset figures are shown in teal whereas Nottingham dataset figures are shown in purple.

4.1.2 Pitch Prediction Accuracy

Firstly, individual component heuristic performance for pitch accuracy is evaluated for the improvisation dataset in Figure 4.1 and for the Nottingham dataset in Figure 4.2. Pitch accuracy is calculated by judging a prediction as correct if the predicted pitch is correct, regardless of the predicted time interval.

The key and modal and pitch interval heuristics unsurprisingly performed the best for pitch prediction, as they both focus on pitch. The repetition heuristic also performed well since it incorporates pitch information. As expected, the key and modal heuristic and the repetition heuristic performed significantly better on the Nottingham dataset, where repetition and consistent keys were much more common, than they did on the improvisation dataset. The pitch interval heuristic performed strongly on both datasets, as it is not dependent on clear structural features. For the tempo and swing, and the time multiple heuristics, the performance was similar to unguided prediction in the Nottingham dataset, and notably lower for the improvisation dataset. While it might be expected that these heuristics should perform very similarly to unguided prediction, given that they do not focus on pitch, a lowered prediction accuracy is not unexpected. This is because heuristic guidance which is not relevant to prediction accuracy only serves to dilute the relative probabilities of note options already provided by the MDRNN. While the MDRNNs most likely prediction could be the correct pitch, if a pitch agnostic heuristic is used, it will bias the search toward potential options with more likely time intervals,

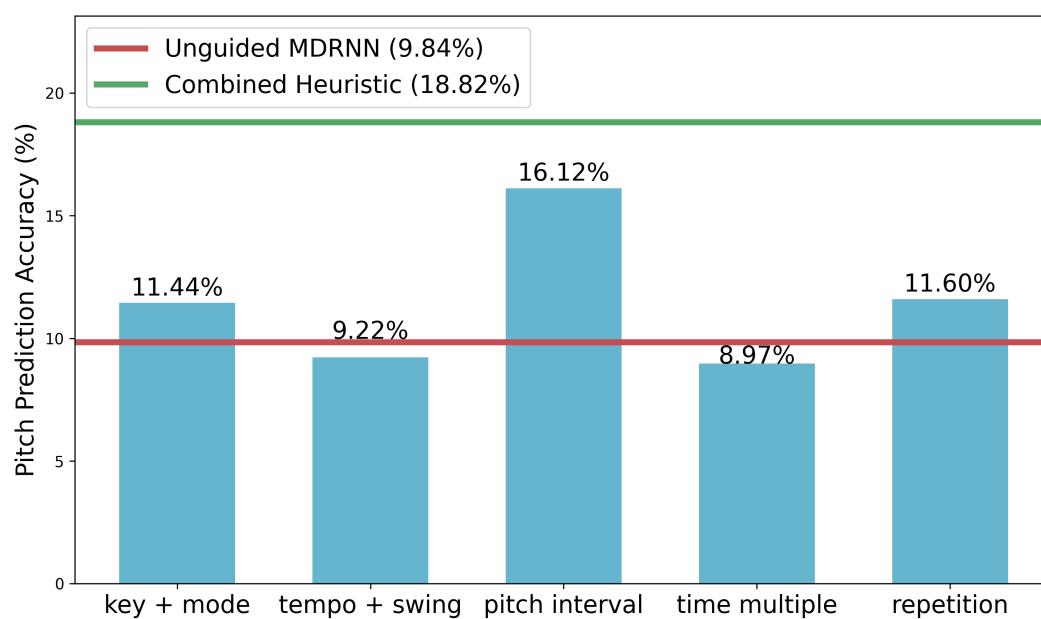


Figure 4.1: Pitch prediction accuracy of each component heuristic for the improvisation dataset.

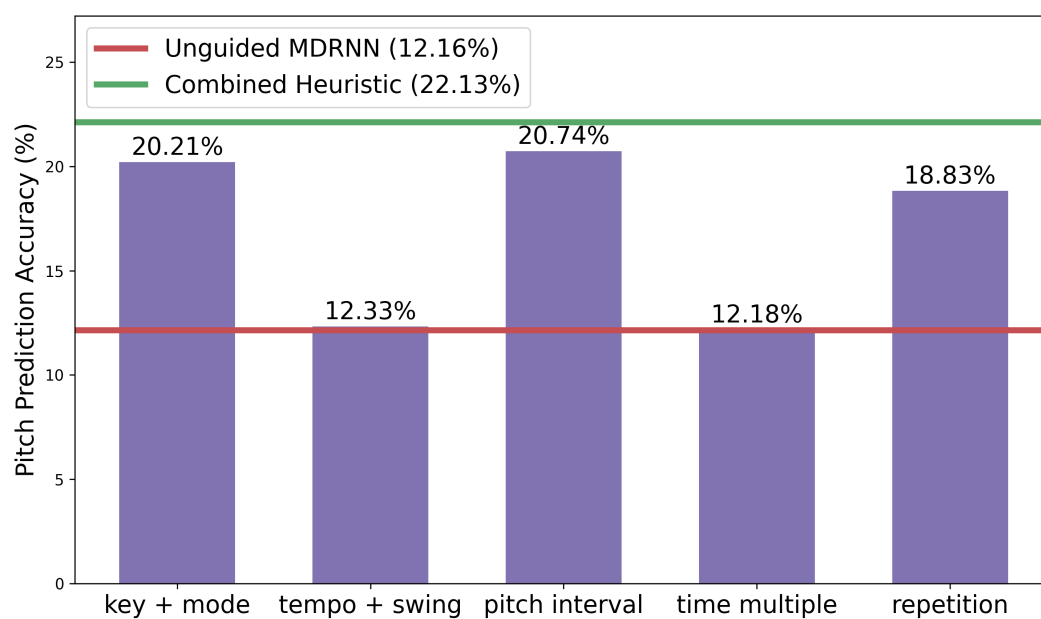


Figure 4.2: Pitch prediction accuracy of each component heuristic for the Nottingham dataset.

4 Evaluation

regardless of whether these options have significantly less likely pitch options. This is a key point to note, as it displays how heuristics must have a clear actively positive impact on prediction accuracy to provide a benefit to the system, as a neutral heuristic will have a negative impact by diluting the probability of the MDRNNs most likely options. Hence, the prediction accuracy not decreasing for the time interval focused heuristics in the Nottingham dataset is the more surprising result. This could be explained by stronger correlation of rhythm and pitch, more regular rhythmic patterns, or some other factor or combination of factors in the Nottingham dataset.

4.1.3 Duration Prediction Accuracy

Next, the duration prediction accuracy of each component heuristic is evaluated for the improvisation dataset in Figure 4.3 and for the Nottingham dataset in Figure 4.4. Duration accuracy is calculated by judging a prediction as correct if the predicted time interval is within 8% of the correct value, regardless of the predicted pitch.

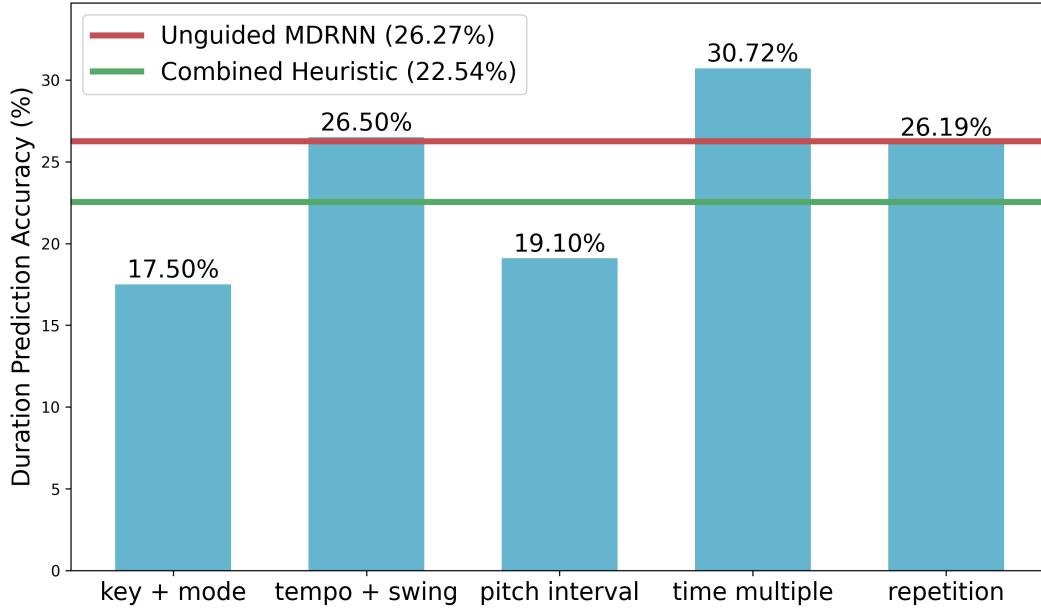


Figure 4.3: Duration prediction accuracy of each component heuristic for the improvisation dataset.

Once again, the relative performance of the heuristics is unsurprising. The tempo and swing heuristic and time multiple heuristic performed the best, as they are both focused on time intervals, with the time multiple heuristic performing well for both datasets, and the tempo and swing heuristic performing significantly better on the more structured Nottingham dataset, with barely any effect on the improvisation dataset where tempo was much less consistent. Interestingly, the negative effect of neutral heuristics was significantly more pronounced for the duration prediction accuracy than it was for

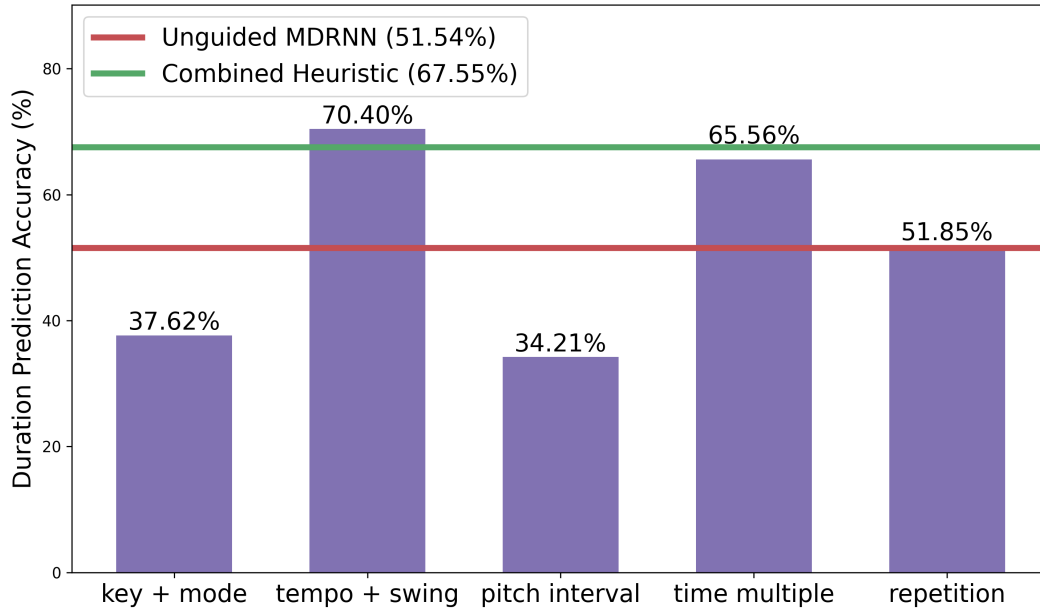


Figure 4.4: Duration prediction accuracy of each component heuristic for the Nottingham dataset.

the pitch prediction accuracy. This effect was so strong that it brought the repetition heuristic back to being level with the unguided MDRNN for both datasets, and for the improvisation dataset it led to the combined heuristic underperforming unguided MDRNN predictions. It is hard to know exactly why this is the case, but a potential explanation is that rhythm is more foundational to the structure of a melody than pitch is. This means that when predicting pitch, the underlying rhythmic structure still provides valuable context and a strong basis for the prediction, even if the heuristic is not directly relevant to pitch. However, when predicting duration, pitch information provides significantly less relevant context for the underlying rhythmic structure, and so pitch focused heuristics have a much more pure dilutive effect on duration prediction accuracy, whereas choices about rhythm are unlikely to dilute the most likely pitch options.

4.1.4 Total Prediction Accuracy

The final evaluation of the individual component heuristics is the total prediction accuracy, which is calculated by judging a prediction as correct if both the predicted pitch and time interval are correct. This is evaluated for the improvisation dataset in Figure 4.5 and for the Nottingham dataset in Figure 4.6.

The total prediction accuracy is the most relevant metric for evaluating the system, as it is the only metric which directly measures the system's ability to predict the next note. Ideally, all component heuristics should perform above unguided predictions for

4 Evaluation

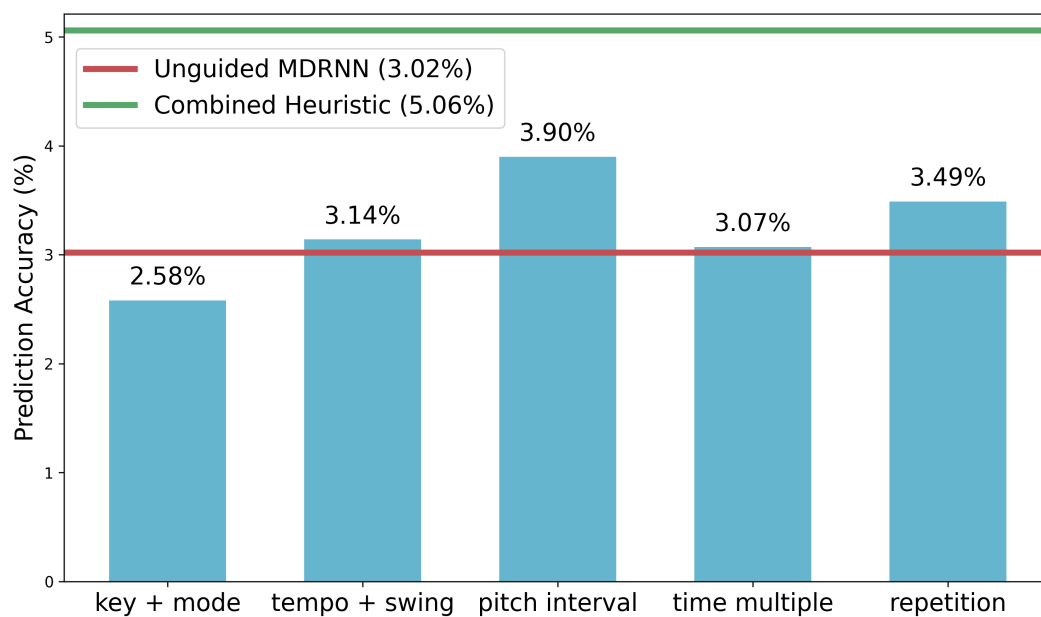


Figure 4.5: Total prediction accuracy of each component heuristic for the improvisation dataset.

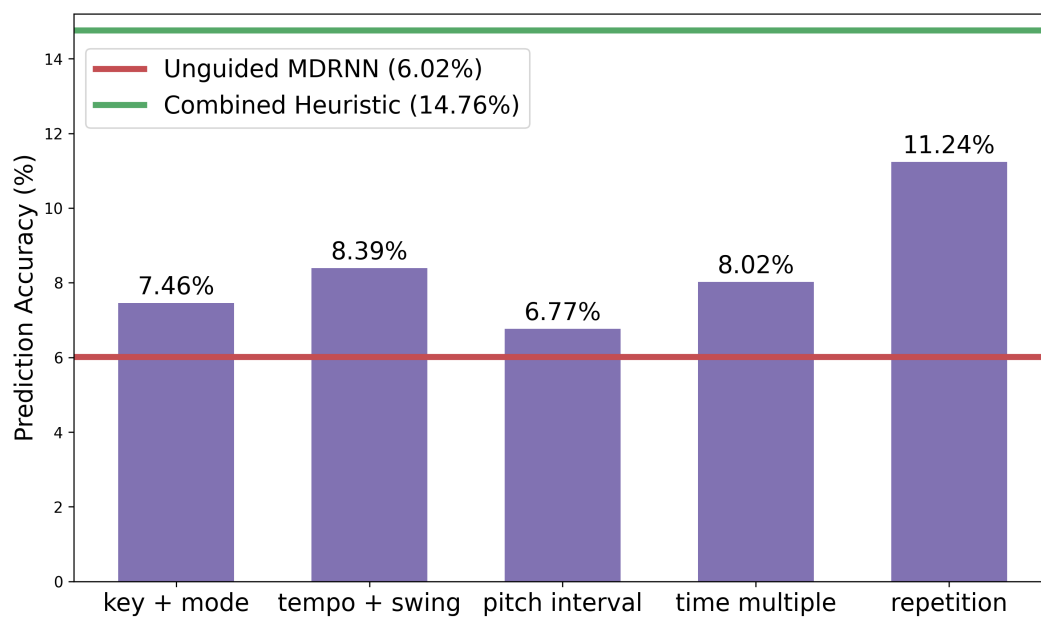


Figure 4.6: Total prediction accuracy of each component heuristic for the Nottingham dataset.

this metric. The results look, as expected, part way in-between their pitch and duration prediction results relative to the unguided MDRNN. The pitch interval Markov model heuristic had the strongest performance for the improvisation dataset, as its pitch prediction accuracy was by far the highest, and its negative effect on duration prediction accuracy was not enough to reduce its overall performance significantly. The repetition heuristic performed the best for the Nottingham dataset, due to its very strong pitch prediction accuracy, and neutral effect on duration prediction accuracy. Interestingly, the pitch interval Markov model heuristic had the weakest performance of any heuristic for the Nottingham dataset, despite being the strongest for the improvisation dataset. This displays how different datasets with different structures, styles, and sizes, can have a significant impact on which heuristics perform best. The only heuristic to underperform unguided predictions was the key and modal heuristic for the improvisation dataset, as it appears that its significant negative effect on duration prediction accuracy was not outweighed by its positive effect on pitch prediction accuracy. However, this does not mean that it is not a useful component of the combined heuristic, as it could be that the inclusion of other heuristics help to cancel out its negative effects while allowing its positive effects to remain.

4.1.5 Heuristic Component Contribution Assessment

This line of examination leads us to the next evaluation, which is the performance of the combined heuristic when each component heuristic is excluded. This is evaluated for the improvisation dataset in Figure 4.7 and for the Nottingham dataset in Figure 4.8. The combined heuristic is given its standard heuristic component weights, but with the excluded heuristic given a weight of 0. This shows how much the overall performance of the combined heuristic is affected by each component heuristic, and whether the combination of heuristics makes some component heuristics redundant, or whether they help to balance each others weaknesses.

It is critical that the exclusion of any given component heuristic leads to a notable drop in performance for at least one of the datasets, and at a minimum doesn't increase performance for the other dataset. Fortunately, for both datasets the exclusion of any component heuristic results in a drop in performance. And the exclusion of each component heuristic leads to a significant drop in performance for at least one of the datasets. This shows that none of the component heuristics are redundant, and that they all provide a unique contribution to the overall performance of the system. It is also important to note that the key and modal heuristic, which had a negative effect of prediction accuracy on the improvisation dataset when used in isolation, caused a notable drop in performance when removed from the combined heuristic. This shows that the inclusion of the other heuristic components helped to cancel out its negative effects on duration prediction accuracy, and allowed it to have an impact mostly when it was relevant for pitch prediction. Another notable result is the two heuristic components which had relatively neutral effects on prediction accuracy when removed from the combined heuristic. This was the repetition heuristic for the improvisation dataset, likely because there is

4 Evaluation

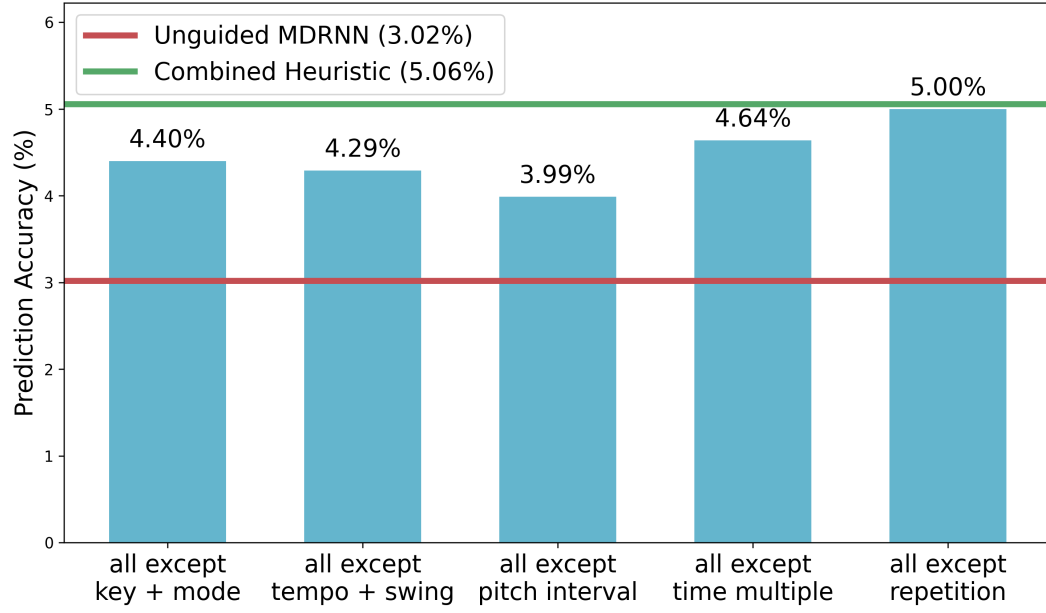


Figure 4.7: Total prediction accuracy of the combined heuristic when excluding each component heuristic for the improvisation dataset.

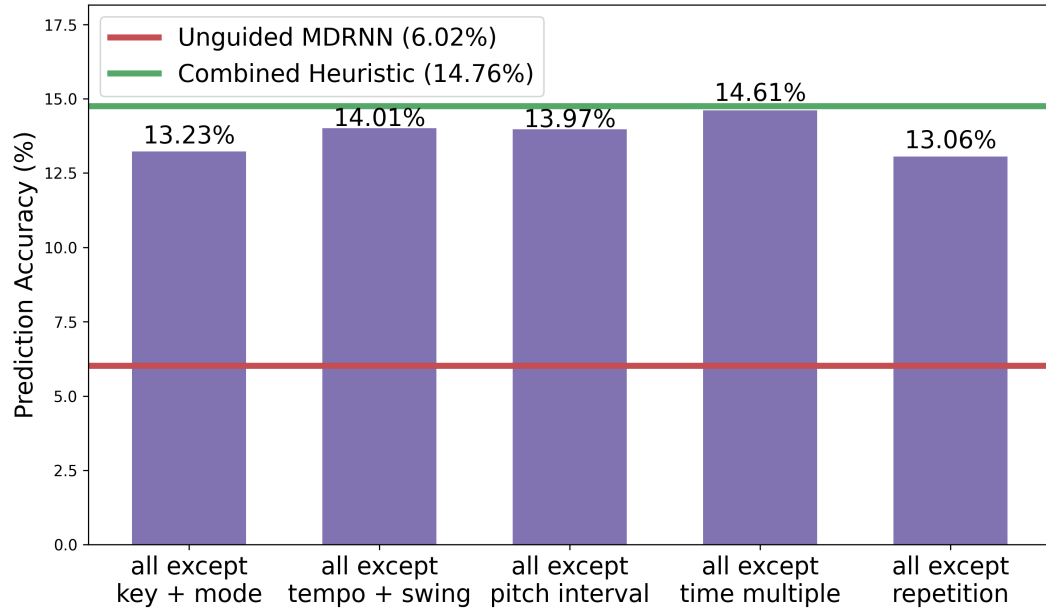


Figure 4.8: Total prediction accuracy of the combined heuristic when excluding each component heuristic for the Nottingham dataset.

very little repetition in the improvisation dataset, and so it is made redundant by the pitch interval and time multiple heuristics, which it likely partly replaced the function of when used individually to generate its small positive effect on prediction accuracy. In contrast, for the Nottingham dataset the loss of the repetition heuristic caused the largest drop in performance, whereas the loss of the time multiple heuristic caused very little. In this dataset repetition is much more common, and so the repetition heuristic is able to provide a much larger positive effect on prediction accuracy, whereas note duration is much more consistent and predictable, and so the MDRNN is likely able to predict it quite well. For individual performance, the time multiple heuristic likely gained most of its performance through partially replacing the function of the repetition heuristic, and so in the combined heuristic it is made relatively redundant.

4.1.6 Performance Scaling of the Combined Heuristic

Evaluation of the combined heuristic has so far been limited to lines of each previous figure, providing a point of comparison for judging the performance of each component heuristic. However, it is also important to evaluate and display the performance of the combined heuristic in more detail. This performance is measured relative to the important metrics of number of search iterations and search time. The number of search iterations is the number of simulations conducted in the MCTS search, and given that more simulations leads to a larger search tree and more rigorous search, it is the main parameter impacting the performance of the system. However, increasing the number of search iterations also increases the search time, and so it is important to evaluate the trade-off between these two parameters.

Firstly, the total prediction accuracy of the combined heuristic is evaluated for the improvisation dataset in Figure 4.9 and for the Nottingham dataset in Figure 4.10 relative to the number of MCTS iterations on a log scale. Data for 10^n iterations with n increasing by increments of 0.25 from 0 to 4 are shown, with an asymptotic fit calculated by non-linear regression.

The first important point to note is that significant performance improvements are seen for both datasets with even a very small number of MCTS iterations. This is a very positive result, and demonstrates the effectiveness of MCTS-IMPSY even on slow hardware and with short search times. Given that one of the main limitations of MCTS-IMPSY is the significant increase to prediction time when using a form of MCTS search which requires many MDRNN outputs for a single prediction, as opposed to one output for the previous method, it is promising to see the immediate benefit of small numbers of MCTS iterations. However, the limitations of this slow form of MCTS search are still clear. For MCTS search where the action space can be generated without a computationally costly MDRNN prediction, MCTS search with 10000 iterations is seen as a low, as the benefits from iterations above this amount are still often very noticeable. However, for MCTS-IMPSY, even reaching 1000 iterations is impractical for real-time performance. The continuing improvements in performance with more iterations are clearly shown by the asymptotic fits, which show that at 1000 iterations the system still has a significant

4 Evaluation

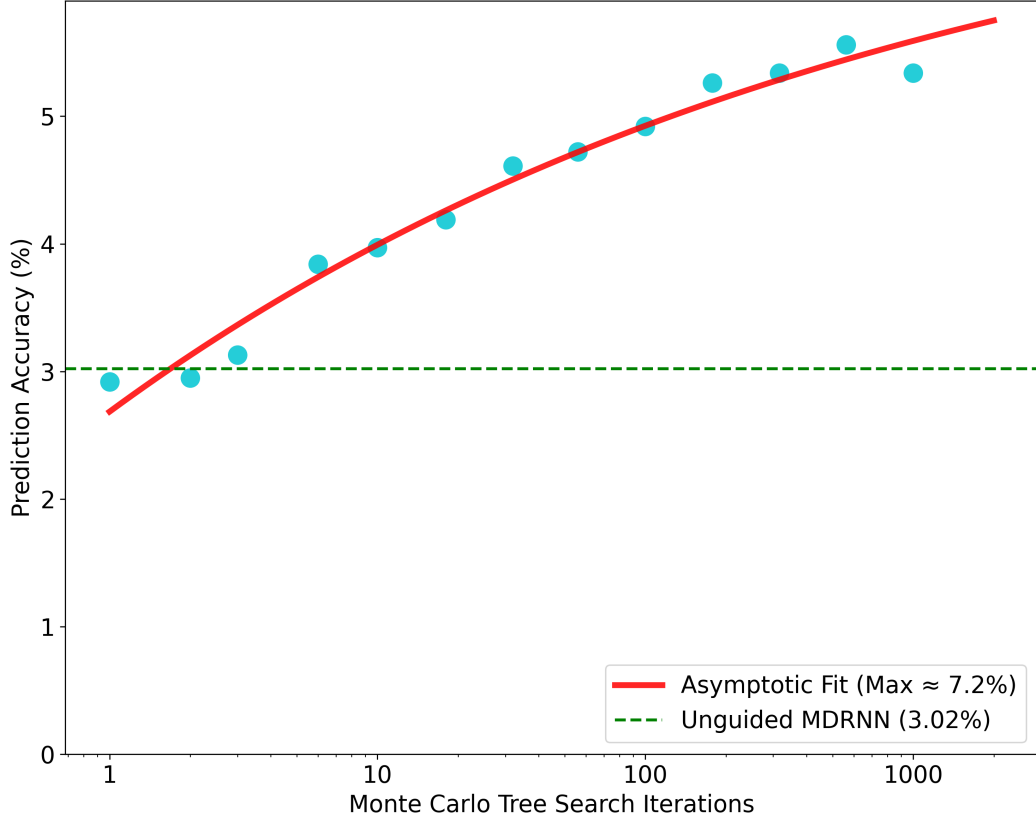


Figure 4.9: Total prediction accuracy of the combined heuristic for the improvisation dataset by number of MCTS iterations (log scale).

amount of estimated room for improvement. It is important to note that the asymptotic maximums estimated by the fits are very loose estimations and cannot be relied on to be accurate without much more data at higher amounts of iterations, but they do provide a useful educated guess at the maximum potential performance of the system. Another useful point to note is that the asymptotic fits for the Nottingham dataset begin to level out at a lower number of iterations than for the improvisation dataset. This is likely for the same reasons that the Nottingham dataset sees larger prediction accuracy improvements in general, being that because it is a dataset with more structured and consistent pieces, the structurally focused heuristics in MCTS-IMPSY are able to provide a larger benefit, and can establish this benefit with a smaller search tree. For the improvisation dataset, the branching factor of the search tree is larger due to the more freeform nature of the pieces, and so it requires more iterations to effectively explore.

While the number of MCTS iterations is the more relevant metric for evaluating the performance of the system from a designers perspective, from a performers perspective the search time is the more relevant metric. Due to the search time of MCTS iterations

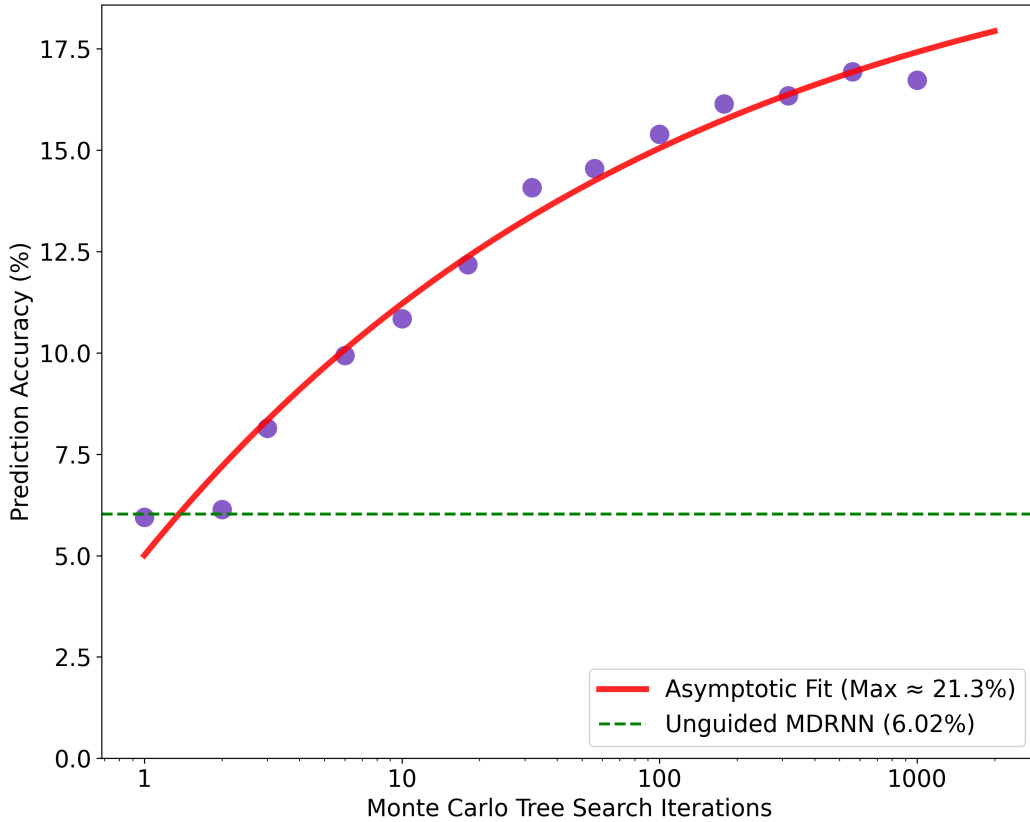


Figure 4.10: Total prediction accuracy of the combined heuristic for the Nottingham dataset by number of MCTS iterations (log scale).

being concentrated in generating MDRNN predictions, search time grows very close to linearly with the number of MCTS iterations. This is shown in Figure 4.11, which provides a linear line of best fit to show the estimated gradient of the search time with respect to the number of MCTS iterations. This allows for conversion of accuracy calculated from the previous figures into search time based metrics.

4.2 Qualitative Performance Experience

While prediction accuracy is a very important metric, and one which is much easier to rigorously evaluate than others due to being objective and quantifiable, it is not the only metric which is important for evaluating the system. Wagstaff (2012) notes that accuracy only measures a systems ability to recreate data in the style of the dataset it was trained on, but this may not always be the desired result. If a system produces an output which is not the correct prediction, but through increased creativity is a prediction which a performer views more highly than the ‘correct’ prediction, then this

4 Evaluation

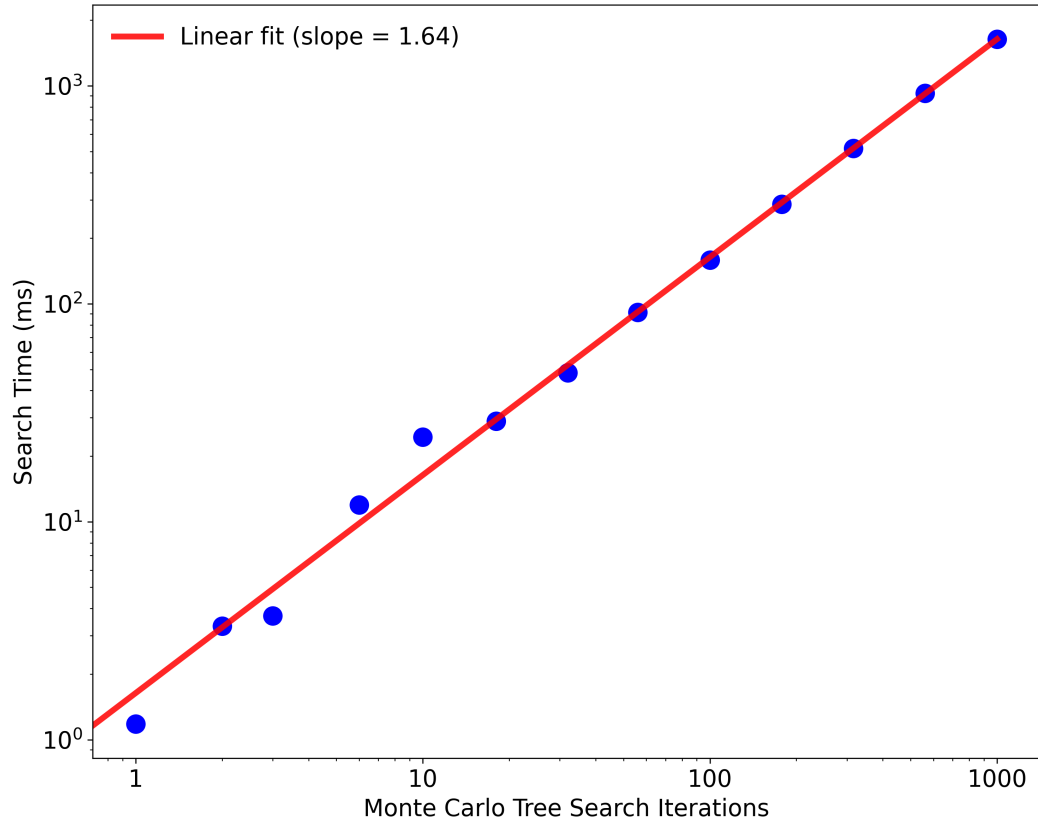


Figure 4.11: Search time by MCTS iterations.

should be considered a success. During its practical use, MCTS-IMPSY is being viewed from the perspective of the performer, not the designer, and so evaluating it from this perspective is important. Many of the modifications to IMPSY were made to help it conform to important structural musical parameters of a piece, and this could potentially lead to more rigid and less interesting improvisations despite the increase in prediction accuracy, a potential limitation which requires testing. The system is also designed to be interactive, acting in a call and response manner with the performer, and this aspect of the system has not yet been evaluated. This type of evaluation also allows for qualitative perspectives on the outputs to be shared, which, given the level of subjectivity and nuance in music, provide important commentary that quantitative methods cannot provide. However, this type of evaluation essentially served as a check and balance that improved prediction accuracy did not come at the cost of expressiveness and creativity, and so the extent and level of detail of analysis is not as rigorous.

To evaluate the system from the perspective of a performer, a series of improvisations has been conducted with MCTS-IMPSY, and will be subjectively and qualitatively analysed on their musical qualities. For this section of the evaluation, I will act as the performer,

and only the model trained on my own improvisations will be used, imitating a practical use of the system. I have performed and recorded 4 call and response improvisations with the system, each lasting slightly over 5 minutes. The first pair of improvisations tested a structured performance, with clear key signatures and tempos that were adhered to in specific sections. The first of these was a [structured improvisation with MCTS-IMPSY](#) and the second was a [structured improvisation using MDRNN-IMPSY](#). The goal of these structured improvisations was to test the musical coherence and playability of the system, to see how well it was able to remain within the structure of the piece, and respond to the performers input in an appropriate manner. The second pair of improvisations tested a more freeform performance, with drifting time signatures and tempos. The first of these was a [freeform improvisation with MCTS-IMPSY](#) and the second was a [freeform improvisation using MDRNN-IMPSY](#). The goal of these freeform improvisations was to test the expressiveness and creativity of the system, to see how well it was able to generate interesting and unique musical ideas, and whether it could still maintain coherent call and response interactions with the performer when the performer was not adhering to a strict structure. A summary of these improvisations and their results is shown in table 4.1. A more detailed analysis of each improvisation pair, including examples with timestamps and sheet music transcriptions, is provided in the following subsections.

Table 4.1: Summary of conducted improvisations.

Improvisation	Link	Results
Structured MDRNN-IMPSY	Video	Used as a control.
Structured MCTS-IMPSY	Video	Improved coherence and playability.
Freeform MDRNN-IMPSY	Video	Used as a control.
Freeform MCTS-IMPSY	Video	Slightly improved expressiveness.

4.2.1 Structured Improvisations

MCTS-IMPSY was able to maintain a coherent call-and-response improvisation throughout the structured improvisation, adhering very well to the key signature and pitch based patterns, and passably well the tempo and rhythmic patterns. The system would continue human melodies, and also if left alone transition into new creative and convincing melodies of its own creation. Comparatively, MDRNN-IMPSY struggled to generate convincing melodies, and would often generate outputs which were not musically sensible continuations of the human input. The overall coherence, playability, expressiveness, and creativity of the MCTS-IMPSY improvisation session was significantly higher than for the MDRNN-IMPSY improvisation session. A review of key moments in the structured MDRNN-IMPSY performance will now be conducted, including timestamps for the videos, and sheet music where helpful.

The structured improvisation begins in the key of D Major with a tempo of 120 BPM. When the system takes over at [0:22](#), the key, mode, and tempo are all corrected identi-

4 Evaluation

fied. The pitch interval Markov model also notes the interval patterns of the descending melodic line and alternating neighbouring semitones at the end of the phrase, and immediately continues this pattern to ensure a melodically graceful transition, before beginning its own melodic ideas starting at 0:26. Figure 4.12 shows this transition in a musical score.

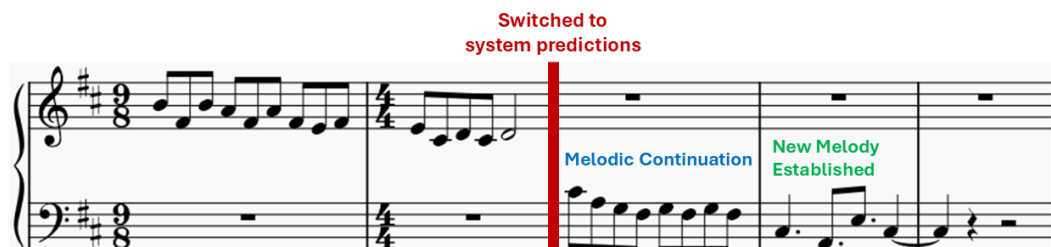


Figure 4.12: Score of the transition from human to system predictions at 0:19 in the Structured MCTS-MDRNN performance.

In comparison, the first switch to system prediction in the structured MDRNN-IMPSY performance at 0:22 is much less coherent. The key of D Major with a tempo of 120 BPM is set up similarly during initial human input, but the MDRNN predictions fail to maintain this key and play a very out of place descending semitone line from D4 to B3, including the clearly out of key C4 note. This descending semitone line is also not a continuation of very structured descending line from the human input leading into this transition, and so the switch feels quite jarring. Out of key notes continue to be played, and the MDRNN doesn't establish a clear melodic line of its own. The MDRNN feels consistently like it is beginning to form a musical phrase, but never completes it, whereas the MCTS-IMPSY system is able to form fully formed musical phrases.

The remainder of the performance continues along similar trends. Another notable moment is MCTS-IMPSY creating a coherent key change to a neighbouring key of A Major, which has one extra sharp, at 0:32. This moment sounds completely natural, and allows for me to take over again and continue in the key of G Major gracefully. The next switch to the system in the MCTS-IMPSY performance occurs at 1:36. The system correctly identifies the human input has ended at the conclusion of a melodic idea, and so it begins its own. While the rhythm of this melody is slightly stunted and off, it is close to reasonable, and the pitch of the melody is very convincing.

This strong structural and melodic coherence is maintained throughout the rest of the performance, and made the playability of improvisation with MCTS-IMPSY quite strong. MDRNN-IMPSY continues to provide decent predictions on a short time scale, but struggles to fit the structure of the piece, and adds many off key notes and random jumps which break the flow of the performance. Overall, for the structured improvisations, the MCTS-IMPSY system significantly outperformed the MDRNN-IMPSY system on all relevant metrics. Not only were MCTS-IMPSY predictions coherent, fitting the parameters of the performance, but the system also produced more expressive and cre-

ative melodies. It seems that the Markov model heuristic components influence melody creation in a positive way, on top of leading to more natural transitions between human and system input.

4.2.2 Freeform Improvisations

The performance results during a more freeform improvisation were less impressive for MCTS-IMPSY. While MDRNN-IMPSY performed at a similar level for both the structured and freeform improvisations, MCTS-IMPSY clearly performed better with more structure. However, MCTS-IMPSY still performed better than MDRNN-IMPSY for freeform improvisations, and maintained a similar level of creativity. This shows that the modified prediction algorithm does not detract from the creativity of the system, and that it is still able to generate interesting and unique musical ideas, and to a higher standard due to the increased musical coherence.

An example which provides a good illustration of the general performance of MCTS-IMPSY during the freeform improvisation is at 1:23. The continuation starts off strong, but begins to wonder into odd musical directions beginning at 1:31. At this point the system has picked up on a key and tempo used in the recent human input, and continues with these same parameters, but the time interval and repetition Markov models have little useful information, leading to the odd and stunted rhythmic structure at this point in the performance.

The MDRNN-IMPSY performance is even less coherent than this, and with no more creativity or expressiveness. Transitions such as at 2:58 are jarring, no structural elements are maintained for any significant amount of time even in portions of the performance where they are present, and the system fails to develop melodic ideas into melodic phrases. MDRNN-IMPSY performs very similarly for this improvisation as it did for the structured improvisation, although its weaknesses are slightly less obvious in this performance due to less built up expectation of particular structural parameters.

The freeform improvisations provide less clear examples than the structured improvisations, but at least show that the expressiveness and creativity of MCTS-IMPSY in such a performance is not diminished by the use of MCTS search. The benefits of the MCTS system are much less obvious for this improvisation, as would be expected, as during many points in the performance the key and modal and tempo and swing heuristics are disabled due to not being able to establish clear and static values during human performance. However, the Markov model heuristic components still seem to provide a notable benefit.

4.2.3 General Observations

From qualitative analysis of the system from a performers perspective, it appears that the prediction accuracy improvements MCTS-IMPSY provides are not at the cost of playability or expressiveness. During more structured improvisations, the use of MCTS

4 Evaluation

significantly improved musical coherence, playability, and even expressiveness when compared to the previous MDRNN only version of IMPSY. The system was able to maintain a coherent call and response interaction with the performer, and was able to generate interesting and unique musical ideas. During more freeform improvisations, the difference in performance was less clear, and the MCTS system provided weaker outputs than during structured improvisations. This is unsurprising, as multiple of the component heuristics in MCTS-IMPSY are defined with specific structures like tempo and key signature in mind, and so when these structures are not present these components deactivate and cannot provide any benefit. However, even in these freeform improvisations, MCTS-IMPSY generated more coherent outputs than the pure MDRNN method, and this made it more easy to continue on from in call and response. This did not come at the expense of expressiveness and creativity either, as improvements in prediction accuracy seem from qualitative analysis to be mostly based on improving musical coherence of otherwise incoherent musical phrases that MDRNN-IMPSY often generates, which allows for more interesting musical ideas which don't immediately dissolve into incoherence. Overall, the system appears to be a significant improvement over the previous MDRNN-IMPSY system, and does not have any obvious qualitative weaknesses in comparison.

Discussion

This thesis introduces a novel prediction system for real-time interactive musical performance, building upon the IMPSY system. The core innovation lies in integrating a Monte Carlo Tree Search (MCTS) algorithm with a Mixture Density Recurrent Neural Network (MDRNN) to guide musical predictions. Unlike previous approaches that rely on random sampling from a Gaussian Mixture Model (GMM), MCTS-IMPSY employs a heuristic-based search to ensure musical coherence and expressiveness. The research specifically focuses on the call-and-response mode of IMPSY, using a monophonic 2D keyboard as a proof of concept. The evaluation demonstrates that the MCTS-enhanced system significantly improves prediction accuracy and qualitative aspects like playability and expressiveness, especially in structured improvisations, by generating more musically coherent outputs.

5.1 Impact

The integration of MCTS and heuristic-based guidance into the IMPSY system offers significant advancements for New Interfaces for Musical Expression (NIMEs) and human-computer interaction as a whole. This thesis serves as a proof of concept for the potential that search integration with current machine learning prediction and interaction systems can have.

The most obvious area of impact for this research is on musical prediction systems. By leveraging MCTS to guide machine learning based predictions, prediction accuracy improved significantly. Of course, this research has had a direct impact on the performance of the IMPSY system ([Martin and Tørresen, 2019](#)), but it has implications for the field as a whole. Prediction systems which use rule-based approaches or Markov models for prediction, such as the Continuator by [Pachet \(2002\)](#) or the CONCERT system by [Mozer \(1994\)](#), can have their prediction ability converted into a component heuristic, and then

integrated into a neural network based system such as the set of interactive systems developed by [Collins \(2007\)](#). This can of course be extended to any neural network based system being combined with any number of rule or Markov based systems, each of which can be used to create a heuristic component for the MCTS search. By combining systems with different strengths, the weaknesses of each system can be mitigated, and the strengths of each system could be combined to create a more powerful system. This doesn't require combined pre-existing systems however, as MCTS-IMPYSY acts as a broad proof of concept for the idea of combining search with any machine learning based system for music generation, and this has a large impact on future research methodology and direction in these fields, allowing developments in each to more directly impact each other.

This thesis also has significant implications for research into any interactive artistic system which functions with call-and-response generation. Each component heuristic is fine tuneable and editable, and so by creating a user interface for the heuristic components of the system, the user can have significant control over the system's output. This allows for previously entirely generative composition systems to be modified to allow for a human to influence the system's output over time, and for interactive systems to allow for a human to influence the system's output over time, shifting compositions in a direction shaped by the human user, even when that user is not actively performing. This has significant implications for the future of interactive systems as a whole. For example, a non-interactive music generation approach, such as DeepBach ([Hadjeres et al., 2017](#)) which attempted to generate convincing chorale harmonizations in the style of Bach, could by shifting to an MCTS prediction process add heuristic components such as tempo, key signature, and other musical parameters. Instead of using these to improve prediction accuracy, they could by default be disabled, meaning that DeepBach would function as normal, but could be enabled and modified by a user as DeepBach was composing, allowing a human user to guide the composition process toward particular stylistic choices. Applying this concept generally, any generative music system could be modified into an interactive system using this approach, which has a large impact on the field, as typically these two types of systems have been researched separately.

Interactive systems are also impacted by the generalisability and customisability of the system. The ability to create and modify heuristics provides a flexible framework which can be adapted to many fields. This include non-musical fields, such as generative visual art, text, and video, where the same concept of using heuristics to guide generative processes could be applied. Within the musical setting, gestural performance systems could train machine learning models on individual and hard to formally define control parameters such as mood, and then link them to a body part or action, so that a system could be created which creates generative music, but with individual control parameters trained on human input being controlled by human gestures. Another subfield where this concept could have significant impact is the hyperinstruments project by [Machover and Chung \(1989\)](#), which has influenced the creation of a multitude of instruments over recent decades which include enhanced control capabilities for human performers through

the development of complex mapping systems. Integrating the research from this thesis, hyperinstruments could implement partially generative components to instruments, where certain aspects are controlled by the human performer. For example, hyperinstruments could be created where the controls of a human performer influence heuristic components which provide strong guidance for a prediction system which outputs each note.

This research also bridges the gap between musical research and research into other field where combined MCTS and machine learning systems are used. This includes game playing, natural language processing, and robot motion planning. The use of similar techniques to these fields provides a framework for potential crossover between these fields. For example, a modified version of MCTS-IMPSY could be used within the game AI field to create musical games where the search tree is generated using a similar approach. Crossovers with robot motion planning could also be made, for example robot instrument playing. These crossover applications do not have to be a mix of the two fields however, as there is likely even more impact provided by the new link in system architecture between these fields. For example, robot motion planning research using C-MCTS search has assisted in the development of many new advanced search techniques, such as the KR-AUCB C-MCTS selection strategy (Lei et al., 2021), and now whenever these new developments in the robot motion planning field are made, they can be applied to the musical domain as well. This is a significant impact of this research, as it allows for the two fields to share and develop new techniques together, rather than having to develop them separately.

Impact can also be found in human performance parameter detection. MCTS-IMPSY is able to detect and track human performance parameters, and then use these to inform the MCTS search. Hence, this links MCTS-IMPSY to all other research where musical parameters such as genre, mood, etc are tracked. There is now a way to link these detection based papers into interactive systems, by utilising their parameter detection systems to generate heuristics which can inform interactivity and prediction. For example research into mood detection in music, such as by Lu et al. (2006) could be directly applied to generative musical processes aiming to be able to control the mood of generated musical output by integrating its findings into a heuristic component in an MCTS based prediction system. The method of using performance parameter detection to inform heuristics which guide musical output could also be very using for collaborative networked performance. Weinberg et al. (2009) developed a system where AI and human performers must interact and improvise in real-time, and if these AI were continuously tracking musical parameters in the human performance, they could use a combined search and neural network generation process to generate musical outputs which more coherently match with the human improvisation.

5.2 Limitations

While the results of this thesis are strong and offer promising potential, there are several limitations to the current system and how it was evaluated. These limitations are important to note, as they provide context for the results, balance the potential impact and scope of this research with what has actually been achieved, and provide a basis for future work.

Firstly, human performance parameters, such as key signature and tempo, were assessed and used statically in the system. For example, the improvisational dataset contained many sections where tempo would slowly increase or decrease, and generally flow as the performance progressed. However, tempo calculations were made on the assumption that if there was a clear tempo, it would be static for the entire duration of the output memory, and this also applies to parameters calculated for other component heuristics, such as key and mode. Ideally, detection of parameters should be done constantly, tracking changes over time, and hence gaining a more accurate understanding of the current state of the performance. This could be done by both recording parameters at each time step, and by modifying heuristic components that rely on static parameters. For example, while the work of [Laroche \(2001\)](#) which relies on static tempo was used as the basis for the tempo parameter, other methods, such as the one by [Scheirer \(1998\)](#), are designed to be able to track changing tempos, and this research could be used as the basis for the heuristic component instead. Another limitation is that detected performance parameters are left constant throughout the duration of the systems output, and are only modified once the human performer takes over again and changes them. Given that in human performances these parameters are often constantly shifting, ideally this would be recognised and accounted for by the system. This limitation could be mitigated by training a second LSTM-RNN to take the change in parameters over time during human performance, and use this to predict the next state of the parameters at each time step during the system’s output.

The computational cost of using MCTS with as MDRNN prediction for the generation of each nodes action space is very high. The evaluation showed that with a 12th Gen Intel Core i7-12700H processor, each MCTS iteration would take 1.6ms on average, leading to practical use cases often being able to utilise less than 100 iterations for each prediction. This is also with a simple 2D instrument, and for another instrument where more dimensions are tracked and involved in the heuristic, this could slow down performance even more. Fortunately, figures [4.9](#) and [4.10](#) displayed how even with tiny numbers of iterations, the system was able to achieve a significant improvement in prediction accuracy. But, the current number of iterations were not close to the estimated maximum potential performance of the system estimated by these figures, and so optimisations would be helpful. One of the main limitations in the system design causing potentially unnecessary computational cost is the use of the full MDRNN to generate simulations during each iteration. MCTS was designed to use random selection during the simulation phase [Coulom \(2006\)](#), and so computationally cheap simulations are the norm. How-

ever, because there is no known action space for musical continuations without the use of a neural network, cheap simulations are more complicated to devise in this context. Possible approaches include using a much more simple model for simulation predictions, or even switching to a Markov model or rules based approach for the simulation phase.

Another notable limitation of this system is its scope. Since the type of method used in this thesis is entirely novel for an interactive musical system, this thesis was always intended to serve only as a proof of concept for the type of prediction system developed in this thesis. While the strong results of this approach are enough to warrant further research, there is no firm proof that the system is generalisable to other instruments, genres, or even improvisational contexts, despite this generalisation potential being one of the key motivations for this research. Unfortunately, the scope of a research project evaluating whether this approach is truly generalisable is outside of what was possible in this thesis, and so will have to be left for future work.

While the customisability and generalisability of the heuristics used in this thesis is a strength, it is also a limitation. The system is only customisable by editing its code, and so for a user who does not know how to do this it is not a helpful advantage. Practical implementations of the system for the use of performers without these skills would require only a set of general heuristics, which limits the potential of the system. While this is not technically a limitation, it is a caveat to one of the key strengths of the system. However, creating a user friendly interface for the system, and allowing the self training of heuristics, would allow the customisability of the system to be accessible to a wider audience, and multiple avenues for future work which mitigate this limitation are discussed in the next section.

While results were very strong, it is also important to note that they were considerably stronger for more structured datasets and performances than they were for freeform ones. Prediction accuracy was much higher for the more structured Nottingham dataset than it was for the improvisational dataset, and the qualitative evaluation of the improvisational dataset showed that the while MCTS-IMPSY was better to perform with for both structured and freeform improvisation, the difference was much more pronounced for structured improvisations. This could be because specific heuristic components such as the key signature, tempo, and repetition heuristics are much more relevant in structured improvisations, but it also could be that even despite this the system is not as good at freeform improvisation. This is relatively unsurprising given that rules-based an heuristic based systems generally follow this trend, and this was the component that was added to IMPSY, but it is worth noting regardless.

There were also limitations with the methodology used to evaluate the system. The quantitative evaluation was conducted using two datasets, one of which was a structured dataset and one of which was an improvisational dataset. While this is a good start, it would be beneficial to conduct further evaluations on a wider range of datasets, including more structured and freeform improvisations, as well as different instruments and genres. This would provide a more comprehensive understanding of the system's performance

and its generalisability across different musical contexts. The quantitative evaluation was also conducted on note by note prediction accuracy, but did not test prediction ability over longer phrases, or account for how poor incorrect predictions were. The qualitative evaluation was also limited in scope, as I was the only participant, and an evaluation from an audience perspective was not conducted. These types of qualitative evaluations were beyond the scope of this thesis, but would be beneficial for future work. MDRNN-IMPSY was also used as the control for the entire evaluation, however, it could have been beneficial to add a state-of-the-art rule or heuristic based prediction system as a second control, to verify that the combined approach improves upon both approaches individually.

5.3 Future Work

Much of the future work influencing other fields and NIMEs has already been discussed in the impact section, and so this section will focus the possibilities for future expansion of the MCTS-IMPSY system developed in this thesis specifically. Future work which primarily addresses the limitations of the current system has also already been discussed in the limitations section, and so will be excluded here as well. After excluding these two categories of future work, there are broadly four categories of future work which could be pursued. These are improving the MCTS search algorithm, improving and expanding upon the heuristics used to guide that search algorithm, improving user interactivity and control options, and integrating further structural components into the system. Each of these categories will be discussed in turn.

5.3.1 Improving the MCTS Search

One idea for improving the search algorithm could be to not limit action discovery to the MDRNN output. The current system only uses the MDRNN output as the action space for the MCTS search, but this could be expanded to include other sources of information. For example, there were cases during testing where the system chose an odd note, and when it was manually checked, the heuristic had labelled the note as a very strong choice, but the MDRNN hadn't found it, and hence there was no way for it to be included in the search space. Often these notes had each individual dimensional within the action space, for example a note duration rated highly by time focused heuristic components, and in a separate child note a pitch rated highly by pitch focused heuristics, but no child which combined both. Some form of alternative action injection based on heuristic evaluation could be an interesting line of future research. This type of action injection could also be a cheaper way to generate simulations. A risk with this idea is that it could remove some of the benefits of using a neural network in the first place by occasionally ignoring its probabilities. It could be that simply by improving the training of the MDRNN, this issue could be mitigated without any additional complexity. However, this would need to be tested, and hence this is an interesting line of future research.

Another avenue for future work is to implement advanced C-MCTS selection and expan-

sion strategies developed by other fields, such as KR-AUCB selection (Lei et al., 2021) and Bayesian optimisation expansion (Mern et al., 2021). These strategies have been shown to improve the performance of C-MCTS in other fields where multiple continuous dimensions are present. The limited scope of this thesis, with a 2D model where only one dimension was entirely continuous, meant that these advanced strategies were not necessary, and their minimal benefit would be outweighed by the added computational cost. However, in the future, if the system is expanded to include more dimensions, or if the computational cost of the current system is reduced, these advanced strategies could be beneficial.

Another area that could be explored is efficiencies for branch prediction based heuristics. Unlike the majority of MCTS implementations, where a heuristic evaluates the state at the end of the simulation, with its parents not being relevant to the evaluation, MCTS-IMPSY uses a heuristic which evaluates the branch as a whole which reaches the final simulated node. This could lead to some differences in evaluation method being able to optimise for this type of heuristic. For example, heuristic scores could be biased toward the earlier nodes in the branch, as these are more immediately relevant to the final node choice. It would be worthwhile exploring other research on using analysis of sequences in a heuristic, and see if there are any known improvements which could be applied to this research.

5.3.2 Improving and Expanding upon Heuristics

The heuristics used in this thesis were designed to be generalisable and customisable, and a small set of component heuristics were created and used for the final system used in the evaluation. This leaves the potential for many avenues of future work. Other than the obvious avenues of creating more heuristics in similar styles to the ones in this thesis, but customised for different instruments, genres, or performance goals, there are also many other avenues of future work which could be pursued.

Firstly, heuristics which are less structured could be explored. The current system doesn't have to include hard-coded heuristics, and could instead use multiple individual neural networks for separate heuristics. This could allow for heuristics to more easily cover more abstract concepts, such as mood or texture. Heuristics such as these could have the potential to improve the system's performance in freeform improvisations, where the current heuristics are less effective. These could be trained by manual human annotation of parameters throughout a testing set, which could then be used to train a neural network to predict the values of these parameters. It is important to note that this would also be a user trained heuristic, which would be an important future work contribution as well. By allowing a user to for example listen back to the data used to train a model with a slider for a certain parameter they want to train, which they move according to their perception of the value of that parameter as they listen, they could train their own heuristic component on whatever abstract concept they wanted to. It would be interesting to establish whether future work in this area could produce promising results, as one of the important motivating drivers for this research was to unite

rules based and machine learning based approaches to music generation, and this would essentially replace those added rule based components with machine learning again. If future work in this area is successful, it would show that the strength of this approach comes from the search process significantly more than from the integration of rule-based parameters.

Another idea is the integration of parameters specifically relating to long term structure of a piece. For example, MCTS-IMPSY does not currently have any ability to repeat choruses, understand song forms, or recognise when a bridge is being played. For improvisation, this is not particularly important, but if IMPSY was being used for composition generation it would be very important. Methods for including musical form in generative composition have been explored ([Collins, 2009](#)), and integrating these methods as parameters which track the likely boundaries between sections and how this should influence search would be a useful avenue of future work. This would likely need to be combined with the action injection idea discussed earlier, or some alternative to it, as even the repetition heuristic in MCTS-IMPSY cannot repeat long musical phrases consistently, as the ability to do so relies on the MDRNN consistently providing the next note in a repeating phrase as an option.

5.3.3 Improving User Interactivity and Control

IMPSY was originally designed to focus on user interaction just as much if not more than predictive ability ([Martin and Tørresen, 2019](#)), but this thesis focuses entirely on improving the predictive ability of the system. Integrating these improvements back into a more user controllable design would be a useful avenue of research. The idea of user trained heuristics is one example of this which has already been discussed, but there are many more possibilities.

A UI could be created for the system to allow the user to modify the optimal values of musical parameters which the search heuristics will aim to match. For example, during system performance the user could manually set the preferred key signature, which would reward the system if it modulates to this key signature. This would allow user control of output, even when they are not actively performing, and as was briefly covered in the impact section could even be implemented into entirely generative musical systems as a way to add an element of interactivity. Users could also disable or adjust the weighting of heuristic components during performance.

MCTS-IMPSY could also provide future research related to live-looping, which is a type of performance where usually a single person performs live with multiple layered instruments through the use of looping pedals and other techniques which control multiple instruments simultaneously, and has been a popular area of NIME research ([Barbosa et al., 2017](#)). By modifying the current system to allow for multiple instruments to play at once, each sharing some heuristic parameters (such as rhythm), and with heuristic components which are purposefully restrictive to ensure adherence to previously played parameters, a type of semi-generative live-looping system could be created. Instead of

standard live-looping where after stopping playing one instrument a loop is triggered, the performer could instead switch over to using IMPSY as a performer, which will keep to the same parameters the performer has set, but will not exactly loop and will add its own variations to the performance. Hence, a single performer could create an improvisation with many instruments at once using MCTS-IMPSY without resorting to precise repetition or other live-looping techniques.

Conclusion

This thesis presents MCTS-IMPSY, a novel prediction architecture for interactive musical prediction, which combines the strengths of both machine learning and rule-based approaches to generate coherent and expressive musical responses in real-time. By integrating Monte Carlo Tree Search (MCTS) with a Mixture Density Recurrent Neural Network (MDRNN) to define its action space, MCTS-IMPSY combines the strengths of heuristic based music generation in structural and musical cohesive with the strengths of neural network based systems in creative and expressive range, while mitigating the inverse weaknesses of each approach when used individually. MCTS replaced the stochastic sampling of probabilistic MDRNN outputs with an intelligent and guided selection process based on a set of musical parameter measurements which combine to form a heuristic. This is the first time that this approach to prediction has been applied to an interactive musical system. To conclude this thesis, a short summary of the key findings is provided, and then answers to the three research questions posed in the introduction will be given.

6.1 Summary of Key Findings

The evaluation, conducted using both quantitative and qualitative metrics across different datasets, demonstrates the effectiveness of an MCTS-enhanced approach. Evaluations of MCTS-IMPSY against the previous MDRNN-IMPSY system show a 67.5% and 145.2% improvement in prediction accuracy for each of the two datasets used. Qualitative analysis of improvisations performed with the system show that this increase did not come at the cost of expressiveness, with the playability and practical usefulness of the system being significantly improved with the new prediction architecture, particularly for more structured improvisations. This work represents a significant proof of concept for human-computer interaction as a whole, showcasing the potential of combining search algorithms with machine learning in interactive artistic systems such as NIMES.

This research serves as a proof of concept for similar prediction architectures to be used in various fields. The customizability and generalisability of heuristic components in MCTS-IMPSY mean that the system can be adapted to various musical styles, instruments, and performance goals, offering a flexible foundation for future developments in interactive artistic systems. The implications extend beyond music, with potential use cases in generative music systems, gestural control, networked instruments, and even other creative domains. This thesis provides a solution to the common problem of balancing structure and coherence with creativity in AI-based prediction systems, with is a problem present in all of these fields.

Because of the function of this research in creating a generalisable and easily customisable prediction framework, there are a multitude of avenues for future work. User interfaces could be developed which allow users to adjust the parameters controlling music generation in real-time, user-generated parameters could be used as heuristics to guide search, the system presented in this thesis could be applied to other fields, and much more.

6.2 Answers to the Research Questions

In the introduction of this thesis, three research questions were posed, which will now be answered based on findings established earlier in this thesis.

1. What are the strengths and weaknesses of heuristic and machine learning based approaches in NIMEs, and how can they be effectively combined?

Machine learning-based approaches, particularly neural networks like MDRNNs, excel in capturing the small patterns and details in music which humans build their musical intuition around, allowing them to generate a wide range of creative and expressive musical ideas. However, they often lack inherent mechanisms for ensuring structural or musical coherence, sometimes leading to incoherent or unpredictable outputs. Heuristic-based approaches, conversely, are strong in providing structural coherence and adhering to specific musical rules, but they may lack the flexibility and expressive range of machine learning models.

These two approaches can be effectively combined by leveraging the strengths of each. This thesis demonstrates that by searching a tree of potential musical continuations, with a neural network to generate the action space and a rule-based heuristic to guide the search process, a system that explores the rich and expressive space of musical possibilities generated by machine learning methods is created, while still focusing on musical paths which are musically coherent and follow the established parameters of the piece.

2. How can an effective heuristic be designed for MCTS-IMPSY to intelligently guide the search process?

An effective heuristic for MCTS-IMPSY is designed to mimic a human performer's real-time checks and balances during improvisation, ensuring that musically intuitive options

are also coherent. The heuristic is primarily based on musical parameters that psychological research and musical studies have shown to be relevant to human perception and prediction of music. The algorithm is designed to be flexible, allowing for customization of the exact heuristics based on the instrument, performance goals, or specific musical styles.

In this thesis, musical parameters that have been established as useful include key signature, tempo, and the Markov models describing the probability of certain changes in pitch and duration over successive notes. These parameters are first measured while the human performer plays, and when they stop, a set of important musical features are established based on the current state of the music. For example, this could be that the current key signature is C major and the tempo is 120 BPM. Each clearly established musical feature then forms a component of the heuristic, which for each explored musical continuation, assigns a score based on how well the continuation fits with the established musical features. For example if a continuation contains multiple notes which are not in C major, it would be assigned a lower score. The heuristic is designed to be flexible and customizable, allowing for the addition of new parameters or the adjustment of existing ones based on the specific needs of the performance or the instrument being used. This flexibility allows for a wide range of musical styles and performance goals to be accommodated within the MCTS-IMPSY framework.

3. Can MCTS-IMPSY improve on the prediction accuracy and playability of MDRNN-IMPSY, while maintaining creativity?

Yes, MCTS-IMPSY significantly improves upon the prediction accuracy and playability of MDRNN-IMPSY, while successfully maintaining creativity. The evaluation demonstrates that the MCTS-enhanced system leads to 67.5% increase in correct predictions for a dataset trained on my own improvisations, and a 145.2% increase for a more structured and larger dataset trained on over 1000 folk melodies. Qualitatively, a structured improvisation test showed that MCTS-IMPSY increased playability due to more coherent outputs improving the ability to have meaningful call-and-response interactions. In freeform improvisations, the improvement was less pronounced due to the reliance of some heuristic components on specific structures like tempo and key signature. However, MCTS-IMPSY was preferred to MDRNN-IMPSY in this case too. Crucially, these improvements in prediction accuracy and coherence did not come at the expense of expressiveness and creativity. Instead, by refining musically incoherent predictions, the system in a sense fostered creativity by blocking non-sensical musical ideas in favour of creative expression which remained reasonable and coherent.

Bibliography

- AHLBÄCK, S., 2004. *Melody beyond notes: A study of melody cognition*. Ph.D. thesis, University of Gothenburg. <https://www.diva-portal.org/smash/record.jsf?pid=diva2:1366565>. [Cited on page 50.]
- ALLAN, M. AND WILLIAMS, C. K. I., 2004. Harmonising chorales by probabilistic inference. In *Proceedings of the 17th International Conference on Neural Information Processing Systems (NIPS 2004)*, 25–32. MIT Press. https://proceedings.neurips.cc/paper_files/paper/2004/file/b628386c9b92481fab68fbf284bd6a64-Paper.pdf. [Cited on page 12.]
- ANDERS, T. AND MIRANDA, E. R., 2011. Constraint programming systems for modelling music theories and composition. *ACM Computing Surveys*, 43, 4 (2011). doi: 10.1145/1978802.1978809. [Cited on pages 2 and 20.]
- BARBOSA, J.; MALLOCH, J. W.; WANDERLEY, M. M.; AND HUOT, S., 2015. What does ‘evaluation’ mean for the NIME community? In *Proceedings of the 15th International Conference on New Interfaces for Musical Expression (NIME 2015)*, 156–161. Zenodo. doi:10.5281/ZENODO.1179010. [Cited on page 55.]
- BARBOSA, J.; WANDERLEY, M. M.; AND HUOT, S., 2017. Exploring playfulness in NIME design: The case of live looping tools. In *Proceedings of the 17th International Conference on New Interfaces for Musical Expression (NIME 2017)*, 87–92. Zenodo. doi:10.5281/zenodo.1176181. [Cited on page 80.]
- BENGIO, Y.; SIMARD, P. Y.; AND FRASCONI, P., 1994. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5, 2 (1994), 157–166. doi:10.1109/72.279181. [Cited on page 15.]
- BISHOP, C. M., 1994. Mixture density networks. Technical Report NCRG/97/004, Neural Computing Research Group, Aston University. [Cited on page 18.]
- BISIANI, R., 1992. Beam search. In *Encyclopedia of Artificial Intelligence*, vol. 2, 1467–1468. John Wiley and Sons, 2 edn. [Cited on page 22.]

Bibliography

- BROWN, N. AND SANDHOLM, T., 2019. Superhuman AI for multiplayer poker. *Science*, 365, 6456 (2019), 885–890. doi:10.1126/science.aay2400. [Cited on page 26.]
- BROWNE, C.; POWLEY, E. J.; WHITEHOUSE, D.; LUCAS, S. M.; COWLING, P. I.; ROHLFSHAGEN, P.; TAVENER, S.; LIEBANA, D. P.; SAMOTHRAKIS, S.; AND COLTON, S., 2012. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4, 1 (2012), 1–43. doi:10.1109/TCIAIG.2012.2186810. [Cited on page 24.]
- BRÜGMANN, B., 1993. Monte carlo go. Technical report, Physics Department, Syracuse University. [Cited on page 24.]
- CHEN, J.; ZHANG, C.; LUO, J.; XIE, J.; AND WAN, Y., 2020. Driving maneuvers prediction based autonomous driving control by deep monte carlo tree search. *IEEE Transactions on Vehicular Technology*, 69, 7 (2020), 7146–7158. doi:10.1109/TVT.2020.2991584. [Cited on page 27.]
- COLLINS, N., 2009. Musical form and algorithmic composition. *Contemporary Music Review*, 28, 11 (2009), 103–114. doi:10.1080/07494460802664064. [Cited on page 80.]
- COLLINS, N. AND D’ESCRIVÁN, J., 2012. *The Cambridge Companion to Electronic Music*. Cambridge University Press. doi:10.1017/CCOL9780521868617. [Cited on page 8.]
- COLLINS, N. M., 2007. *Towards autonomous agents for live computer music : realtime machine listening and interactive music systems*. Ph.D. thesis, University of Cambridge. <https://api.semanticscholar.org/CorpusID:26846600>. [Cited on pages 10 and 74.]
- COULOM, R., 2006. Efficient selectivity and backup operators in monte-carlo tree search. In *Proceedings of the 5th International Conference on Computers and Games (CG 2006)*, 72–83. Springer International Publishing. doi:10.1007/978-3-540-75538-8_7. [Cited on pages 24 and 76.]
- DANNENBERG, R. B.; THOM, B.; AND WATSON, D., 1997. A machine learning approach to musical style recognition. In *Proceedings of the 23rd International Computer Music Conference (ICMC 1997)*. Michigan Publishing. <https://hdl.handle.net/2027/spo.bbp2372.1997.090>. [Cited on page 15.]
- DESAIN, P. AND HONING, H., 2002. Rhythmic stability as explanation of category size. In *Proceedings of the 7th International Conference on Music Perception & Cognition (ICMPC 2002)*, 111–115. Architecture, Media, Politics, Society and Causal Productions. <https://hdl.handle.net/11245/1.206041>. [Cited on page 10.]
- DOLSON, M., 1989. Machine tongues XII: Neural networks. *Computer Music Journal*, 13, 3 (1989), 28–40. [Cited on page 14.]

- DOWLING, W. J.; BARBEY, A.; AND ADAMS, L., 1999. Melodic and rhythmic contour in perception and memory. *Music, Mind, and Science*, (1999), 166–188. [Cited on page 47.]
- DOWLING, W. J. AND FUJITANI, D. S., 1971. Contour, interval, and pitch recognition in memory for melodies. *Journal of the Acoustical Society of America*, 49, 2B (1971), 524–531. doi:10.1121/1.1912382. [Cited on pages 45 and 47.]
- DUANE, B., 2016. Repetition and prominence: The probabilistic structure of melodic and non-melodic lines. *Music Perception*, 34, 2 (2016), 152–166. doi:10.1525/mp.2016.34.2.152. [Cited on page 48.]
- EBCIOĞLU, K., 1988. An expert system for harmonizing four-part chorales. *Computer Music Journal*, 12, 3 (1988), 43–51. doi:10.2307/3680335. [Cited on page 20.]
- ECK, D. AND SCHMIDHUBER, J., 2002. Finding temporal structure in music: blues improvisation with LSTM recurrent networks. In *Proceedings of the 12th IEEE Workshop on Neural Networks for Signal Processing (NNSP 2002)*, 747–756. IEEE. doi:10.1109/NNSP.2002.1030094. [Cited on page 17.]
- EITAN, Z. AND GRANOT, R. Y., 2006. How music moves: Musical parameters and listeners’ images of motion. *Music Perception: An Interdisciplinary Journal*, 23, 3 (2006), 221–248. doi:10.1525/mp.2006.23.3.221. [Cited on page 50.]
- FERREIRA, L. N.; MOU, L.; WHITEHEAD, J.; AND LELIS, L. H. S., 2022. Controlling perceived emotion in symbolic music generation with monte carlo tree search. In *Proceedings of the 18th AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE 2022)*, vol. 25, 3763–3772. AAAI Press. doi:10.1109/TMM.2022.3165718. [Cited on pages 21 and 22.]
- FIEBRINK, R.; TRUEMAN, D.; AND COOK, P. R., 2009. A meta-instrument for interactive, on-the-fly machine learning. In *Proceedings of the 9th International Conference on New Interfaces for Musical Expression (NIME 2009)*, 280–285. Zenodo. doi:10.5281/ZENODO.1177513. [Cited on page 9.]
- FOXLEY, E. The nottingham music database. Original source no longer available online. [Cited on page 57.]
- FU, X.; DENG, H.; YUAN, X.; AND HU, J., 2023. Generating high coherence monophonic music using monte-carlo tree search. *IEEE Transactions on Multimedia*, 25 (2023), 3763–3772. doi:10.1109/TMM.2022.3165718. [Cited on pages 2, 21, and 22.]
- GAULDIN, R., 2013. *A Practical Approach to 18th Century Counterpoint: Revised Edition*. Waveland Press. [Cited on page 50.]
- GRAVES, A., 2013. Generating sequences with recurrent neural networks. *ArXiv e-prints*, 1308.0850 (2013). doi:10.48550/arXiv.1308.0850. [Cited on page 19.]

Bibliography

- HA, D. AND ECK, D., 2017. A neural representation of sketch drawings. *ArXiv e-prints*, 1704.03477 (2017). doi:10.48550/arXiv.1704.03477. [Cited on page 19.]
- HADJERES, G.; PACHET, F.; AND NIELSEN, F., 2017. DeepBach: a steerable model for bach chorales generation. In *Proceedings of the 34th International Conference on Machine Learning (ICML 2017)*, vol. 70, 1362–1371. Proceedings of Machine Learning Research. doi:10.48550/arXiv.1612.01010. [Cited on pages 18 and 74.]
- HAILSTONE, J. C.; OMAR, R.; HENLEY, S. M. D.; FROST, C.; KENWARD, M. G.; AND WARREN, J. D., 2009. It’s not what you play, it’s how you play it: timbre affects perception of emotion in music. *The Quarterly Journal of Experimental Psychology*, 62, 11 (2009), 2141–2155. doi:10.1080/17470210902765957. [Cited on page 50.]
- HILLER, L. A. AND ISAACSON, L. M., 1959. *Experimental Music: Composition With an Electronic Computer*. McGraw-Hill, 2nd edn. Reprinted, Westport, Conn.: Greenwood Press, 1979. [Cited on page 20.]
- HOCHREITER, S. AND SCHMIDHUBER, J., 1997. Long short-term memory. *Neural Computation*, 9, 8 (1997), 1735–1780. doi:10.1162/NECO.1997.9.8.1735. [Cited on page 16.]
- HOKAMP, C. AND LIU, Q., 2017. Lexically constrained decoding for sequence generation using grid beam search. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL 2017)*, 1535–1546. Association for Computational Linguistics. doi:10.18653/v1/P17-1141. [Cited on page 26.]
- HUNT, A.; WANDERLEY, M. M.; AND KIRK, R., 2000. Towards a model for instrumental mapping in expert musical interaction. In *Proceedings of the 26th International Computer Music Conference (ICMC 2000)*. Michigan Publishing. <https://hdl.handle.net/2027/spo.bbp2372.2000.173>. [Cited on page 8.]
- HURON, D., 2008. *Sweet anticipation: Music and the psychology of expectation*. MIT press. doi:<https://doi.org/10.7551/mitpress/6575.001.0001>. [Cited on page 38.]
- JI, S.; YANG, X.; AND LUO, J., 2024. A survey on deep learning for symbolic music generation: Representations, algorithms, evaluations, and challenges. *ACM Computing Surveys*, 56, 1 (2024). doi:10.1145/3597493. [Cited on pages 16 and 18.]
- JOHNSON, D. D., 2017. Generating polyphonic music using tied parallel networks. In *Proceedings of the 6th International Conference on Computational Intelligence in Music, Sound, Art and Design (EvoMUSART 2017)*, 128–143. Springer International Publishing. doi:10.1007/978-3-319-55750-2_9. [Cited on page 18.]
- JUKEDECK, 2017. Nottingham dataset. <https://github.com/jukedeck/nottingham-dataset>. GitHub repository containing cleaned MIDI versions of the original Nottingham Music Database. [Cited on page 57.]

- KARKUS, P.; HSU, D.; AND LEE, W. S., 2017. QMDP-net: Deep learning for planning under partial observability. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS 2017)*, 4705–4714. MIT Press. doi: 10.48550/arXiv.1703.06692. [Cited on page 27.]
- KHATRI, N.; LEE, S.; AND NAM, S. Y., 2023. Transfer learning-based intrusion detection system for a controller area network. *IEEE Access*, 11 (2023). doi: 10.1109/ACCESS.2023.3328182. [Cited on page 17.]
- KIEFER, C.; COLLINS, N.; AND FITZPATRICK, G., 2008. HCI methodology for evaluating musical controllers: A case study. In *Proceedings of the 8th International Conference on New Interfaces for Musical Expression (NIME 2008)*, 87–90. Zenodo. doi:10.5281/zenodo.1179577. [Cited on page 8.]
- KOCSIS, L. AND SZEPESVÁRI, C., 2006. Bandit based monte-carlo planning. In *Proceedings of the 17th European Conference on Machine Learning (ECML 2006)*, 282–293. Springer International Publishing. doi:10.1007/11871842_29. [Cited on pages 23, 24, and 33.]
- LADEN, B. AND KEEFE, D. H., 1989. The representation of pitch in a neural net model of chord classification. *Computer Music Journal*, 13, 4 (1989), 12–26. doi: 10.2307/3679550. [Cited on page 14.]
- LAROCHE, J., 2001. Estimating tempo, swing and beat locations in audio recordings. In *Proceedings of the 8th IEEE Workshop on the Applications of Signal Processing to Audio and Acoustics (WASPAA 2001)*, 135–138. IEEE Press. doi:10.1109/ASPAA.2001.969561. [Cited on pages 44, 50, and 76.]
- LAWSON, J. AND MATHEWS, M. V., 1977. Computer program to control a digital real-time sound synthesizer. *Computer Music Journal*, 1, 4 (1977), 16–21. [Cited on page 8.]
- LECUN, Y.; BENGIO, Y.; AND HINTON, G., 2015. Deep learning. *Nature*, 521, 7553 (2015), 436–444. doi:10.1038/nature14539. [Cited on page 15.]
- LEI, L.; LUO, R.; ZHENG, R.; WANG, J.; ZHANG, J.; QIU, C.; MA, L.; JIN, L.; ZHANG, P.; AND CHEN, J., 2021. KB-Tree: Learnable and continuous monte-carlo tree search for autonomous driving planning. In *Proceedings of the 34th International Conference on Intelligent Robots and Systems (IROS 2021)*, 4493–4500. IEEE Press. doi:10.1109/IROS51168.2021.9636442. [Cited on pages 4, 25, 27, 35, 75, and 79.]
- LOWERRE, B. T., 1976. *The harpy speech recognition system*. Ph.D. thesis, Carnegie Mellon University. [Cited on page 22.]
- LU, L.; LIU, D.; AND ZHANG, H., 2006. Automatic mood detection and tracking of music audio signals. *IEEE Transactions on Audio, Speech, and Language Processing*, 14, 1 (2006), 5–18. doi:10.1109/TSA.2005.860344. [Cited on page 75.]

Bibliography

- MACHOVER, T. AND CHUNG, J. T., 1989. Hyperinstruments: Musically intelligent and interactive performance and creativity systems. In *Proceedings of the 15th International Computer Music Conference (ICMC 1989)*. Michigan Publishing. <https://hdl.handle.net/2027/spo.bbp2372.1989.046>. [Cited on pages 9 and 74.]
- MAO, H. H.; SHIN, T.; AND COTTRELL, G. W., 2018. DeepJ: Style-specific music generation. In *Proceedings of the 12th IEEE International Conference on Semantic Computing (ICSC 2018)*, 377–382. IEEE Computer Society. doi:10.1109/ICSC.2018.00077. [Cited on page 18.]
- MARTIN, C. P. AND TØRRESEN, J., 2018. RoboJam: A musical mixture density network for collaborative touchscreen interaction. In *Proceedings of the 7th International Conference on Computational Intelligence in Music, Sound, Art and Design (EvoMUSART 2018)*, 161–176. Springer International Publishing. doi:10.1007/978-3-319-77583-8_11. [Cited on pages 2 and 19.]
- MARTIN, C. P. AND TØRRESEN, J., 2019. An interactive musical prediction system with mixture density recurrent neural networks. In *Proceedings of the 19th International Conference on New Interfaces for Musical Expression (NIME 2019)*, 260–265. Zenodo. doi:10.5281/ZENODO.3672952. [Cited on pages 2, 3, 9, 10, 19, 22, 28, 31, 32, 55, 73, and 80.]
- MATHEWS, M. V., 1991. The radio baton and conductor program, or: Pitch, the most important and least expressive part of music. *Computer Music Journal*, 15, 4 (1991), 37–46. doi:10.2307/3681070. [Cited on page 8.]
- MCPHERSON, A., 2010. The magnetic resonator piano: Electronic augmentation of an acoustic grand piano. *Journal of New Music Research*, 39, 3 (2010), 189–202. doi:10.1080/09298211003695587. [Cited on page 9.]
- MERN, J.; YILDIZ, A.; SUNBERG, Z.; MUKERJI, T.; AND KOCHENDERFER, M. J., 2021. Bayesian optimized monte carlo planning. In *Proceedings of the 35th AAAI Conference on Artificial Intelligence (AAAI 2021)*, 11880–11887. AAAI Press. doi:10.1609/AAAI.V35I13.17411. [Cited on pages 25, 36, and 79.]
- METROPOLIS, N. AND ULAM, S., 1949. The monte carlo method. *Journal of the American Statistical Association*, 44, 247 (1949), 335–341. doi:10.2307/2280232. [Cited on page 24.]
- MOORER, J. A., 1972. Music and computer composition. *Communications of the Association for Computing Machinery*, 15, 2 (1972), 104–113. doi:10.1145/361254.361265. [Cited on page 20.]
- MOZER, M. C., 1994. Neural network music composition by prediction: Exploring the benefits of psychoacoustic constraints and multi-scale processing. *Connection Science*, 6 (1994), 247–280. doi:10.1080/09540099408915726. [Cited on pages 15 and 73.]

- PACHET, F., 2002. The continuator: Musical interaction with style. In *Proceedings of the 28th International Computer Music Conference (ICMC 2002)*. Michigan Publishing. <https://hdl.handle.net/2027/spo.bbp2372.2002.044>. [Cited on pages 10 and 73.]
- PUCKETTE, M. S., 1997. Pure data. In *Proceedings of the 23rd International Computer Music Conference (ICMC 1997)*. Michigan Publishing. <https://hdl.handle.net/2027/spo.bbp2372.1997.060>. [Cited on page 8.]
- PUCKETTE, M. S., 2002. Max at seventeen. *Computer Music Journal*, 26, 4 (2002), 31–43. doi:10.1162/014892602320991356. [Cited on page 8.]
- RENAUD, A.; CARÔT, A.; AND REBELO, P., 2007. Networked music performance: State of the art. In *Proceedings of the 30th AES International Conference on Intelligent Audio Environments (AES 2007)*, 16–22. Audio Engineering Society. <https://aes2.org/publications/elibrary-page/?id=13914>. [Cited on page 10.]
- RON, D.; SINGER, Y.; AND TISHBY, N., 1996. The power of amnesia: Learning probabilistic automata with variable memory length. *Machine Learning*, 25 (1996), 117–149. doi:10.1023/A:1026490906255. [Cited on page 12.]
- SCHEIRER, E. D., 1998. Tempo and beat analysis of acoustic musical signals. *The Journal of the Acoustical Society of America*, 103, 1 (1998), 588–601. doi:10.1121/1.421129. [Cited on page 76.]
- SCHMUCKLER, M. A. AND MORANIS, R., 2023. Rhythm contour drives musical memory. *Attention, Perception, & Psychophysics*, 85, 7 (2023), 2502–2514. doi:10.3758/s13414-023-02700-w. [Cited on page 47.]
- SILVER, D.; HUANG, A.; MADDISON, C. J.; GUEZ, A.; SIFRE, L.; VAN DEN DRIESSCHE, G.; SCHRITTWIESER, J.; ANTONOGLOU, I.; PANNEERSHELVAM, V.; LANCTOT, M.; DIELEMAN, S.; AND JOHN NHAM, D. G.; KALCHBRENNER, N.; SUTSKEVER, I.; LILICRAP, T. P.; LEACH, M.; KAVUKCUOGLU, K.; GRAEPEL, T.; AND HASSABIS, D., 2016. Mastering the game of go with deep neural networks and tree search. *Nature*, 529 (2016), 484–489. doi:10.1038/nature16961. [Cited on pages 26, 27, and 51.]
- SILVER, D.; HUBERT, T.; SCHRITTWIESER, J.; ANTONOGLOU, I.; LAI, M.; GUEZ, A.; LANCTOT, M.; SIFRE, L.; KUMARAN, D.; GRAEPEL, T.; LILICRAP, T.; SIMONYAN, K.; AND HASSABIS, D., 2018. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362, 6419 (2018), 1140–1144. doi:10.1126/science.aar6404. [Cited on page 26.]
- SILVER, D.; SCHRITTWIESER, J.; SIMONYAN, K.; ANTONOGLOU, I.; HUANG, A.; GUEZ, A.; HUBERT, T.; BAKER, L.; LAI, M.; BOLTON, A.; CHEN, Y.; LILICRAP, T. P.; HUI, F.; SIFRE, L.; VAN DEN DRIESSCHE, G.; GRAEPEL, T.; AND HASSABIS, D., 2017. Mastering the game of go without human knowledge. *Nature*, 550 (2017), 354–359. doi:10.1038/nature24270. [Cited on page 26.]

Bibliography

- SIMON, I.; MORRIS, D.; AND BASU, S., 2008. MySong: automatic accompaniment generation for vocal melodies. In *Proceedings of the 28th Conference on Human Factors in Computing Systems (CHI 2008)*, 725–734. Association for Computing Machinery. doi:10.1145/1357054.1357169. [Cited on page 13.]
- STEEDMAN, M. J., 1984. A generative grammar for jazz chord sequences. *Music Perception: An Interdisciplinary Journal*, 2, 1 (1984), 52–77. doi:10.2307/40285282. [Cited on page 20.]
- STEINBISS, V.; TRAN, B.; AND NEY, H., 1994. Improvements in beam search. In *Proceedings of the 3rd International Conference on Spoken Language Processing (ICSLP 1994)*, 2143–2146. International Science Community Association. doi:10.21437/ICSLP.1994-538. [Cited on page 23.]
- SUTSKEVER, I.; VINYALS, O.; AND LE, Q. V., 2014. Sequence to sequence learning with neural networks. In *Proceedings of the 28th International Conference on Neural Information Processing Systems (NIPS 2014)*, vol. 2, 3104–3112. MIT Press. [dl.acm.org/doi/10.5555/2969033.2969173](https://doi.org/10.5555/2969033.2969173). [Cited on pages 23 and 26.]
- TAKASE, H. AND SHIRAMATSU, S., 2020. Support system for improvisational ensemble based on long short-term memory using smartphone sensor. In *Proceedings of the 20th International Conference on New Interfaces for Musical Expression (NIME 2020)*, 394–398. Zenodo. doi:10.5281/ZENODO.4813434. [Cited on page 8.]
- TEMPERLEY, D., 2008. A probabilistic model of melody perception. *Cognitive Science*, 32, 2 (2008), 418–444. doi:10.1080/03640210701864089. [Cited on pages 40, 45, 46, and 50.]
- TODD, P. M., 1989. A connectionist approach to algorithmic composition. *Computer Music Journal*, 13, 4 (1989), 27–43. doi:10.2307/3679551. [Cited on page 15.]
- WAGSTAFF, K., 2012. Machine learning that matters. In *Proceedings of the 29th International Conference on Machine Learning (ICML 2012)*, 1851–1856. Omnipress. doi:10.48550/arXiv.1206.4656. [Cited on page 67.]
- WALL, L.; LIECK, R.; NEUWIRTH, M.; AND ROHRMEIER, M., 2020. The impact of voice leading and harmony on musical expectancy. *Scientific Reports*, 10, 1 (2020). doi:10.1038/s41598-020-61645-4. [Cited on page 50.]
- WANG, G.; BRYAN, N. J.; OH, J.; AND HAMILTON, R., 2009. Stanford laptop orchestra (SLOrk). In *Proceedings of the 35th International Computer Music Conference (ICMC 2009)*. Michigan Publishing. <https://hdl.handle.net/2027/spo.bbp2372.2009.114>. [Cited on page 10.]
- WEINBERG, G.; BLOSSER, B.; MALLIKARJUNA, T.; AND RAMAN, A., 2009. The creation of a multi-human, multi-robot interactive jam session. In *Proceedings of the 9th International Conference on New Interfaces for Musical Expression (NIME 2009)*, 70–73. Zenodo. doi:10.5281/ZENODO.1177705. [Cited on pages 10 and 75.]

- WEINGERTNER, P.; HO, M.; TIMOFEEV, A.; AUBERT, S.; AND PITA-GIL, G., 2020. Monte carlo tree search with reinforcement learning for motion planning. In *Proceedings of the 23rd International Conference on Intelligent Transportation Systems (ITSC 2020)*, 1–7. IEEE Press. doi:10.1109/ITSC45102.2020.9294697. [Cited on page 27.]
- WHITE, B. W., 1960. Recognition of distorted melodies. *The American Journal of Psychology*, 73, 1 (1960), 100–107. doi:https://doi.org/10.2307/1419120. [Cited on page 45.]
- WU, S.; DONAHUE, C.; WATANABE, S.; AND BRYAN, N. J., 2024. Music controlnet: Multiple time-varying controls for music generation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 32 (2024), 2692–2703. doi:10.1109/TASLP.2024.3399026. [Cited on page 9.]
- WU, Y.; SCHUSTER, M.; CHEN, Z.; LE, Q. V.; NOROUZI, M.; MACHEREY, W.; KRIKUN, M.; CAO, Y.; GAO, Q.; MACHEREY, K.; KLINGNER, J.; SHAH, A.; JOHNSON, M.; LIU, X.; KAISER, L.; GOUWS, S.; KATO, Y.; KUDO, T.; KAZAWA, H.; STEVENS, K.; KURIAN, G.; PATIL, N.; WANG, W.; YOUNG, C.; SMITH, J.; RIESA, J.; RUDNICK, A.; VINYALS, O.; CORRADO, G.; HUGHES, M.; AND DEAN, J., 2016. Google’s neural machine translation system: Bridging the gap between human and machine translation. *ArXiv e-prints*, 1609.08144 (2016). doi:10.48550/arXiv.1609.08144. [Cited on page 26.]
- XU, Y., 2024. Multi-robot source navigation method based on coordination graph monte carlo tree search. *Journal of Electrical Systems*, 20 (2024), 421–431. doi:10.52783/jes.1195. [Cited on page 24.]
- YEE, T.; LISÝ, V.; AND BOWLING, M. H., 2016. Monte carlo tree search in continuous action spaces with execution uncertainty. In *Proceedings of the 25th International Joint Conference on Artificial Intelligence (IJCAI 2016)*, 690–697. AAAI Press. [Cited on page 25.]
- ZHANG, H.; ZHOU, H.; WEI, Y.; AND HUANG, C., 2022. Autonomous maneuver decision-making method based on reinforcement learning and monte carlo tree search. *Frontiers in Neurorobotics*, 16, 996412 (2022). doi:10.3389/fnbot.2022.996412. [Cited on page 4.]
- ZHANG, Z., 2012. Microsoft kinect sensor and its effect. *IEEE Multimedia*, 19, 2 (2012), 4–10. doi:10.1109/MMUL.2012.24. [Cited on page 8.]
- ZICARELLI, D., 1987. M and jam factory. *Computer Music Journal*, 11, 4 (1987), 13–29. doi:10.2307/3680237. [Cited on page 12.]